

思想领袖的视角：力量与责任（A Thought Leader's Perspective: Power and Responsibility）

代理化系统把我们带入了一个新的时代：系统不仅“回答问题”，还会设定子目标、执行操作、与人和其他系统协作，以达成总体目标。能力越强，影响越广，随之而来的便是更重的责任。这不是一段“可选项”的附言，它应当进入设计与落地的每一处细节。本篇从领导者视角，讨论在推动代理化系统时，如何平衡“力量”与“责任”。

为什么是“力量与责任的二律背反”

智能体具备的能力——自主性、长期记忆、链式推理、外部工具访问——让它们能在生产与社会系统中产生复合效应。推进效率、放大创造力、重塑协作流程，这些都是“力量”的正面面。与此同时，风险也在指数级放大：目标错配、幻觉与错误执行、隐私与安全问题、偏见与不公平、责任界定不清。这些问题不再是研究议题，而是上线之前必须被明确管理的工程问题。

领导者应坚持的九项原则

- 以目标为先：用清晰的“业务与人类价值”定义系统目标，并用可衡量指标验证成果，而非仅以模型能力为驱动。
- 透明与可解释：为关键决策提供可追溯的推理链；记录数据来源、工具调用与重要状态变更。
- 人在环路与否决权：设计“人类监督位点”，在高风险场景设置审批与可逆操作；确保随时可暂停与回滚。
- 可信数据与审计：实行数据治理与版本管理，保留审计日志，建立事后问责机制与改进闭环。
- 安全、隐私与合规：最小权限原则、分层隔离、密钥与凭据管理；遵循相关法律与行业标准。
- 公平与包容：进行偏见评估与缓解，关注对弱势群体与边缘场景的影响，采用多元化评审。
- 可恢复与最小损害：默认“先安全后性能”，面向失败进行设计（降级、隔离、熔断与补偿）。
- 素养与教育：为使用者与开发者提供 AI 素养培训，明确能力边界与正确使用方式。
- 开放协作与标准：拥抱可互操作的接口与规范，参与社区与产业联盟，共建健康生态。

将原则落到工程与产品实践

- 需求到风险映射：在 PRD 中加入“风险条目”，为每个用户旅程识别潜在失效模式与缓解策略。
- 守护栏（Guardrails）工程化：输入输出过滤、策略检查、工具白名单、速率限制与预算控制，形成可配置策略层。
- 评估与监控：离线基准 + 在线观测；面向任务的成功率、恢复率、延迟与成本；异常告警与事后复盘。
- 人机协作界面：明确“何时由人决策、何时由机执行”，在 UI 中暴露状态、来源与理由，保留“撤销/回滚”。
- 变更与发布治理：灰度上线、影子模式、双轨评估；对提示、策略与工具变更采用审计化流程。
- 数据与隐私防护：分区存储、合规采集、最小化留存；对敏感数据全链路加密与访问控制。

面向组织的落地建议

- 设立跨职能“AI 治理委员会”，涵盖产品、工程、法务、隐私、合规与伦理。
- 制定“代理化系统设计规范”，将上述原则转化为模板与检查清单，纳入评审流程。
- 建立“事故响应与复盘”机制，形成事后问责与持续改进闭环。
- 推动内部教育与公开分享，提升对代理化系统的共同理解与实践水平。

结语

代理化系统是一股强大的生产力与创造力的引擎。它们的价值不只在能力，更在于是否能以负责的方式落地，真正增进人的福祉、扩展人的可能。作为推动者与建设者，我们既要拥抱技术的锋芒，也要守住对社会的承诺。以“力量与责任”为双轮，才能驶向更长远的未来。

序言 (Foreword)

人工智能正处在一个引人入胜的拐点。我们正在从仅能处理信息的模型，走向能够推理、规划并采取行动以实现复杂目标（哪怕任务本身并不明确）的智能系统。本书所称的“代理化（agentic）系统”代表了 AI 的下一片前沿，而它们的研发让我们在 Google 感到兴奋与鼓舞。

《Agentic 设计模式：构建智能系统的实践指南》正逢其时，为这段旅程提供了指引。书中明确指出，作为这些智能体的“认知引擎”的大语言模型，需要借助结构化与审慎的设计加以驾驭。正如设计模式以通用语言与可复用方案重塑了软件工程一样，本书中的代理化模式将成为构建健壮、可扩展且可靠的智能系统的基础。

AI 的未来，将由能够构建这些智能系统的开发者的创造力与巧思所定义。《Agentic 设计模式》是不可或缺的资源，它不仅帮助我们理解代理化系统的“是什么”“为什么”，更通过实践示例阐明“如何做”。

我非常欣喜地看到这本书走进开发者社区。这些模式与原则必将加速创新且有影响力的 AI 应用的开发，并在未来多年塑造我们的世界。

— Saurabh Tiwary Google CloudAI 副总裁兼总经理 (VP & GM)

献词 (Dedication)

献给我的儿子 Bruno—— 在他两岁时，给我的生命带来了新的、璀璨的光。每当我思考那些将塑造“明天”的系统时，我首先想到的，始终是你们将要继承的世界。

献给我的儿子 Leonardo 与 Lorenzo，以及我的女儿 Aurora—— 看到你们成长为杰出的女性与男性，并共同打造更加美好的世界，我的内心充满自豪。

这本书讲述如何构建智能工具，但它献给一个更深的愿望：愿你们这一代能以智慧与同情心引导这些工具。只要我们学会让这些强大技术服务于人类、推动人类进步，你们的未来、我们的未来将无比光明。

— 致以我全部的爱

致谢 (Acknowledgment)

我谨向促成本书的众多个人与团队致以诚挚谢意。

首先，感谢 Google 坚守其使命，赋能 Googler，并尊重创新的机会。感谢 CTO 办公室给予我探索新领域的机会，坚守“实用的魔法 (practical magic)”这一使命，并能够不断适应涌现的新机遇。

特别感谢我们的副总裁 Will Grannis——感谢他对人的信任以及“服务型领导”的风格；感谢我的经理 John Abel，鼓励我持续推进工作并以其敏锐的英国洞察力给予出色指导。感谢 Antoine Larmanjat 在“LLM 与代码”方面的合作，感谢 Hann Hann Wang 关于智能体的讨论，感谢 Yingchao Huang 在时间序列方面的见解。感谢 Ashwin Ram 的领导、Massy Mascaro 的启发性工作、Jennifer Bennett 的技术专长、Brett Slatkin 的工程能力，以及 Eric Schen 带来的激发性讨论。也要向 OCTO 团队致谢，尤其是 Scott Penberthy。最后，向 Patricia Florissi 表达深深的感激，感谢她对“智能体的社会影响”的鼓舞性愿景。

我也非常感谢 Marco Argenti，就“智能体增强人类劳动力”的挑战性且鼓舞人心的愿景。并感谢 Jim Lanzone 与 Jordi Ribas，将“搜索世界”与“智能体世界”的关系不断推向更高标准。

特别感谢 Yingchao Huang——你是一位出色的 AI 工程师，前途无量；感谢 Hann Wang——你促使我在最初于 1994 年产生兴趣之后重新回到对智能体的关注；感谢 Lee Boonstra——你在提示工程方面的精彩工作。

同样感谢“5 Days of GenAI”团队，包括我们的副总裁 Alison Wagonfeld 对团队的信任，感谢 Anant Nawalgaria 的持续交付，以及 Paige Bailey 的“能做 (can-do)”态度与领导力。

我亦非常感谢 Mike Styer、Turan Bulmus 与 Kanchana Patlolla，帮助我在 Google I/O 2025 发布了三个智能体。感谢你们的卓越工作。

感谢 Demis Hassabis、Pushmeet Kohli 以及整个 GDM 团队，在研发 Gemini、AlphaFold、AlphaGo、AlphaGenome 等项目上的热忱投入，并为“以科学造福社会”的愿景做出贡献。特别感谢 Yossi Matias 对 Google Research 的领导，以及他持续提供的宝贵建议——我从你身上学到了很多。

特别感谢 Patti Maes——她在 90 年代开创了软件智能体的概念，并始终关注“计算机系统与数字设备如何增强人类、协助记忆、学习、决策、健康与福祉”等问题。你在 1991 年的愿景，今天已成为现实。

我也要感谢 Paul Drougas 以及 Springer 的出版团队，使这本书成为可能。

这个项目若没有大家的支持，根本不可能完成。所有功劳都属于你们，所有错误归我一人。

我将把全部版税捐赠给“救助儿童会 (Save the Children)”。

前言

欢迎来到《Agentic 设计模式：构建智能系统的实践指南》。纵观当下人工智能的版图，我们清晰地看到系统从简单、被动响应的程序，进化到能够理解上下文、做出决策，并与其环境及其它系统动态交互的复杂自治实体。这些就是智能体 (agent) 以及由其组成的 agentic 系统。

强大的大语言模型 (LLM) 为理解和生成类人文本与多媒体内容提供了前所未有的能力，成为许多智能体的“认知引擎”。然而，要将这些能力编排为能够稳定达成复杂目标的系统，仅靠一个强模型远远不够；还需要结构化的设计，以及对智能体如何“感知—计划—行动—交互”的深思熟虑。

可以把构建智能系统比作在一块“画布”上进行复杂的工程或创作。这里的画布并非视觉空间，而是支撑智能体存在与运行的基础设施与框架——它为你的应用提供环境与工具，是你构建智能应用的地基，负责状态管理、通信、工具接入与逻辑流转。

要在这块 agentic 画布上有效地构建系统，不能只是把组件堆在一起；需要理解一套经验证的技术——设计模式——来解决设计与实现智能体行为时的常见挑战。就像架构模式指导建筑施工、软件设计模式组织代码结构一

样，agentic 设计模式为你在所选画布上“让智能体栩栩如生”时遇到的重复性问题提供可复用的解决方案。

什么是 Agentic 系统？

从本质上说，agentic 系统是能够感知其环境（数字世界甚至可能是物理世界）、基于这些感知以及预设或习得的目标做出明智决策，并自主执行行动以达成目标的计算实体。与遵循僵化、逐步指令的传统软件不同，智能体具有一定的灵活性与主动性。

设想你需要一个系统来处理客户咨询。传统系统可能按照固定脚本进行。agentic 系统则能够体察客户问题的细微差别，访问知识库，与其他内部系统（如订单管理）交互，必要时提出澄清问题，并主动解决问题，甚至预判后续需求。这些智能体在你的应用“画布”上运行，利用其所能触达的服务与数据。

Agentic 系统通常具备以下特征：自治（无需持续人工监督即可行动）、前瞻性（为达成目标主动发起行动）、反应性（能有效应对环境变化）。它们以目标为中心持续推进任务；一个关键能力是“工具使用”，即与外部 API、数据库或服务交互，从而超越其直接画布的边界。它们具备记忆，能在多轮交互中保留信息，并能与用户、其它系统乃至其它智能体（位于同一或相连画布上）进行通信。

要有效实现这些特征，系统复杂性随之陡增：智能体如何在多步骤流程中维护状态？何时以及如何使用工具？不同智能体之间的通信如何管理？如何构建系统韧性以应对意外结果或错误？

为什么在智能体开发中需要模式？

正因这种复杂性，agentic 设计模式才不可或缺。它们并非死板的规则，而是面向常见设计与实现问题的“久经考验”的模板或蓝图。识别并应用这些模式，你就能获得增强系统结构、可维护性、可靠性与效率的解决方案，从而在你的画布上构建更高质量智能体。

应用设计模式有助于避免为诸如会话状态管理、工具选择与调用、记忆检索与更新、错误处理与恢复、任务分解与路由等基础问题反复“造轮子”。如今，一系列正在巩固的基本原则正在形成：诸如 RAG（检索增强生成）、Reflection（反思）、Routing（路由）、Memory（记忆）等模式正成为基础构件。本书邀请你对这些核心理念进行系统思考，它们为我们后续的构建提供坚实地基。我们需要针对这些基础模式进行必要的“反思时刻”。

所用框架简介

为给代码示例提供具体的“画布”（也请参见附录），本书主要采用三类常用的智能体开发框架：

- LangChain 及其具备有状态扩展的 LangGraph：用于把语言模型与其他组件灵活地串联起来，提供强健的画布以构建复杂的操作序列与有向图。
- Crew AI：面向多智能体系统的编排，强调角色与任务的结构化组织，是特别适合协作式智能体系统的画布。
- Google Agent Developer Kit（Google ADK）：为构建、评估与部署智能体提供工具与组件，常与 Google 的 AI 基础设施集成，是另一类有价值的画布。

这些框架代表了智能体开发画布的不同侧面，各有优势。通过跨工具的示例，你将更广泛地理解如何在不依赖具体技术环境的前提下应用这些模式。示例将清晰呈现每个模式的核心逻辑及其在框架画布上的实现，强调清晰与实用。

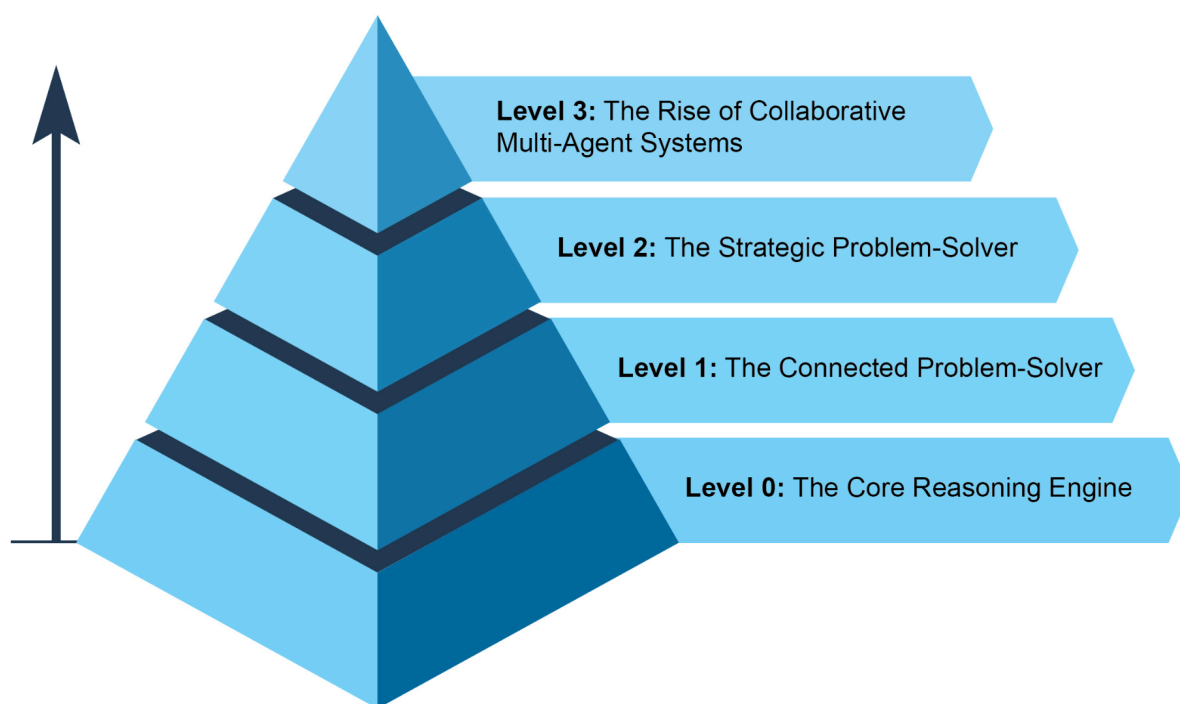
读完本书，你不仅能理解 21 个关键 agentic 模式背后的基本概念，还能掌握将其有效应用的实践知识与代码示例，从而在你所选的开发画布上构建更智能、更有能力、更具自治性的系统。现在，让我们开启这段动手之旅！

什么让一个 AI 系统成为“智能体”？（What makes an AI system an Agent?）

当我们说“智能体（Agent）”，并不是指某个单一的模型或框架，而是指一种具备目标导向行为、能与环境交互并能自主决策与行动的系统。它通常由大语言模型（LLM）等“认知引擎”驱动，同时辅以工具接口、记忆机制、规划模块与守护栏策略。以下是构成“智能体性”的关键元素：

核心能力

- 目标导向：能够从任务或自然语言中提取目标，并持续朝目标推进。
- 自主决策：在不逐条人工指令的情况下，做出下一步行动的合理选择。
- 环境交互：读取与写入外部世界（API、数据库、文件、设备），反馈驱动行为。
- 记忆与上下文：保留短期与长期记忆，形成跨会话与跨任务的持续性。
- 工具使用：选择、调用并组合外部工具完成复杂任务。
- 规划与分解：将总体目标拆分为可执行的子目标与步骤，动态调整计划。
- 反思与改进：在执行后进行评估，发现错误并自我纠正或重试。
- 学习与适应：基于经验更新策略或知识，提升后续表现。

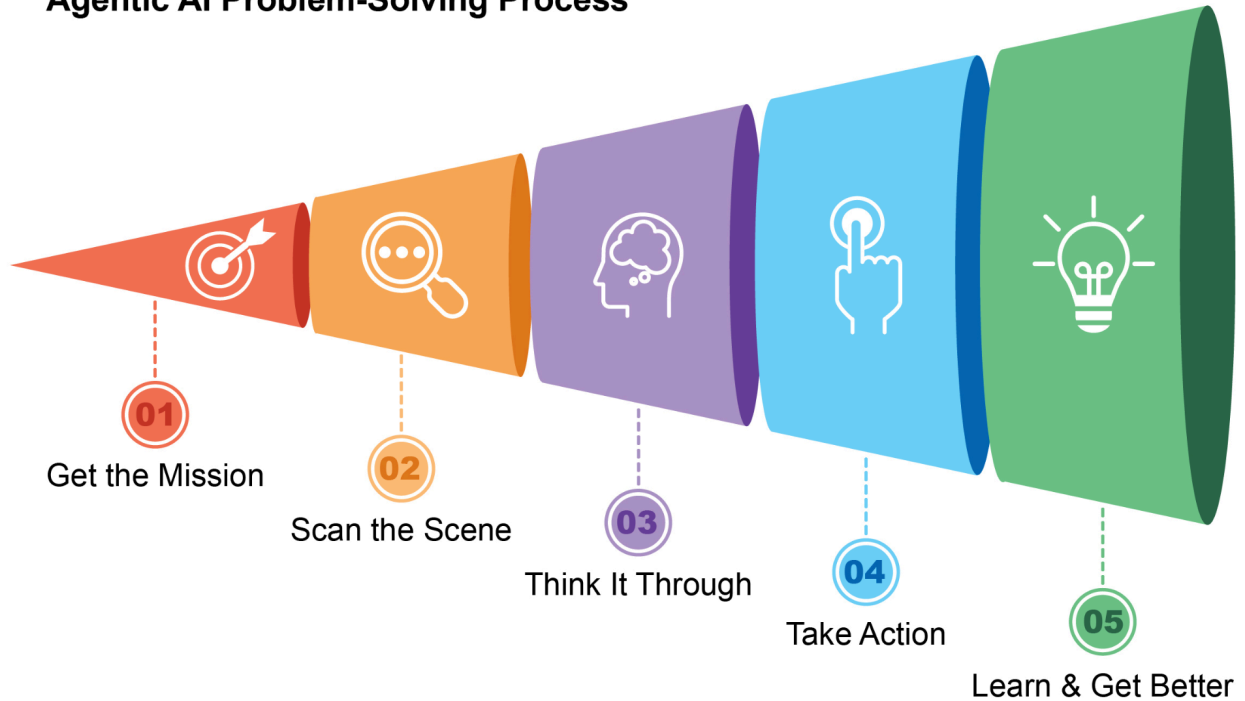


系统组成

- 认知引擎：以 LLM 为核心，承担理解、推理与生成。
- 行动执行层：工具路由、API 适配与环境操作。

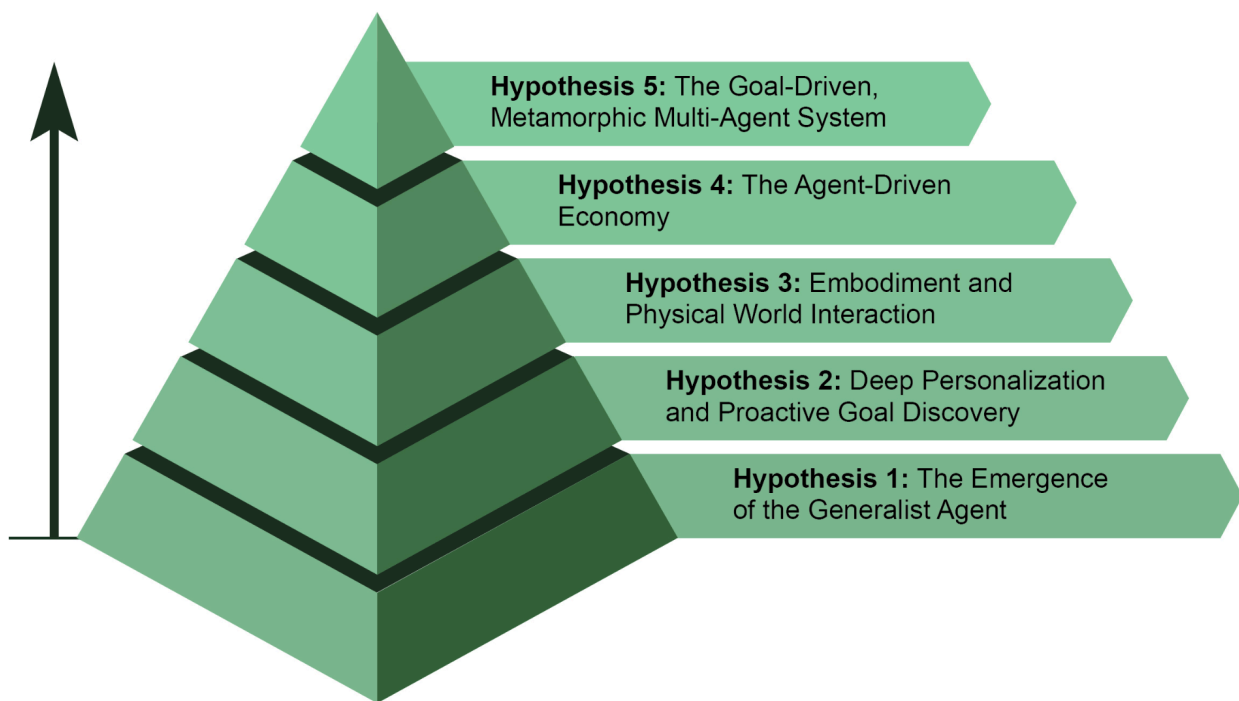
- 记忆层：短期工作记忆与长期知识库（RAG、向量库、日志）。
- 策略与守护栏：安全、隐私、合规与成本控制（白名单、预算、过滤与审批）。
- 评估与监控：面向任务的度量、异常告警与事后复盘。

Agentic AI Problem-Solving Process



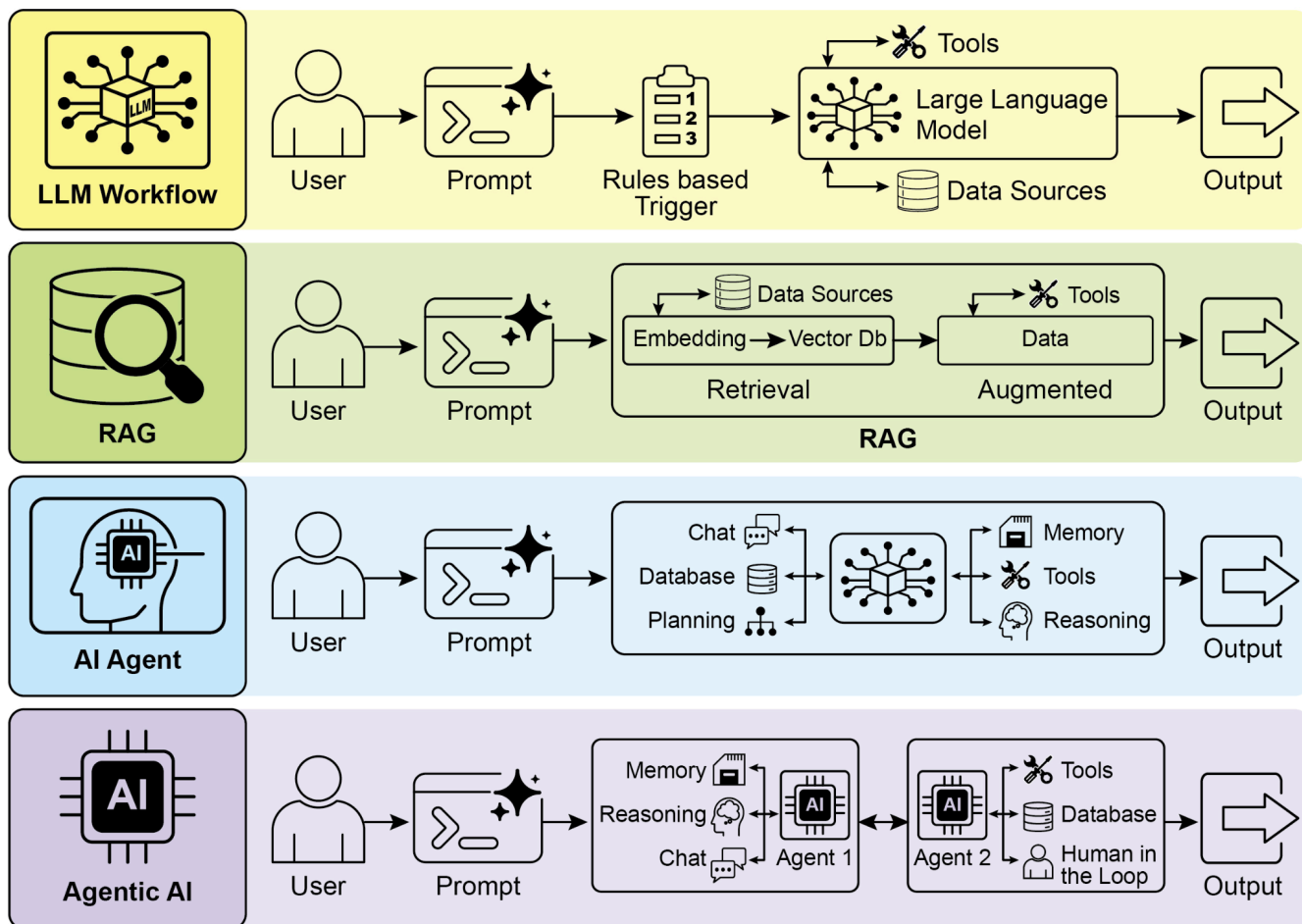
和传统“助手/应用”的区别

智能体不是被动地“被调用”，而是主动地“推进目标”；它更像一个能与人协作的同事：会提出计划、请求资源、寻求澄清，并在必要时进行回滚或再尝试。它应当在“能力可控、影响可追踪”的前提下被部署。



为什么要强调“设计模式”

智能体的失败模式往往来自目标错配、幻觉输出、错误工具调用与隐私问题。通过模式化设计（反思、记忆、规划、异常处理、资源感知等），可以把这些问题转化为工程上的可管理风险，从而构建可复用、可扩展与可治理的系统。



结论

当一个系统具备目标导向、自主决策、环境交互、记忆、工具使用、规划与反思，并以强有力的守护栏与监控保障其行为边界，我们可以合理地称它为“智能体”。而真正让它产生价值的，是恰当的设计与负责任的落地。

概览与导读（中文版）

说明

- 本文件为《Agentic 设计模式》全书的结构化中文导读与概要，帮助你在阅读每章译文前快速把握核心思想与要点。
- 译文输出格式：每章将单独以 Markdown 提供（已完成：Introduction、Chapter 1）。后续可按此目录逐章补全。

总览

- Foreword（序言）
 - 介绍写作动机、读者定位与全书结构。
 - 强调从“模型”走向“系统”的工程化思维。
- Dedication（献词）与 Acknowledgment（致谢）
 - 对贡献者与社区表达感谢。
- Introduction（引言）

- 将“基础设施/框架”视作智能体的开发画布。
- 阐明智能体的特性：自治、前瞻、反应、目标导向、工具使用、记忆与通信。
- 说明为何需要设计模式来管理复杂性。
- 概述 LangChain/LangGraph、Crew AI、Google ADK 等框架作为示例画布。
- Chapter 1: Prompt Chaining (提示链)
 - 分而治之：将复杂任务拆解为多步提示，逐步推进。
 - 输出即输入：上一步结果驱动下一步处理。
 - 引入外部工具与知识，扩展模型能力边界。
 - 结构化输出 (JSON/XML) 确保链内数据稳定传递。
- Chapter 2: Routing (路由)
 - 基于策略选择最合适的子模型/子智能体/工具。
 - 主题分类、门控 (gating)、兜底 (fallback) 与聚合 (aggregation)。
 - 在多工具、多代理的系统中提高准确性与效率。
- Chapter 3: Parallelization (并行化)
 - 并发执行可独立的子任务以降低总时延。
 - Map-Reduce 与分片处理 (sharding)。
 - 控制并发度、避免资源争用与竞态条件。
- Chapter 4: Reflection (反思)
 - 自我评审与修订循环，提升输出质量与一致性。
 - 自动化评价 (auto-eval)、错误定位与迭代修正。
- Chapter 5: Tool Use (工具使用)
 - 设计安全、明确的工具接口与模式 (schema)。
 - 错误处理、幂等性与重试策略。
 - 与外部 API/数据库/服务集成，形成闭环能力。
- Chapter 6: Planning (计划)
 - 层次化规划 (HTN)、任务分解与依赖管理。
 - 动态重规划与执行监控。
- Chapter 7: Multi-Agent Collaboration (多智能体协作)
 - 角色分工、通信协议与协调机制。
 - 监督-工作者 (Supervisor-Worker)、黑板 (Blackboard)、专家团队 (Team-of-Experts)。
- Chapter 8: Memory Management (记忆管理)
 - 短期/长期记忆、检索与更新策略。
 - 摘要、老化 (aging) 与遗忘 (forgetting) 机制。
- Chapter 9: Learning and Adaptation (学习与自适应)
 - 基于反馈的策略更新，强化学习/在线学习。
 - 环境变化下的动态适配。
- Chapter 10: Model Context Protocol (MCP)

- 工具、资源与上下文的标准化接入。
 - 供应商中立的交互协议，提升可移植性。
- Chapter 11: Goal Setting and Monitoring（目标设定与监控）
 - SMART 目标、遥测（telemetry）与进度跟踪。
 - 偏差纠正与过程治理。
- Chapter 12: Exception Handling and Recovery（异常处理与恢复）
 - 重试、断路器（circuit breaker）、补偿（compensation）与回退。
 - 提升系统韧性与可用性。
- Chapter 13: Human-in-the-Loop（人类在环）
 - 审批、复核与干预节点设计。
 - 在关键环节引入人类判断以保障质量与安全。
- Chapter 14: Knowledge Retrieval（RAG，检索增强生成）
 - 索引、分块、嵌入与查询重写。
 - 重排序（re-ranking）与融合（fusion）。
- Chapter 15: Inter-Agent Communication（A2A，智能体间通信）
 - 消息模式、协议与调度。
 - 协商机制与冲突解决。
- Chapter 16: Resource-Aware Optimization（资源感知优化）
 - 预算/时间/Token 管理与配额策略。
 - 缓存、分级降级（graceful degradation）与吞吐优化。
- Chapter 17: Reasoning Techniques（推理技术）
 - CoT（链式思维）、ToT（树式思维）、GoT（图式思维）、PoT（程序式思维）。
 - 选择合适的推理范式提升准确性与稳健性。
- Chapter 18: Guardrails / Safety Patterns（护栏与安全模式）
 - 内容过滤、策略执行与工具安全。
 - 红队测试与安全监控。
- Chapter 19: Evaluation and Monitoring（评估与监控）
 - 指标体系、测试基准与 A/B 实验。
 - 漂移检测与在线监测。
- Chapter 20: Prioritization（优先级）
 - 基于价值/风险/成本的多目标权衡。
 - 任务队列与调度策略。
- Chapter 21: Exploration and Discovery（探索与发现）
 - 面向新颖性的搜索与假设生成。
 - 宽度/深度的平衡与数据资源汇聚。

- Appendices (附录 A–G)
 - A: 高级提示技术 (结构化提示、系统提示、模板化等)。
 - B: 从 GUI 到真实世界的 Agentic 交互方式。
 - C: 主流 Agentic 框架一览与比较。
 - D: 基于 AgentSpace 的构建方法 (在线附录)。
 - E: 命令行上的 AI 智能体 (CLI 工具与 workflows)。
 - F: 智能体推理引擎的内部机制 (Under the Hood)。
 - G: Coding Agents (编程型智能体) 的模式与实践。
- What makes an AI system an Agent? (何谓“智能体”)
 - 从能力维度界定 agent: 感知、推理、行动、记忆与交互。
 - 与传统软件的差异与联系。
- Glossary (术语表) 与 Index of Terms (索引)
 - 统一概念与术语, 便于跨章节检索。
- Online contribution - FAQ (常见问题)
 - 社区贡献的问答与实践建议。
- Conclusion (结论)
 - 总结关键模式与工程思维。
 - 给出未来方向与落地建议。

阅读建议

- 先读引言与第 1 章 (提示链) 以建立核心心智模型; 随后按需阅读路由、工具使用、规划与记忆等基础模式, 再进入协作、安全、评估等系统性章节。
- 在使用具体框架 (LangChain/LangGraph、Crew AI、Google ADK) 时, 优先掌握抽象模式, 再映射到框架 API 与组件, 以降低耦合、提升可迁移性。

前置内容导航 (中文译文)

- 序言 (Foreword): [Foreword.md](#)
- 献词 (Dedication): [Dedication.md](#)
- 致谢 (Acknowledgment): [Acknowledgment.md](#)
- 引言 (Introduction): [Introduction.md](#)
- 思想领袖的视角: 力量与责任: A Thought Leader's Perspective_ Power and Responsibility.md
- 何谓“智能体” (What makes an AI system an Agent?): What makes an AI system an Agent_.md

附录导航 (中文译文)

- 附录 A: 高级提示技术 (Advanced Prompting Techniques)
 - Appendix A_ Advanced Prompting Techniques.md
- 附录 B: AI 智能体交互 (从 GUI 到真实世界)
 - Appendix B - AI Agentic Interactions_ From GUI to Real world environment.md
- 附录 C: Agentic 框架概览 (Quick overview of Agentic Frameworks)
 - Appendix C - Quick overview of Agentic Frameworks.md

- 附录 D：使用 AgentSpace 构建智能体（在线版）
 - Appendix D - Building an Agent with AgentSpace (on-line only).md
- 附录 E：命令行上的 AI 智能体（AI Agents on the CLI）
 - Appendix E - AI Agents on the CLI.md
- 附录 F：深入引擎内部（Under the Hood: Reasoning Engines）
 - Appendix F - Under the Hood_ An Inside Look at the Agents' Reasoning Engines.md
- 附录 G：编程型智能体（Coding Agents）
 - Appendix G - Coding agents.md

相关参考（统一术语）

- 术语表（Glossary，中文版）：[Glossary.md](#)
 - 术语索引（Index of Terms，中文版）：[\[Index of Terms.md\]\(Index of Terms.md\)](#)
-

第 1 章：提示链（Prompt Chaining）

模式概览

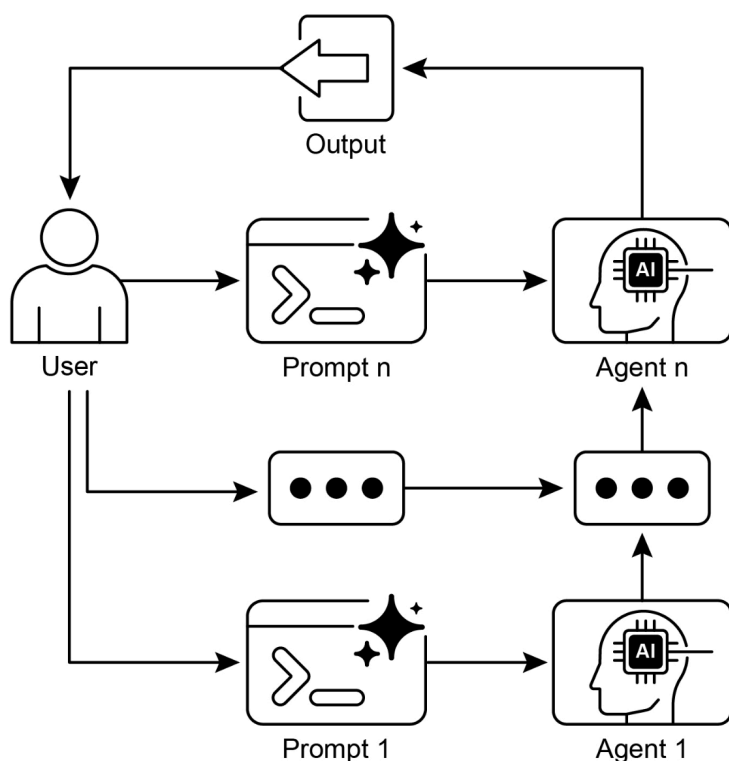
提示链是一种处理复杂任务的强有力范式，也常被称为“流水线（Pipeline）”模式。与其让大语言模型（LLM）在单一、庞大的请求中一次性解决复杂问题，提示链提倡“分而治之”的策略：将原先艰巨的任务拆解为一系列更小、更易管理的子任务。每个子任务通过专门设计的提示（prompt）单独完成，并将上一步的输出有策略地作为下一步的输入。

这种顺序式处理为与 LLM 的交互引入了模块化与清晰性。通过分解复杂任务，更容易理解和调试每个步骤，使整体流程更健壮、更可解释。链中的每一步都能围绕更大问题中的一个具体方面进行精雕细琢与优化，从而得到更准确、聚焦的结果。

提示链的核心在于“输出即输入”：上一步的结果作为下一步的输入形成依赖链，从而让先前的上下文和成果引导后续处理。模型可以在其既有工作成果之上不断迭代，逐步逼近目标解。

此外，提示链不仅是拆解问题的手段，还使外部知识与工具的集成成为可能。在每一步中，LLM 都可以被指示与外部系统、API 或数据库交互，从而将其能力扩展到训练数据之外。这显著提升了模型的潜力，使其不仅是孤立的模型，更是更大、更智能系统中的核心组件。

图示：提示链工作流示意



说明：图示展示“分而治之”的链式流程：复杂任务被拆解为多个阶段，每个阶段输出作为下一阶段输入，形成清晰可控的流水线。

当作为构建复杂智能体的基础技术时，提示链尤为关键。通过精心组织提示序列，智能体可以在动态环境中自主进行规划、推理与行动，完成需要多步推理、计划与决策的任务。这类 workflow 更贴近人类的思维过程，使其在复杂领域与系统中的交互更自然、更有效。

单一提示的局限

对于多面向（多子任务、多约束）的复杂场景，试图用一个“超长且复杂”的提示让 LLM 一步到位，往往会导致：

- 指令忽略：模型无法同时遵守所有约束，遗漏部分要求；
- 上下文漂移：模型逐渐偏离最初目标或背景；
- 错误扩散：前期误差被后续步骤不断放大；
- 上下文窗口压力：过长的提示导致关键信息被截断或权重不足。

通过拆分任务、逐步推进，提示链能降低模型的“认知负荷”，显著提升整体输出的可靠性与可控性。

示例：三步链式分析与写作

以“识别行业趋势并生成报告”为例，可以设计如下三步链：1) 趋势识别（角色：市场分析师）：

- 输入：领域与时间范围；
- 操作：汇总并提炼最具代表性的趋势；
- 输出：结构化的趋势条目集合。

2) 影响评估（角色：交易分析师）：

- 输入：第 1 步的趋势条目；

- 操作：分析各趋势对市场的潜在影响与风险；
- 输出：每个趋势的影响分析与证据引用。

3) 撰写报告（角色：资深写作者）：

- 输入：第 2 步的评估结果；
- 操作：生成一篇面向决策者的可读性强的报告；
- 输出：完整报告，含摘要、正文、小结与参考。

这种分解实现了更细粒度的过程控制。每一步更简单、更不含糊，减少模型负担，提升准确性与稳定性。模块化特性也类似计算流水线：每个函数完成特定操作再将结果传递给下一步。为了确保每一阶段都能给出精确回答，可以为模型在各阶段分配不同的“角色”身份（如上示例中的 Market Analyst、Trade Analyst、Expert Documentation Writer）。

结构化输出的作用

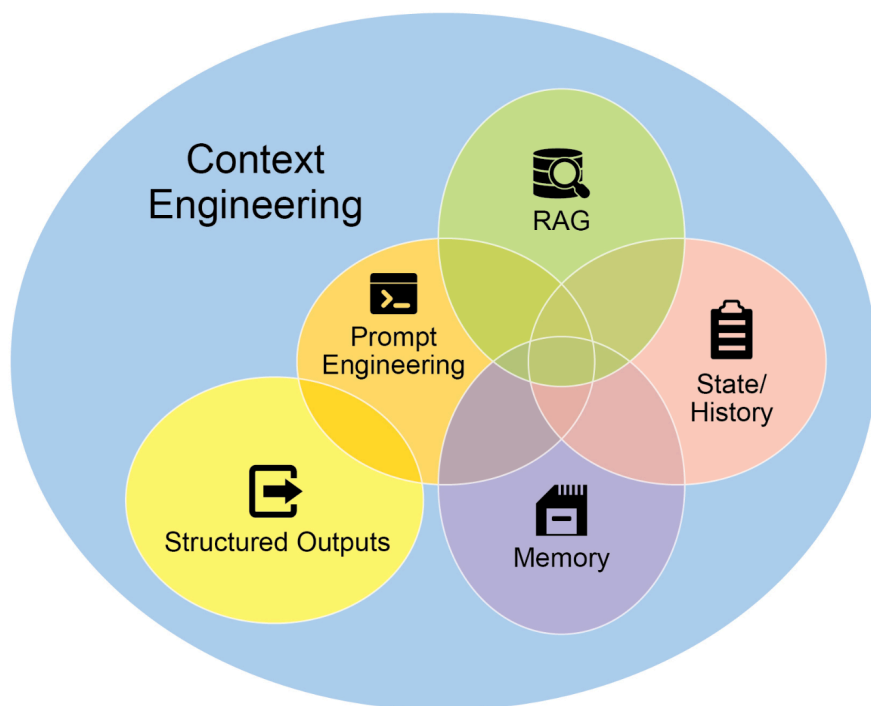
提示链的可靠性高度依赖步骤之间传递数据的完整性。如果某一步的输出含糊或格式不规范，下一步很可能因输入不良而失败。为此，指定结构化输出（如 JSON、XML）至关重要。

例如，趋势识别步骤的输出可以采用如下 JSON 格式：

```
{
  "trends": [
    {
      "trend_name": "AI-Powered Personalization",
      "supporting_data": "73% 的消费者更倾向与能使用个人信息提升相关性的品牌进行业务往来。"
    },
    {
      "trend_name": "Sustainable and Ethical Brands",
      "supporting_data": "过去五年，带有 ESG 相关主张的产品销量增长 28%，而无此主张的产品增长 20%。"
    }
  ]
}
```

这种格式使数据具备机器可读性，可被精确解析并注入下一步提示，避免自然语言理解带来的歧义，是构建稳健的多步 LLM 系统的关键实践。

图示：结构化输出在链中的传递



说明：示意图强调在链式流程中使用结构化格式（如 JSON）进行数据传递，降低歧义并提升稳健性。

典型应用场景

提示链在众多场景中适用，包括但不限于：

- 报告与内容生成：先检索与聚合，再结构化撰写与校对；
- 代码生成与修复：需求澄清 → 方案设计 → 代码实现 → 测试与修复；
- 数据问答与决策支持：问题解析 → 检索与计算 → 推理与建议；
- 多工具编排：根据上下文路由到特定工具，分步集成外部能力；
- 任务分解与计划执行：将宏观目标拆解为可执行的细粒度步骤。

框架支持与落地实现

- LangChain / LangGraph：提供表达式语言（LCEL）与有向图（graph）式工作流，支持链式与图式编排；
- Google ADK：提供构建、评估与部署智能体的组件，便于把链式流程嵌入更大的代理系统；
- Crew AI：强调角色、任务与流程的结构化，让多智能体协作中的链式编排更清晰。

结论

通过将复杂问题拆解为一系列更简单、更可管理的子任务，提示链为引导 LLM 提供了可靠框架。这种“分而治之”策略通过让模型一次只专注于一个明确操作，显著提升输出的可靠性与可控性。作为基础模式，它支撑多步推理、工具集成与状态管理，帮助我们构建具备强上下文感知能力、能执行复杂工作流的智能系统。

参考资料

- LangChain 文档（LCEL）：https://python.langchain.com/v0.2/docs/core_modules/expression_language/
- LangGraph 文档：<https://langchain-ai.github.io/langgraph/>

- Prompt Engineering Guide (Chaining): <https://www.promptingguide.ai/techniques/chaining>
 - OpenAI API 文档 (通用提示指南): <https://platform.openai.com/docs/guides/gpt/prompting>
 - Crew AI 文档 (任务与流程): <https://docs.crewai.com/>
 - Google AI for Developers (提示指南): <https://cloud.google.com/discover/what-is-prompt-engineering?hl=en>
 - Vertex Prompt Optimizer: <https://cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/prompt-optimizer>
-

第 2 章：路由 (Routing)

模式概览

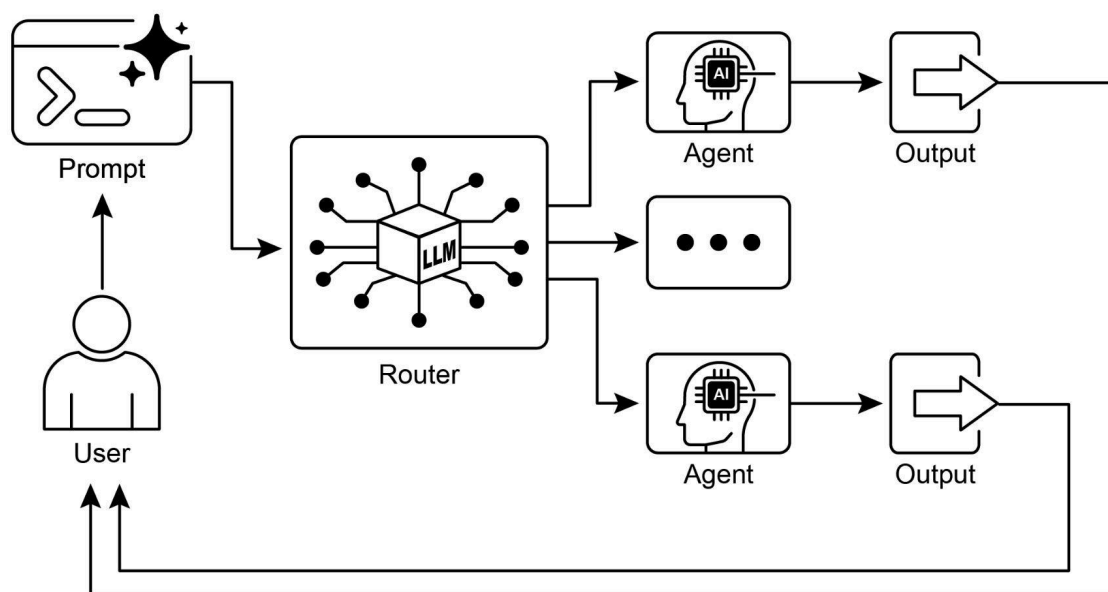
虽然基于提示链 (Prompt Chaining) 的顺序处理是执行可确定、线性工作流的基础技术，但在需要自适应响应的场景中其适用性有限。真实世界的 agentic 系统常常需要根据环境状态、用户输入或前置步骤的结果在多种潜在行动之间进行裁决。这种“让控制流去往不同的专用函数、工具或子流程”的动态决策能力，正是通过“路由 (Routing)”实现的。

路由将条件逻辑引入智能体的运行框架，使系统从固定执行路径转向一种“根据特定判据在多种后续行动中进行动态选择”的模式，从而表现出更灵活、更具上下文感知的系统行为。

例如，一个处理客户咨询的智能体，如果具备路由功能，首先会对来询进行分类以判断用户意图；随后，它可以将问题导向问答型子智能体、账户信息数据库检索工具，或在复杂情况时触发升级流程，而不是总走一条预设的单一路径。高度智能化的路由智能体大致会：

- 分析用户问题；
- 按意图路由：
 - 如果意图是“查询订单状态”，则路由到与订单数据库交互的子智能体或工具链；
 - 如果意图是“产品信息”，则路由到搜索产品目录的子智能体或链；
 - 如果是“技术支持”，则路由到故障排查指南的链或升级到人工；
- 如果意图不明确，则路由到澄清子智能体或提示链。

图示：路由模式视觉摘要 (示意图)



核心组件与实现方式

路由模式的核心是“执行评估并指引控制流”的机制，可通过多种方式实现：

- 基于 LLM 的路由：通过提示让语言模型分析输入并输出指示下一步的类别/指令。例如：“分析用户问题并仅输出类别：Order Status / Product Info / Technical Support / Other”。系统读取该输出并据此指引 workflow。
- 基于嵌入的路由：将输入转为向量嵌入（参见第 14 章 RAG），与各路由或能力的代表性嵌入对比，选择语义最接近的路径。适用于语义路由，即基于“含义”而非关键词进行决策。
- 基于规则的路由：使用预定义规则与逻辑（如 if-else、switch），基于关键词、模式或从输入中抽取的结构化数据进行判断。速度快、确定性强，但在处理细微差异或新颖输入时灵活性较低。
- 基于判别模型的路由：训练一个小规模标注语料上的分类器，用以执行特定路由任务。其特征在于监督微调，使路由逻辑编码在模型参数中。不同于 LLM 路由，推理时不需调用生成式模型执行提示；尽管可用 LLM 生成合成数据以扩充训练集，但实时路由决策由微调模型完成。

路由机制可以在智能体的多个阶段应用：任务起始用于主任务分类；处理链中途用于决定下一步行动；子程序内部用于从一组工具中选择最合适的一个。

框架支持与架构选择

LangChain、LangGraph 与 Google 的 Agent Developer Kit (ADK) 都提供了用于定义与管理条件逻辑的显式构件：

- LangGraph 的“基于状态的图架构”非常适合复杂路由场景，决策可依赖于系统累积状态；
- Google ADK 提供结构化的能力定义与交互模型，这些组件可作为路由逻辑实现的基础。

在执行环境中，路由还涉及：

- 状态维护与过渡：清晰建模状态、转移条件与动作。
- 可观测性与审计：对路由决策进行记录，以便追踪与调试。
- 兜底与回退：当路由不确定或各分支失败时，提供安全退路（如默认响应、人工升级或澄清）。

实践要点

- 设计意图空间：明确智能体支持的意图类别与边界情况。
- 选择路由器：根据场景选择 LLM、嵌入、规则或判别模型方案，必要时混合使用。
- 定义数据契约：为路由输入/输出指定结构化模式（如 JSON），确保上下游可解析与健壮。
- 评估与迭代：对路由准确率、分支成功率与端到端 KPI（如一次解决率、时延）进行评估并迭代优化。

结论

路由是构建真正动态、可响应的 agentic 系统的关键一步。通过实现路由，我们超越了简单的线性执行流，赋予智能体“如何处理信息、响应用户输入以及调用可用工具或子智能体”的智能决策能力。从客服聊天机器人到复杂数据处理管线，能够分析输入并有条件地引导工作流，是让智能体应对真实任务可变性的基石。

在实现层面，LangGraph 的图式结构提供了可视、显式的状态与转换定义，适合复杂多步工作流与精细路由逻辑；Google ADK 则强调“能力（Tools）”的定义，并依赖框架将用户请求路由到相应的工具处理器，更适合离散动作集明确的智能体。熟练掌握路由模式有助于构建能智能穿梭于不同场景、并基于上下文提供定制响应或行动的高质量应用。

参考资料

- LangGraph 文档：<https://www.langchain.com/>
- Google Agent Developer Kit 文档：<https://google.github.io/adk-docs/>

第 3 章：并行化（Parallelization）

模式概览

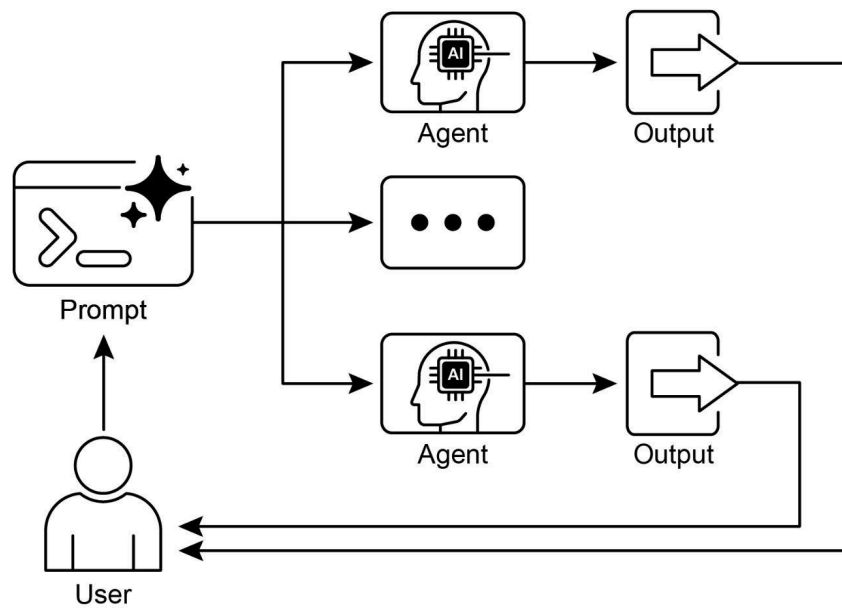
在前两章中，我们分别讨论了用于顺序工作流的提示链（Prompt Chaining）与用于动态决策与路径切换的路由（Routing）。在许多复杂的 agentic 任务中，存在可以同时执行的多个子任务，而非必须串行完成的步骤，这正是“并行化（Parallelization）”发挥关键作用的地方。

并行化是指将多个组件（如 LLM 调用、工具使用，乃至完整的子智能体）并发执行，而不是等待一个步骤结束再启动下一步。对于可拆分为相互独立部分的任务，并行执行能显著降低整体时延。

设想一个“研究并总结某主题”的智能体：

- 串行方案：依次搜索来源 A → 总结来源 A → 搜索来源 B → 总结来源 B → 将两份总结综合为最终答案。
- 并行方案：并发搜索来源 A 与 B；待两次搜索完成后，并发总结 A 与 B；最后顺序地进行综合（通常需等待并行步骤结束）。

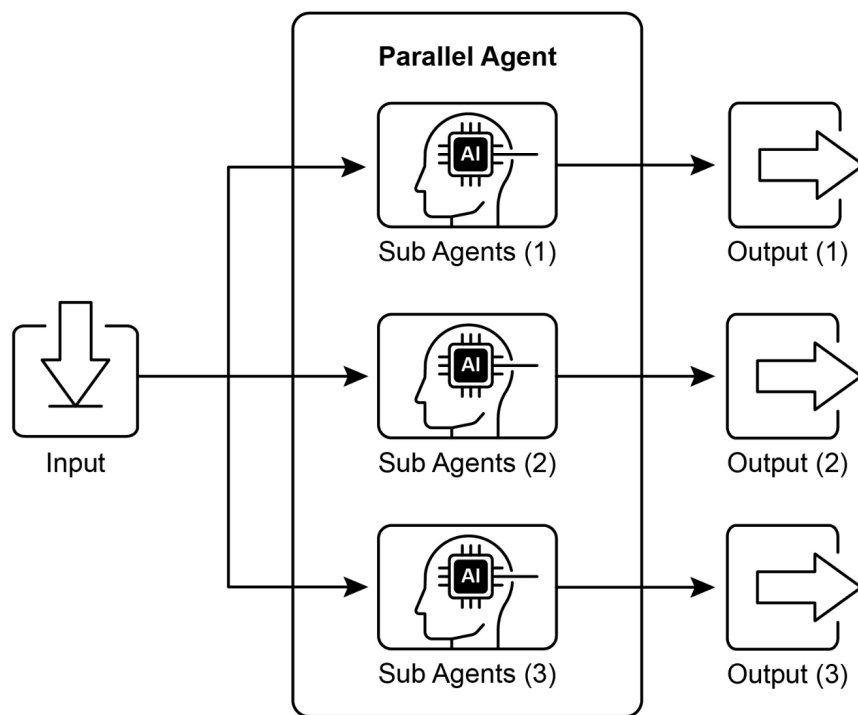
图示：并行化模式视觉摘要（示意图）



核心思想是识别 workflow 中“彼此无依赖”的部分并并行执行。在涉及外部服务（API/数据库）且存在网络或处理延迟的场景中，多路并发请求尤为有效。

实现并行化往往需要框架支持异步执行、多线程或多进程。现代 agentic 框架普遍以异步为设计目标，便于定义可并发运行的步骤。

图示：并行执行与异步化（示意图）



框架支持（示意）

- LangChain / LCEL：通过组合可运行对象（runnable），以顺序（如 |）与并行结构定义多分支链/图；
- LangGraph：基于图的状态与转移，支持在一次状态跃迁中触发多个节点并行执行；
- Google ADK：原生支持多智能体并行与管理，通过“LLM 驱动的委派（Delegation）”由协调者（Coordinator）识别独立子任务并交由专门子智能体并发处理。

并行化是提升 agentic 系统效率与响应性的关键技术，特别是在涉及多路独立检索、计算或外部服务交互的任务中。

典型应用场景

1) 信息聚合与研究：

- 用例：研究某家公司；
- 并行任务：同时抓取新闻、股票数据、社媒提及与公司数据库；
- 收益：相较串行检索，更快获得全面视图。

2) 数据处理与分析：

- 用例：分析客户反馈；
- 并行任务：并发情感分析、关键词抽取、分类与紧急问题识别；
- 收益：快速得到多维度结果。

3) 多 API/工具交互：

- 用例：行程规划智能体；
- 并行任务：并发查询机票价格、酒店可订性、当地活动与餐厅推荐；

- 收益：更快产出完整行程方案。

4) 多组件内容生成：

- 用例：生成营销邮件；
- 并行任务：同时生成标题、正文草稿、相关图片与行动号召文案；
- 收益：更高效地组装最终内容。

5) 校验与验证：

- 用例：用户输入校验；
- 并行任务：并发检查邮箱格式、手机号、地址规范与黑名单比对；
- 收益：低延迟完成多项独立验证。

工程注意事项

- 依赖图与并发度：明确各步骤的依赖关系，并设置合理的并发上限以避免资源争用；
- 幂等与重试：并发调用外部服务时，确保接口幂等与可重试；
- 超时与降级：为每个并行分支设置超时与降级策略，避免整体阻塞；
- 聚合与顺序化：并行结果的聚合通常是后续顺序步骤，注意聚合的容错与缺省值；
- 可观测性与日志：记录并行分支的开始/结束、异常与重试，便于追踪与优化。

结论

并行化通过并发执行彼此独立的子任务来优化计算 workflow，尤其适用于包含多次模型推理或外部服务调用的复杂操作。不同框架提供各自的并行实现机制：LangChain 倾向于用可运行对象显式组合并行分支；Google ADK 则常通过多智能体委派实现“由协调者分派、由专家并行执行”的模式。将并行处理与顺序（链式）及条件（路由）控制流结合，可构建高性能、能高效管理多样复杂任务的系统。

参考资料

- LCEL 并行文档：<https://python.langchain.com/docs/concepts/lcel/>
- Google ADK（多智能体）：<https://google.github.io/adk-docs/agents/multi-agents/>
- Python asyncio 文档：<https://docs.python.org/3/library/asyncio.html>

第 4 章：反思（Reflection）

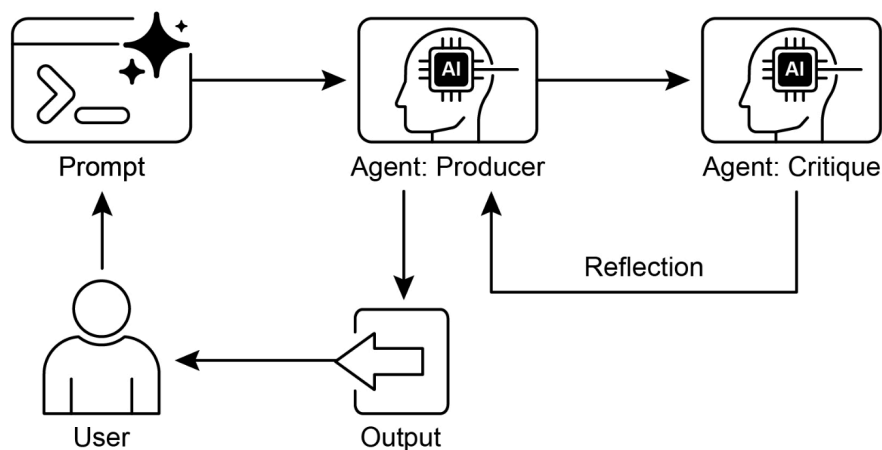
模式概览

在前述基础模式（提示链、路由、并行）帮助智能体高效、灵活地执行复杂任务的同时，即便拥有精心设计的工作流，智能体的初次输出或计划仍可能并不最优、准确或完整。这时，“反思（Reflection）”模式登场。

反思模式要求智能体评估自身的工作、输出或内部状态，并据此改进性能或优化响应。通过引入反馈环，智能体可基于反馈、内在批评或与既定标准的对比进行迭代自我修正。这一过程也常由一个专门的“批评者（Critic）”智能体辅助完成，以提升效果与客观性。

区别于简单的顺序链（直接把输出传至下一步）或路由（选择路径），反思引入了“生成—评估—改进”的回路：智能体不仅生成输出，还审视输出（或生成过程），识别问题或改进空间，并利用这些洞见产生更佳版本或调整后续行动。

图示：反思模式视觉摘要（示意图）



基本流程

1) 执行 (Execution): 智能体执行任务或生成初始输出。2) 评估/批评 (Evaluation/Critique): 使用另一次 LLM 调用或规则集分析上一步结果, 可检查事实准确性、连贯性、风格、完整性、指令遵循度等。3) 反思/改进 (Reflection/Refinement): 依据批评意见进行改进, 可能是生成修订版、调整下一步参数, 甚至修改整体方案。4) 迭代 (可选但常见): 执行修订版并重复上述反思过程, 直到达到满意结果或满足终止条件。

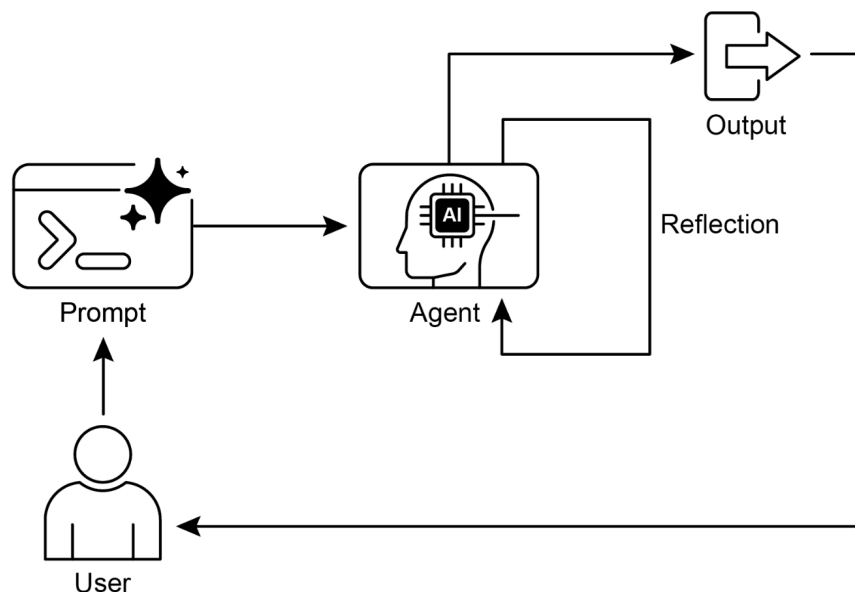
生成者—批评者模型 (Producer-Critic)

一种高效的实现是将流程拆分为两个逻辑角色:

- 生成者 (Producer): 负责初始执行与内容生成 (如写代码、撰文、制定计划), 产出第一版结果;
- 批评者 (Critic): 仅负责评估生成者输出, 通常给予不同的系统指令/人格设定 (如“资深软件工程师”“严格事实核查员”), 并根据事实准确性、代码质量、风格与完整性等标准, 发现问题、提出改进并输出结构化反馈。

这种“关注点分离”能避免单个智能体自评的偏见: 批评者以全新视角专注于发现错误与改进点; 随后生成者依据批评者的结构化反馈生成更优版本。LangChain 的示例可通过特定反思提示 (reflector_prompt) 塑造批评者人格; ADK 的示例则显式定义生成者与审稿者两类智能体。

图示: 生成者—批评者反思回路 (示意图)



架构与落地

实现反思通常需要在工作流中嵌入反馈环与状态管理：

- 通过代码中的循环与条件，或借助支持状态与条件转移的框架（LangChain/LangGraph、ADK、Crew AI）定义“评估结果 → 转入修订”逻辑；
- 单次“生成—批评—改进”即可体现核心原则，而真正的“多轮迭代反思”往往需要更复杂的编排能力与稳健的状态存储。

与目标设定与监控（第 11 章）存在交叉：明确、可度量的目标与指标可作为批评者的评估基准，并驱动迭代收敛。

框架支持（示意）

- LangGraph：基于状态的图式架构，适合在多步反思中维护上下文与转移条件；
- LangChain / LCEL：即便无复杂状态机，也可通过一次“输出—批评—修订”的管线展示反思；
- Google ADK：可通过顺序工作流由一个智能体生成，另一个智能体批评，再进行后续修订；
- Crew AI：在多智能体协作中设置“生产者—审稿者”角色与流程，提升质量与一致性。

结论

反思模式为智能体工作流提供了关键的自我修正机制，突破单次执行的局限，通过“生成—评估—改进”的闭环实现迭代提升。作为一种控制结构，它与其他基础模式（链、路由、并行）相结合，可构建更稳健、功能更复杂的 agentic 系统，使智能体更能胜任高质量输出、细致任务处理与具备一定的自我意识与适应性。

参考资料

- 训练语言模型通过强化学习进行自我纠错：<https://arxiv.org/abs/2409.12917>
- LangChain Expression Language (LCEL)：<https://python.langchain.com/docs/introduction/>

- LangGraph 文档: <https://www.langchain.com/langgraph>
 - Google ADK (多智能体): <https://google.github.io/adk-docs/agents/multi-agents/>
-

第 5 章：工具使用（函数调用）

模式概述

到目前为止，我们讨论的智能体模式大多聚焦于在智能体内部编排语言模型（LLM）的交互与信息流（如链式调用、路由、并行化、反思）。但要让智能体真正与现实世界或外部系统互动，它必须具备“使用工具”的能力。

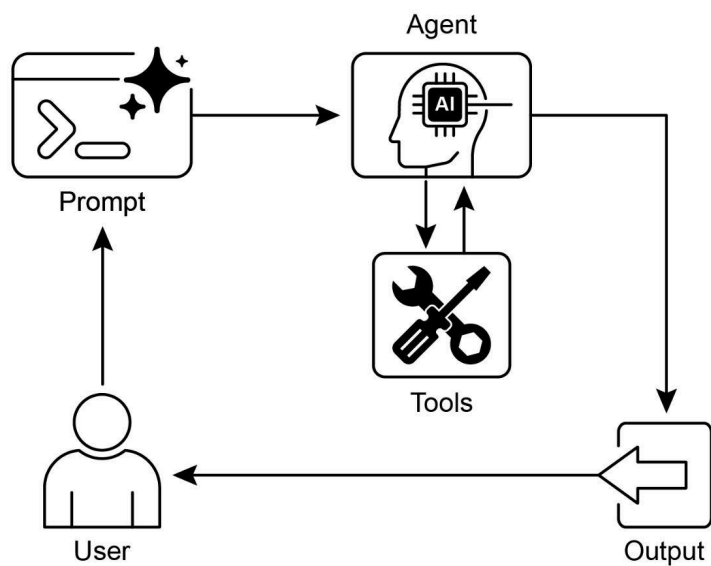
工具使用（Tool Use）通常通过“函数调用（Function Calling）”实现，使智能体可以与外部 API、数据库、服务交互，甚至执行代码。核心思想是：LLM 在理解用户请求与当前任务状态后，判断是否需要调用某个外部功能；一旦需要，LLM 会生成结构化的数据（常为 JSON），指明要调用的函数及其参数；编排层执行该函数，获取结果，并把结果作为“观察”反馈给 LLM，以便其整合最新信息或执行动作，从而给出更准确、更有用的最终响应。

运行流程（典型步骤）

- 工具定义：向 LLM 描述外部功能（工具/函数）的用途、名称、参数及类型与说明。
- LLM 决策：LLM 接收用户请求与可用工具定义，基于其理解判断是否需要调用一个或多个工具。
- 函数调用生成：若需要，LLM 生成结构化输出（常为 JSON），其中包含要调用的工具名以及参数（从用户请求或上下文提取）。
- 工具执行：智能体编排层拦截该结构化输出，识别请求的工具并以提供的参数执行真实的外部函数。
- 观察/结果：工具执行的输出作为“观察”返回给智能体。
- LLM 处理（常见）：LLM 将工具输出纳入上下文，据此生成最终回复或决定下一步（例如再调用其他工具、进行反思，或直接给出答案）。

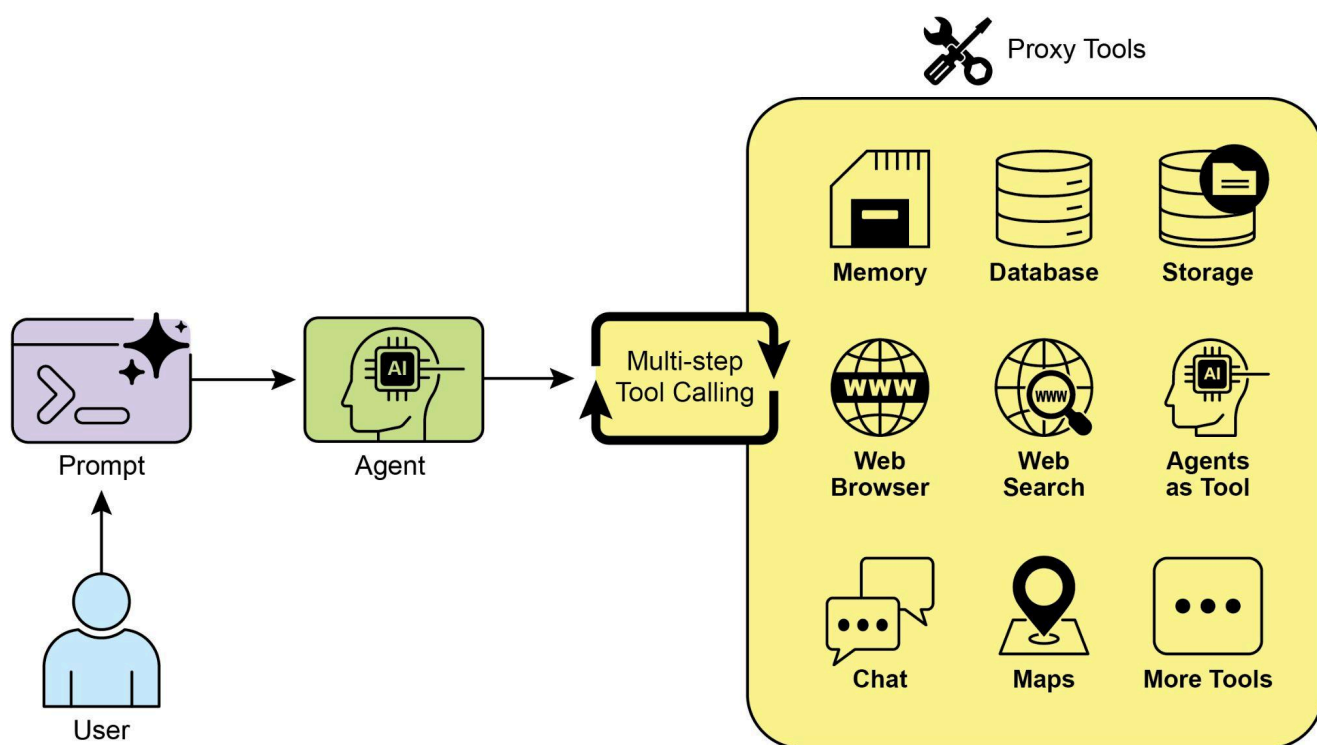
这一模式的关键价值在于：它突破了 LLM 仅依赖训练语料的局限，使其能够访问最新信息、进行精确计算、读取用户特定数据，或触发现实系统中的动作。函数调用作为技术机制，架起了 LLM 的推理能力与外部海量功能之间的桥梁。

进一步地，除了“函数调用”，我们也可用更广义的“工具调用（Tool Calling）”来描述。工具不仅可以是传统代码函数，还可能是复杂的 API 端点、数据库查询，乃至指派给另一位专用智能体的请求。从这一视角出发，主智能体可以在更广泛的资源生态中充当编排者，例如将复杂的数据分析任务委派给“分析师智能体”，或通过 API 查询外部知识库。



关键点

- 工具使用（函数调用）让智能体能与外部系统交互并获取动态信息。
- 工具需要以清晰的名称、用途、参数与类型说明对 LLM 可理解化。
- LLM 负责决定何时使用工具，并生成结构化的函数调用请求。
- 编排层执行真实的工具调用，并将结果返回给 LLM。
- 工具使用是构建可行动、能提供最新信息的“现实世界”智能体的基石。
- LangChain 通过 `@tool`、`create_tool_calling_agent` 与 `AgentExecutor` 简化工具定义与智能体构建。



框架支持

- LangChain / LCEL：提供便捷的工具体义方式与基于函数调用的智能体组件，统一工具接口与参数模式。
- LangGraph：在图式编排中承载工具节点与条件流转，使工具调用融入更复杂的状态与控制逻辑。
- Google ADK：内建多种常用工具（如 Google Search、代码执行、Vertex AI Search），并提供将工具公开给模型、解析模型工具请求的结构化组件。
- 现代 LLM（如 Gemini、OpenAI 系列）原生支持函数/工具调用能力，便于在上述框架“画布”上进行集成。

典型应用与用例

1. 外部信息检索：访问实时数据或训练语料之外的信息。
 - 示例：天气智能体。
 - 工具：天气 API（输入位置，返回当前天气）。
 - 流程：用户问“伦敦天气如何？”→ LLM 识别需调用天气工具 → 使用“London”调用 → 工具返回数据 → LLM 格式化友好答复。
2. 数据库与 API 交互：对结构化数据执行查询、更新或调用业务接口。
 - 示例：电商智能体。
 - 工具：库存查询、订单状态、支付处理等 API。
 - 流程：用户问“X 产品有货吗？”→ LLM 调用库存 API → 工具返回库存数 → LLM 告知库存状态。
3. 计算与数据分析：使用外部计算器、数据分析库或统计工具。
 - 示例：金融智能体。
 - 工具：计算函数、股票数据 API、表格处理工具。
 - 流程：用户问“当前 AAPL 价格是多少？若我以 \$150 买入 100 股潜在利润是多少？”→ LLM 先查价再计算 → 返回结论。

4. 代码与脚本执行：在受控环境中运行片段或任务脚本，完成数据转换、抓取或自动化运维。
5. 控制外部系统：触发现实世界动作，如智能家居设备、工单系统或消息/邮件发送。
6. 校验与安全：调用内容审核、权限与合规模型或网关，在生成前后进行审查与授权。

工程实践建议

- 结构化模式优先：为工具定义清晰的模式（模式/Schema），包括严格的参数类型与边界条件。
- 描述清晰：为每个参数提供简洁明确的用途说明，减少模型误用概率。
- 错误与异常处理：为工具调用建立重试、超时、回退与幂等逻辑，并在编排层做健壮的错误分流。
- 安全与权限：在工具层实现认证、鉴权、审计与速率限制，避免越权调用与数据泄露。
- 观察可观测性：记录每次调用的输入/输出与延迟，便于故障定位与质量评估。
- 多轮融合：合理将工具结果注入到后续对话上下文，必要时结合反思（Reflection）或再次调用其它工具。
- 循环与终止：控制调用链的深度与次数，防止工具调用的“无限循环”。
- 降级与兜底：在工具不可用时提供替代路径（离线回答、缓存数据或提示用户稍后重试）。

小结

工具使用是扩展 LLM 能力边界的关键架构原则。通过为模型配备与外部软件和数据源交互的能力，智能体可以执行动作、进行计算、并从其他系统检索信息。当模型判断需要时，会生成结构化的工具调用请求；框架（如 LangChain、Google ADK、Crew AI 等）提供抽象与组件，帮助把工具规范暴露给模型并解析其后续的工具使用请求。借助这些框架，开发者可以更轻松地构建能与外部数字环境交互、并在其中采取行动的高级智能体系统。

参考资料

- LangChain 文档（工具）：<https://python.langchain.com/docs/integrations/tools/>
 - Google Agent Developer Kit (ADK) 文档（工具）：<https://google.github.io/adk-docs/tools/>
 - OpenAI 函数调用文档：<https://platform.openai.com/docs/guides/function-calling>
 - CrewAI 文档（工具）：<https://docs.crewai.com/concepts/tools>
-

第 6 章：规划（Planning）

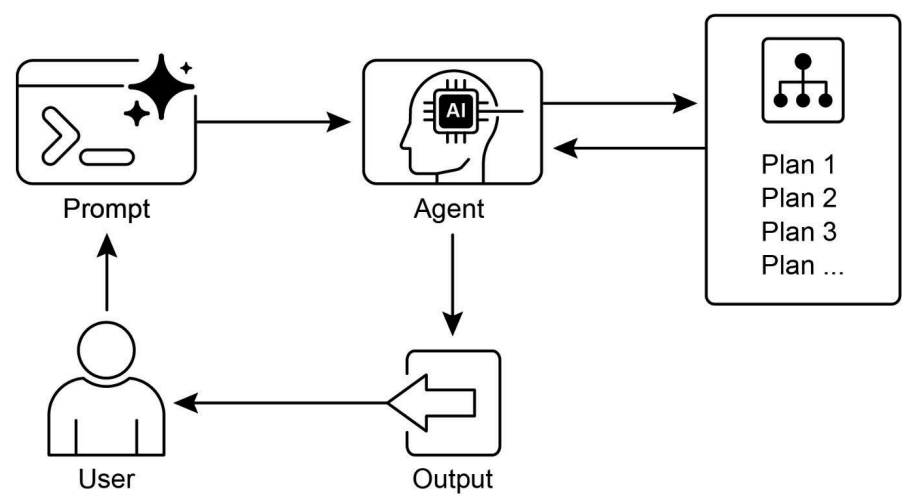
模式概述

智能行为往往不仅是对眼前输入的实时反应，更需要前瞻性思考、把复杂目标拆解为可管理的步骤，并制定达成目标的策略。规划（Planning）模式的核心，是让智能体（或智能体系统）能够从初始状态出发，形成一系列有序的行动，逐步迈向目标状态。

在 AI 语境中，可以把“规划智能体”理解为你委派复杂目标的专职专家：你给它“要做什么”（目标与约束），而不是“怎么做”。它需要先理解初始状态（如预算、参与人数、日期偏好等）与目标状态（例如成功预定的团建活动），然后自主制定从起点到终点的最优行动序列。这个计划并非预先存在，而是基于请求即时生成。

规划的一大特征是“适应性”。初始计划只是起点，而不是固定剧本。强大的智能体应能在获取新信息时重新导航。例如，若首选场地不可用或餐饮商家已满档，智能体不会直接失败，而是登记新约束、重新评估选项，并提出替代场地或日期，更新计划。

需要注意的是，灵活性与可预测性之间存在权衡。动态规划是一种特定工具，而非万能方案。当问题的解法已被充分理解且可重复时，采用“预设、固定的工作流”通常更有效。这能减少系统行为的不确定性与漂移，提升一致性与可控性。



典型能力与实现

- LLM 生成步骤式方案：依据任务描述，语言模型可输出“逐步达成”的方案，用于复杂目标的拆解与执行指导。

- Prompt 促进规划：在提示语中明确要求“先规划再执行”，或将任务设计为必须给出计划的格式，有助于稳定产出可执行的步骤列表。
- 计划—执行—反思闭环：在多层任务中，智能体先给出计划，再分步执行，并在关键节点进行反思（Reflection），按需修订计划。
- 结合工具与检索：在规划阶段引入工具使用（Tool Use）或检索（RAG），以实时数据与外部信息校准计划的可行性与优先级。

 Europe VC Investment Analysis

Export to Docs



European Venture Capital Investment Landscape: Trends and Accelerating Countries

Executive Summary

This report provides a comprehensive analysis of the venture capital (VC) investment landscape in Europe, focusing on the trends observed in 2024 and the first quarter of 2025. The analysis identifies the total VC investment volume in Europe during these periods, highlights the top-performing countries, and pinpoints those experiencing an acceleration in investment activity. Furthermore, the report examines the key sectors that are attracting the most significant capital, offering insights into the evolving dynamics of the European venture ecosystem. Findings indicate a stabilization of the market in early 2025 after a mixed performance in 2024, with specific countries like Spain, the UK & Ireland, and the Netherlands demonstrating notable growth. The leading sectors driving investment include Artificial Intelligence and Machine Learning, Healthcare and Biotech, Fintech, Cleantech and Energy, and Deep Tech and Defense Tech.

State of European VC Investment in 2024

Total VC Investment Volume in Europe in 2024

The total volume of venture capital investment in Europe during 2024 presents a varied picture depending on the data source. Crunchbase data indicates a total of \$51 billion invested, representing a 5% decrease compared to the \$54 billion recorded in 2023. In contrast, Invest Europe reported a strong rebound with €18 billion (approximately \$19.4 billion) invested, marking a substantial 26% increase year-over-year. This figure positions 2024 as the second-highest year for VC investment in Europe, surpassed only by the record levels of 2022. GoingVC data, however, suggests a more significant downturn, reporting \$10 billion in VC funding for Europe in 2024, a 39% decline from the previous year. PitchBook data estimates the total deal value at €57.3 billion (approximately \$61.8 billion), indicating a 6.7% drop from the €55.1 billion invested in 2023.

The discrepancies in these figures highlight the challenges inherent in tracking venture capital investment and likely stem from differences in methodologies, including the scope of what is considered VC investment, deal size thresholds, and the inclusion or exclusion of specific funding types or regions. The timing of data collection and reporting can also contribute to these variations. Despite these conflicting numbers, a broader trend emerges when comparing 2024 to previous years. While Crunchbase and PitchBook suggest a decrease or stabilization compared to 2023, Invest Europe points to a strong recovery. However, all sources generally agree that the investment volume in 2024 remained significantly below the peak observed in 2021. This indicates a market recalibration following a period of extraordinary growth. Dealroom data further supports this, showing a 37% drop in European VC investment in 2023 compared to 2022, suggesting that 2024, according to some reports, might be showing signs of a positive shift after a period of decline.

Top European Countries by VC Investment Volume in 2024

The United Kingdom consistently emerged as the leading country in Europe for venture capital investment in 2024. Crunchbase data reported

框架与实践示例

- CrewAI：常见写作与项目型智能体先生成写作/任务计划，再逐步完成各部分，体现“先规划再执行”的序列化流程。
- Google DeepResearch（Gemini / 生态相关）：通过搜索与多步推理，迭代生成研究计划，持续整合新信息与引用数据，形成结构化、可验证的研究报告。
- OpenAI Deep Research：面向严肃研究与商业决策的深度研究代理，可输出带有引文的结构化报告；过程透明（公开中间步骤、搜索查询、运行代码等），并支持 MCP（模型上下文协议）连接私有知识库与内部数据源。
- Perplexity Deep Research：强调网页检索与内容综合的深度研究流程，支持数据来源标注与可溯性。

说明：以 OpenAI 的 Deep Research API 为例，调用时会向 `client.responses.create` 提交请求，指定模型、输入提示（包含系统角色与用户问题），并声明可用的工具（如网页搜索、代码解释器或自定义 MCP 工具）。此

类接口将“规划—检索—执行—综合”的过程结构化、透明化，便于开发者调试与扩展。

what is the volume of VC investment in Europe and the countries that are accelerating



I've put together a research plan. If you need to update it in any way, let me know.

Europe VC Investment Analysis



Research Websites

- (1) Search for the total volume of venture capital investment in Europe for the current year (2025) and the previous year (2024).
- (2) Find reports or articles detailing the top European countries by VC investment volume in recent years.
- (3) Look for data on the year-over-year growth rate of VC investment in different European countries.
- (4) Identify countries in Europe that have experienced the largest percentage increase in VC investment recently.
- (5) Search for analyses or reports that specifically highlight European countries with accelerating VC investment activity.
- (6) Investigate if there are any specific sectors driving the acceleration of VC investment in these countries.



Analyze Results



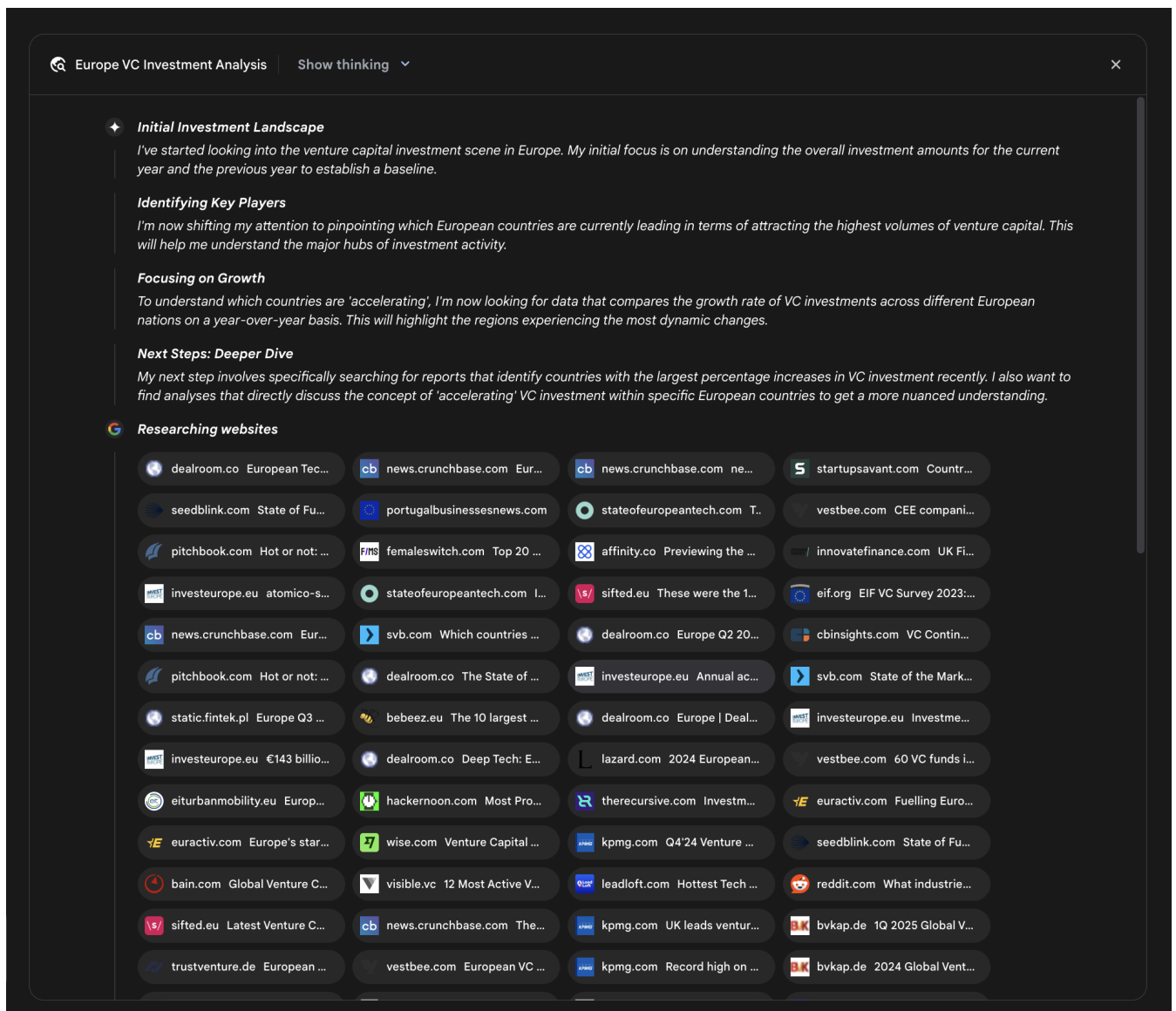
Create Report



Ready in a few mins

Edit plan

Start research



关键要点

- 规划让智能体能将复杂目标拆解为可执行的顺序步骤。
- 它是处理多步骤任务、自动化工作流与在复杂环境中导航的关键。
- LLM 能基于任务描述生成步骤式方案，承担规划角色。
- 通过明确提示或任务设计，鼓励智能体“先规划后执行”。
- 深度研究型代理（如 Google DeepResearch 等）体现了“检索—反思—规划—执行”的闭环能力。

小结

规划模式是将智能体从“被动响应者”提升为“战略执行者”的基石。现代 LLM 能将高层目标自主分解为连贯、可执行的步骤；该模式既可用于简单的顺序任务（如 CrewAI 先生成写作计划再写作），也可扩展到更复杂、更动态的系统（如 DeepResearch 迭代研究计划并随信息更新而演进）。本质上，规划在“人类意图”与“自动执行”之间搭起桥梁，让智能体能够管理复杂工作流并产出结构化、综合性的结果。

参考资料

- Google DeepResearch（Gemini 相关功能）：gemini.google.com

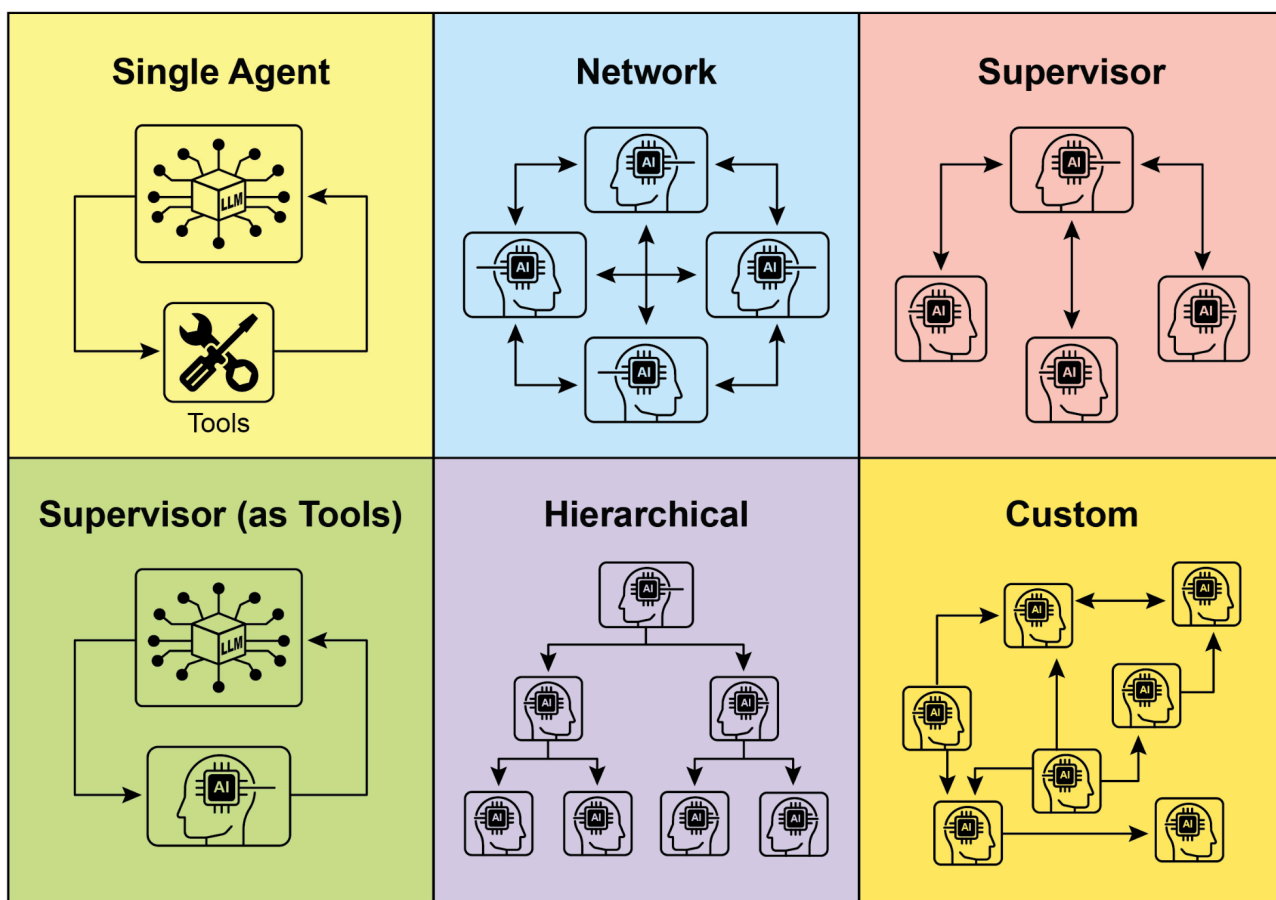
- OpenAI: Introducing Deep Research: <https://openai.com/index/introducing-deep-research/>
 - Perplexity: Introducing Perplexity Deep Research: <https://www.perplexity.ai/hub/blog/introducing-perplexity-deep-research>
-

第 7 章：多智能体协作（Multi-Agent Collaboration）

模式概述

对于定义清晰、流程固定的问题，单体智能体架构往往足够；但面对跨领域、复杂度高的任务时，单体能力会受限。多智能体协作模式通过将系统组织为由多个“具有明确分工的专用智能体”组成的协作体来化解这一限制。其基础原则是任务分解：将高层目标拆解为离散子问题，并把每个子问题分派给最适合的智能体（具备特定工具、数据访问能力或推理能力）。

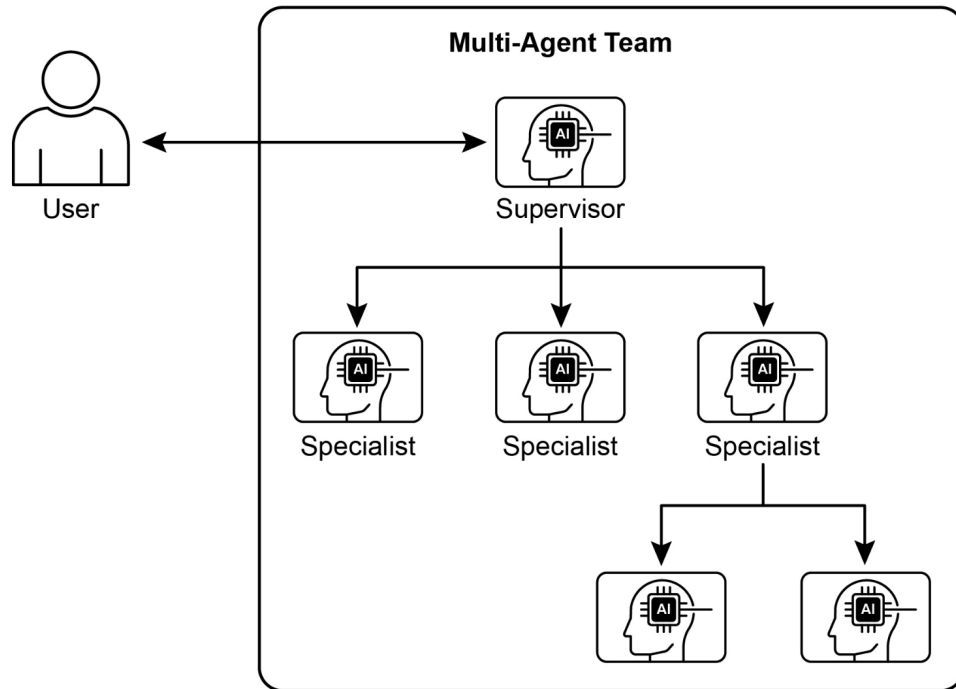
在此模式下，系统成效不仅来自“分工”，更取决于“智能体间的通信与协调”。这需要标准化的通信协议与共享本体（ontology），让智能体能交换数据、委派子任务、协调行动，确保最终结果一致且连贯。该分布式架构的优势包括更高的模块化、可扩展性与鲁棒性：单个智能体失败并不会使整个系统崩溃。协作使整体表现超过任何单个智能体的上限，产生协同效应。



协作形式（常见模型）

- 顺序接力（Sequential Handoffs）：一个智能体完成某步并将结果交给下一智能体继续处理（与“规划”模式相似，但明确涉及不同智能体）。
- 并行处理（Parallel Processing）：多个智能体同时处理问题不同部分，随后合并结果。
- 辩论与共识（Debate and Consensus）：不同视角与信息来源的智能体展开讨论，评估备选方案，最终达成更审慎的决策或共识。
- 层级结构（Hierarchical Structures）：管理者智能体根据工具/插件能力动态分派任务给工作智能体，并综合其结果。每个智能体可管理相关工具组，而非由单体承担全部工具。
- 专家团队（Expert Teams）：具备不同领域专长的智能体（如研究者、写作者、编辑）协作产出复杂成果。

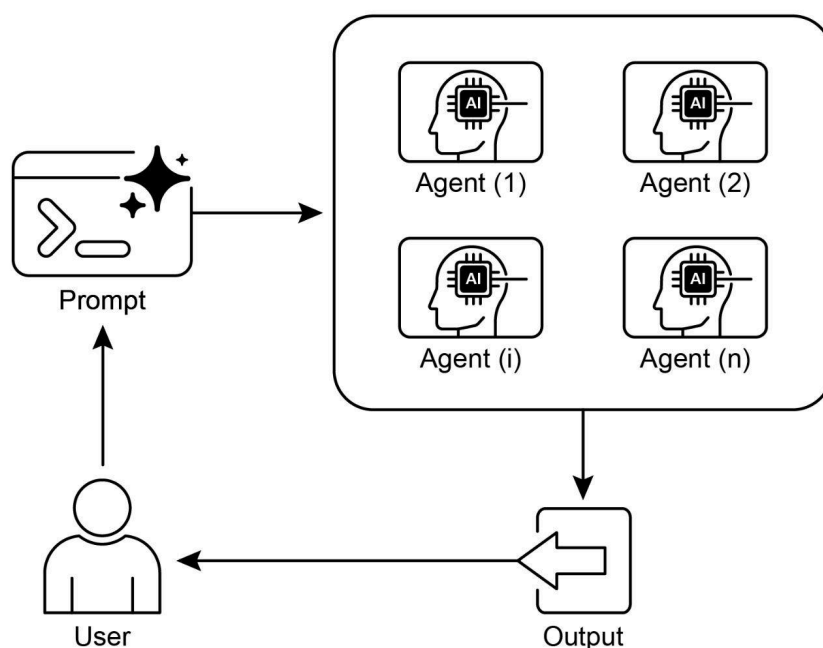
- 批评-审稿 (Critic-Reviewer)：一组智能体先产出计划/草稿/答案，另一组智能体对其进行政策、安全、合规、正确性、质量、组织目标等方面的评审；原作者或最终智能体依据反馈修订。这在代码生成、研究写作、逻辑校验与道德一致性方面尤为有效，可提升鲁棒性与质量、降低幻觉与错误。



一个多智能体系统通常包含：清晰的角色与职责划分、可见的通信通道用于信息交换、以及指导协作行为的任务流程或交互协议。

框架支持

- Crew AI：提供定义智能体、任务与交互流程的结构，使多阶段、并行处理与跨智能体信息校验更易于落地。
- Google ADK：为多智能体范式提供工程化支撑，可结构化指定智能体、任务与交互过程，适用于多阶段、跨专长与并行计算的挑战，并支持与传统 ML 工具链整合。



*Agents can have multiple agents connections.

应用场景

- 复杂研究与分析：团队式代理协同完成研究。一名负责信息检索，一名负责数据分析，一名负责综合与写作，联合产出结构化、可验证的报告。
- 软件开发协作：一个代理做需求与设计，另一个编码，第三个测试，第四个撰写文档；通过产物接力与校验构建并验证组件。
- 创意内容生成：营销活动可由市场研究代理、文案代理、图像生成代理与社媒排期代理共同完成。
- 金融分析：多智能体分工抓取行情、分析新闻情绪、进行技术与生成投资建议。
- 客服分层升级：一线客服代理处理基础问题，复杂问题按需升级到专家代理（如技术或账单专家），体现基于问题复杂度的顺序接力。
- 供应链优化：代理代表供应链各节点（供应商、制造商、分销商），协作优化库存、物流与排程，以应对需求变化或扰动。
- 网络分析与修复：在自治运维中，多智能体协作进行故障定位与修复建议，并与传统 ML/工具集成，兼具既有系统能力与生成式 AI 优势。

经验要点

- 明确分工与接口：为每个智能体定义角色、目标与可用工具，建立共享本体与通信协议。
- 编排与容错：设计任务流转（顺序/并行/层级/辩论），并设置失败隔离与重试机制。
- 数据一致性与可追溯：在跨代理交换数据时维持统一标注与版本，记录来源与决策脉络。
- 安全与治理：对跨智能体的权限、合规与审计进行治理，防范越权与信息泄露。
- 综合与收敛：在末端设置综合/总结代理，负责结果合并、冲突消解与统一输出格式。

小结

多智能体协作模式展现了在系统中编排多个专用智能体的显著优势。通过多种协作模型（顺序、并行、辩论、层级、专家、批评-审稿），该模式能更好地处理跨场景的复杂问题。对智能体协作的理解也自然引出其与外部环境交互的进一步探究与工程化实践。

参考资料

- Multi-Agent Collaboration Mechanisms: A Survey of LLMs: <https://arxiv.org/abs/2501.06322>
 - Multi-Agent System — The Power of Collaboration: <https://aravindakumar.medium.com/introducing-multi-agent-frameworks-the-power-of-collaboration-e9db31bba1b6>
-

第 8 章：记忆管理（Memory Management）

模式概述

高效的记忆管理是智能体保持信息与持续学习的关键。与人类相似，智能体需要不同类型的记忆以高效运作：短期（上下文）记忆用于当前互动的即时推理，长期（持久化）记忆用于跨会话、跨任务保留并复用知识。本章聚焦于两类核心记忆及其工程化实践。

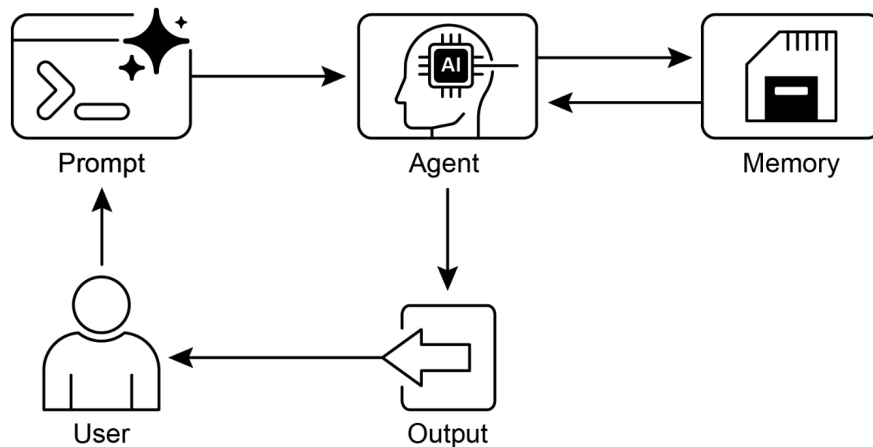
在智能体系统中，“记忆”指智能体保留并利用过往交互、观察与学习经验的能力。这有助于智能体做出更有依据的决策、维持对话上下文、并随时间提升表现。总体上可分为：

短期记忆（Contextual / Working Memory）

- 作用：保存当前正在处理或刚刚访问的信息，主要体现在 LLM 的上下文窗口（context window）中，包括近期消息、代理回复、工具调用结果、反思内容等。
- 约束：上下文窗口容量有限，需通过摘要老对话、突出关键细节等策略将最相关的信息保留在有限空间内。长上下文模型只扩大“短期记忆”容量，但依然是“会话内的临时记忆”，会话结束即消散，且重复处理成本高。

长期记忆（Persistent Memory）

- 作用：作为跨交互、跨任务或长时间段的知识库；通常存储在外部系统（数据库、知识图谱、向量数据库等）。
- 机制：向量库将信息转换为向量，支持基于语义相似度的检索（semantic search），而非仅靠关键词匹配。需要信息时，代理查询外部存储，检索到的相关数据再注入短期上下文供即时使用，从而把既有知识与当前互动结合。



应用场景

- 聊天机器人与对话式 AI：短期记忆维持对话连贯性；长期记忆使其能记住用户偏好、历史问题或讨论，为持续、个性化对话提供支持。
- 个人助理与生产力代理：记录日程、任务、偏好与 workflow 惯例，跨会话保持连续性。
- 企业知识管理与问答：将知识库作为长期记忆，通过 RAG（检索增强生成）检索相关文档与数据，提升回答准确性与可溯性。
- 自主系统：如机器人/自动驾驶需要对地图、路线、目标位置与已学行为进行记忆；短期用于即时环境，长期用于通用环境知识。

框架与实现

Google Agent Developer Kit (ADK)

- 核心概念与服务：
 - Session：代表单次对话线程，记录消息与动作（Events），并保存与该会话相关的临时数据（State）。
 - State (session.state)：会话内的“工作记忆”，以字典存储可序列化的键值数据（字符串、数值、布尔、列表、字典），随对话动态演进。
 - Memory：可搜索的持久化信息库，来源于历史会话或外部数据源，用于跨会话检索。
- SessionService 实现：
 - InMemorySessionService：用于本地开发与测试，不提供跨重启的持久化。
 - DatabaseSessionService：持久化到自管数据库（需配置连接与驱动），适用于生产或需要可靠存储的开发环境。

- VertexAiSessionService: 利用 Vertex AI 基础设施实现可扩展的云端生产级会话管理。
- 交互循环: 每次消息到达时, Runner 通过 SessionService 获取/建立 Session → 智能体基于 Session 上下文 (状态与历史) 处理消息 → 生成响应并可能更新状态 → Runner 记录为 Event, 并调用 `session_service.append_event` 写入存储 → 等待下一消息; 交互结束时调用 `delete_session` 终止会话。该流程通过管理会话历史与临时数据确保连续性。
- State 组织: 可通过键前缀划分作用域与持久性, 例如 `user:` 前缀关联特定用户的跨会话数据, `app:` 前缀标识跨会话共享的应用级数据。无前缀键通常仅作用于当前会话。

LangChain

- 提供实用记忆组件, 如 `ConversationBufferMemory` 自动注入单次对话历史到提示中, 帮助代理回忆即时上下文。

LangGraph

- 支持更高级的长期记忆: 通过存储 (store) 保存并检索语义事实、情节性经历或可更新的过程性规则, 跨不同用户会话复用。

Memory Bank (Vertex AI Agent Engine)

- 托管型服务, 为代理提供持久的长期记忆: 自动抽取、存储与召回用户特定信息, 支持在 Google ADK、LangGraph、CrewAI 等框架中实现个性化、连续对话。

小结

本章区分了短期上下文与长期知识的工程化管理, 并介绍了如何在智能体系统中建立、使用与维护它们。通过 ADK 的 Session/State/Memory 组合、LangChain 的会话记忆、LangGraph 的长期存储与 Memory Bank 的托管能力, 开发者可构建更聪明、更具连续性的智能体。接下来将进入“学习与适应 (Learning and Adaptation)”模式, 关注智能体如何基于新经验或数据调整思维、行为与知识。

参考资料

- ADK Memory: <https://google.github.io/adk-docs/sessions/memory/>
 - LangGraph Memory: <https://langchain-ai.github.io/langgraph/concepts/memory/>
 - Vertex AI Agent Engine Memory Bank: <https://cloud.google.com/blog/products/ai-machine-learning/vertex-ai-memory-bank-in-public-preview>
-

第 9 章：学习与适应 (Learning and Adaptation)

本章讨论代理如何通过学习与适应持续提升能力，使其在复杂、动态或未知环境中表现更稳健、更高效。学习让代理从数据与经验中归纳规律；适应让代理基于新信息调整策略、知识或目标，实现从“按指令行事”到“随时间变聪明”的演进。

模式概述

- 核心目标：让代理通过经验与反馈迭代更新策略、知识库与行为，使其在不确定环境下保持稳健表现。
- 关键能力：自我改进、偏好对齐、在线更新、上下文记忆、策略稳健性、风险控制与回滚、评估与度量。
- 适用场景：实时数据驱动决策、连续交互任务、个性化系统、具备长期目标的自治体、需要对齐人类偏好的大型语言模型 (LLM)。

大局观 (学习范式)

代理通过学习改变“思考—行动—知识”的结构，并据此适应环境。常见范式包括：

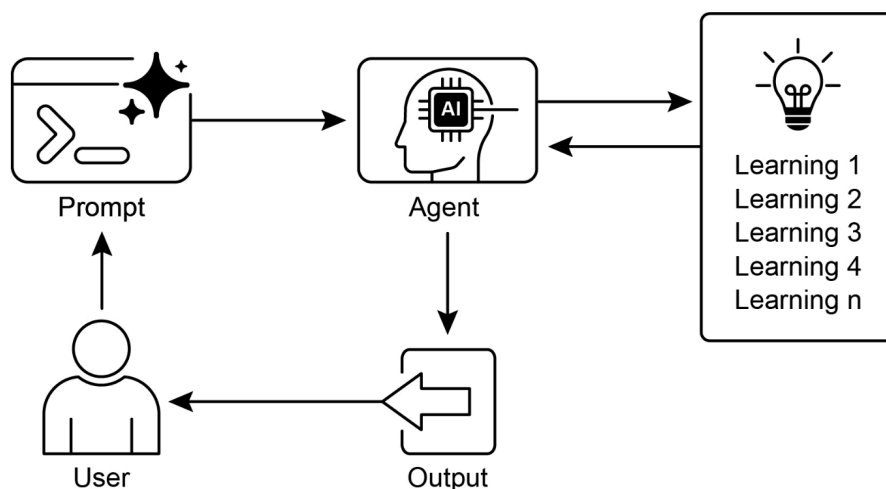
- 强化学习 (Reinforcement Learning)：通过“试错—奖励/惩罚”学习最优策略，适用于机器人控制、游戏与交互式任务。
- 监督学习 (Supervised Learning)：从标注数据学习输入到输出的映射，适用于分类、预测与模式识别（如邮件分类、趋势预测）。
- 无监督学习 (Unsupervised Learning)：在无标注数据中发现潜在结构与关联，帮助构建环境的“心智地图”，适用于探索式分析。
- 小样本/零样本学习 (Few-shot/Zero-shot, 基于 LLM)：借助 LLM 的泛化能力，通过少量示例或清晰指令快速适应新任务。
- 在线学习 (Online Learning)：随数据流持续更新参数与知识库，适用于需要实时反应的动态环境。
- 基于记忆的学习 (Memory-Based Learning)：回忆过往经验并在相似情境调整当前行为，强化上下文敏感性与决策质量。

适应机制与算法

适应指代理基于学习结果改变策略、理解或目标，是在不可预测或变化环境中保持有效性的关键。以下两类在工程实践中尤为重要：

- 近端策略优化 (PPO, Proximal Policy Optimization)：
 - 目标：在连续动作空间等场景中稳定提升策略表现。
 - 思想：使用“截断”目标函数限制更新步长，维持策略在可信区域内变化，避免性能骤降。
 - 流程：交互收集经验 → 评估代理优势与替代目标 (surrogate) → 通过剪切 (clipping) 约束更新 → 重复迭代，兼顾提升与稳定。
- 直接偏好优化 (DPO, Direct Preference Optimization)：
 - 目标：更直接地将 LLM 与人类偏好对齐，减少传统 RLHF 流程复杂性与不稳定性。
 - 对比：传统两步法（训练奖励模型 → 用 PPO 微调 LLM）复杂且可能出现“奖励黑客”；DPO 通过偏好对比信号直接优化生成分布，更简洁稳健。
 - 适用：对齐对话风格、安全性、帮助性与一致性，尤其在大规模人类反馈数据下效果显著。

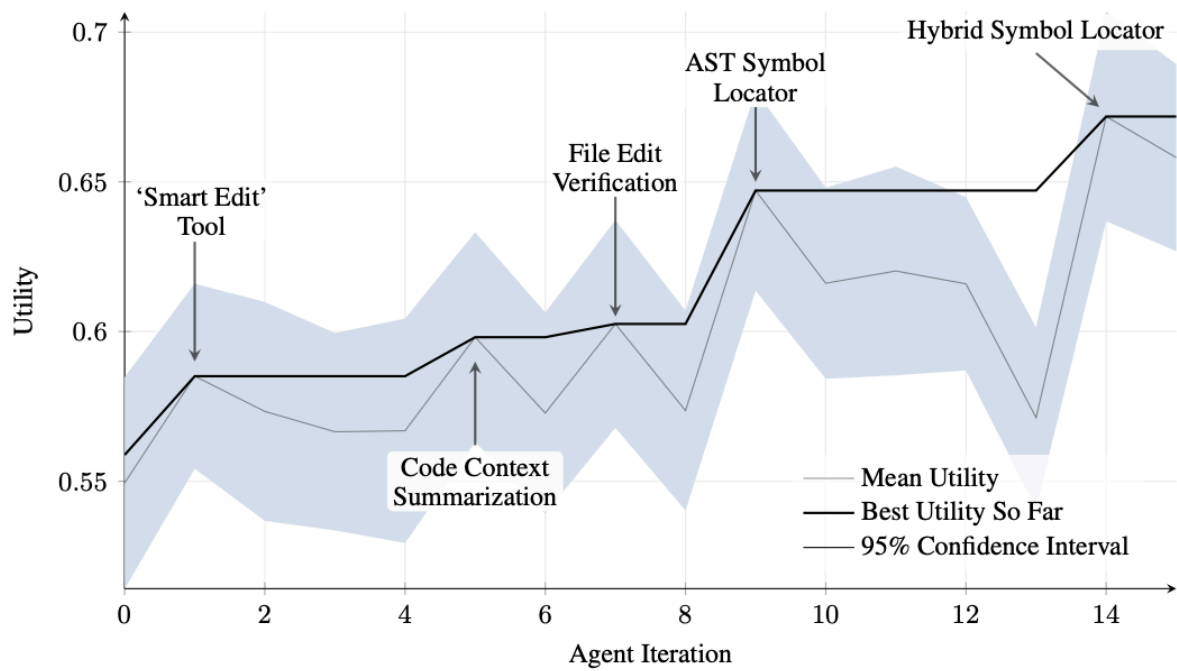
框架与实现



- 强化学习与策略优化：可结合 Gymnasium/RLlib 等进行环境交互训练；在代理系统中通过“策略模块 + 评估反馈 + 安全约束”闭环接入。
- 偏好对齐与人类反馈：DPO/RLHF 工作流可用于对齐 LLM 输出偏好；在应用层结合排名数据、二选一比较或显式反馈收集。
- 检索增强与知识更新（RAG）：通过把成功策略与问题-解答对存入动态知识库，在推理时进行检索增强，实现“知识驱动的适应”。
- 记忆系统：短期对话上下文 + 长期外部知识库（向量检索/结构化数据库），统一在 Memory 服务层管理读写与过期策略。

应用场景

- 个性化助理：基于长期行为分析优化交互协议与回应风格，持续提升满足度与效率。
- 交易机器人：在高频、实时行情中动态调参，优化收益—风险比，维持稳健性。
- 应用代理：根据观察到的用户行为自适应界面与功能，提高参与度与直觉性。
- 机器人与自动驾驶：融合传感器数据与历史行动，提升导航与响应能力，确保安全与效率。
- 欺诈检测：以新识别的异常模式持续更新模型，降低误报与漏报、提升安全性。
- 推荐系统：学习用户偏好与情境，提升内容选择的个性化与相关性。
- 游戏 AI：动态调整策略以匹配玩家水平与偏好，增强挑战与参与感。
- 知识库学习代理：结合 RAG 维护“问题—解法”动态库，在决策时引用成功范式或规避已知陷阱（参见第 14 章）。

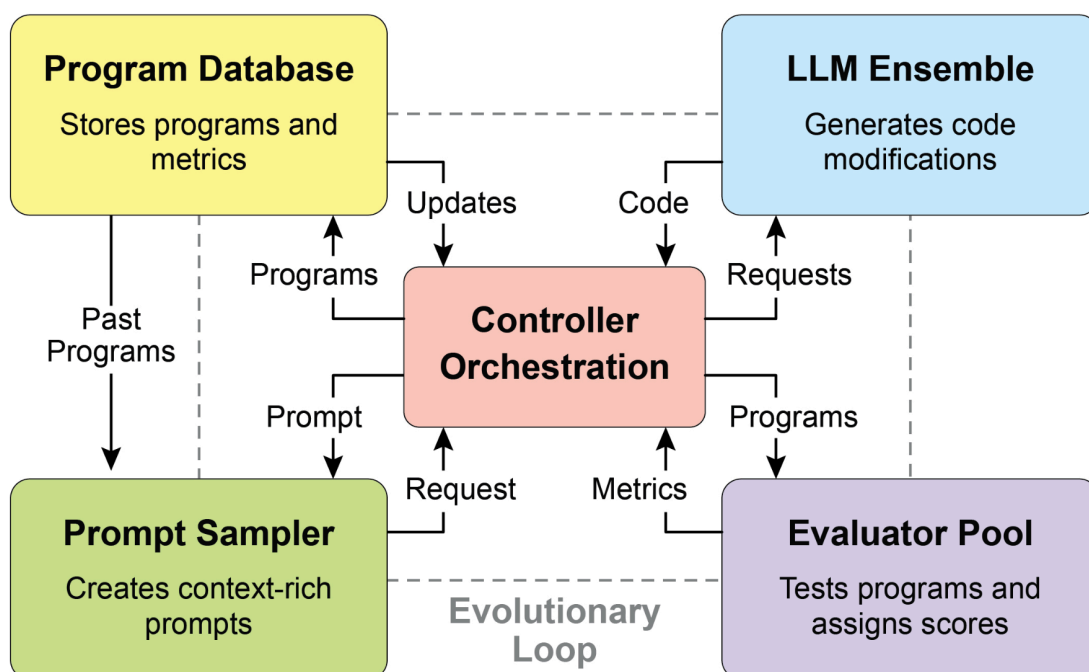


案例研究：自我改进编码代理（SICA）

“Self-Improving Coding Agent (SICA)”展示了代理如何通过自动化评估与迭代修改，实现能力的自主提升：

- 目标：让编码代理在持续迭代中提升代码质量与任务成功率。
- 方法：对生成的代码进行自动化测试与指标评估，记录失败原因与上下文线索，并基于反馈生成有针对性的代码修补（patch）。
- 机制：引入演化式搜索与自我反思（如错误模式记忆、修复策略归纳），逐步形成更稳健的编码行为策略。
- 成果：展现出无需人工密集监督的持续改进能力，验证了“学习—评估—修复”闭环在代理系统中的有效性。

OpenEvolve Architecture



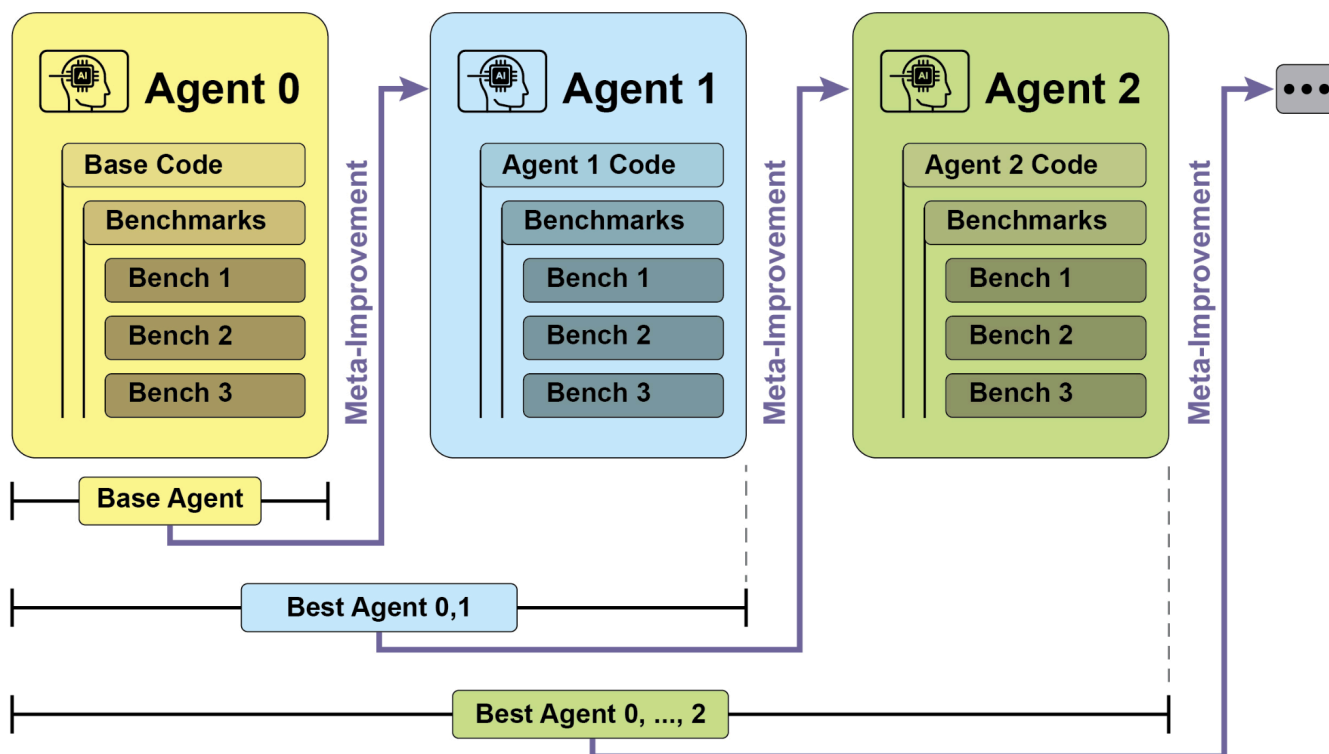
Asynchronous Pipeline Optimized for Maximum Throughput

工程实践要点

- 数据与反馈质量：偏好数据、评测样例与奖励设计决定学习上限；确保覆盖代表性、避免诱导投机策略。
- 稳定性与安全约束：策略更新需设“可信区间”（如 PPO clipping）、失败回滚与熔断，防止性能坍塌。
- 记忆与持久化：统一短期上下文与长期知识的读写策略，设计过期/压缩机制以控资源与噪声。
- 离线评估 + 在线监控：建立基准集、A/B 测试与观测指标（质量、安全、鲁棒性、延迟、成本），持续校准。
- 人类在环与对齐：对于风险敏感领域，保留人工审核通道；在对话与内容生成环节确保与人类偏好一致。
- 多智能体协同学习：在协作系统中记录完整交互轨迹，精确标注各参与代理的输入/输出，以便联合调优与稳定改进。

结论

学习与适应是构建高级代理系统的关键，使其能在开放环境中稳健运行并持续提升。结合规划（第 6 章）、多智能体协作（第 7 章）、记忆管理（第 8 章）等模式，可实现更强的模块化、可扩展性与自我进化能力。诸如 AlphaEvolve 的进展表明，基于 LLM、自动评估与演化策略的组合，代理可以实现算法的自主发现与优化，为科研与工程带来新的可能。



参考文献

- Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction. MIT Press.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- Mitchell, T. M. (1997). Machine Learning. McGraw-Hill.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
- Robeyns, M., Aitchison, L., & Szummer, M. (2025). A Self-Improving Coding Agent. arXiv:2504.15228. <https://arxiv.org/pdf/2504.15228>; 源码: https://github.com/MaximeRobeyns/self_improving_coding_agent
- AlphaEvolve 博文: <https://deepmind.google/discover/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/>
- OpenEvolve 项目: <https://github.com/codelion/openevolve>

插图与流程草图 (Mermaid)

以下草图使用 Mermaid 表示，帮助直观把握第 9 章关键流程。如你的渲染器不支持 Mermaid，可将代码粘贴到支持 Mermaid 的编辑器或渲染器中查看。

学习—适应闭环

```

flowchart LR
    A[感知/数据输入] --> B[学习机制\n( 监督/强化/无监督/在线)]
    B --> C[策略/知识更新]
    C --> D[适应\n( 策略调整/偏好对齐/记忆检索)]
    D --> E[行为执行]
  
```

E --> F[反馈/评估\n(奖励/偏好/指标)]
F --> B

PPO 训练流程（可信更新）

```
flowchart LR
    subgraph Rollout[环境交互]
        S[状态 s] --> P[策略 π(a|s)]
        P --> A[执行动作 a]
        A --> R[奖励 r, 状态 s']
    end
    end
    R --> GAE[优势估计/价值评估]
    P --> OldPi[旧策略]
    GAE --> CLIP[截断目标 L^CLIP\n(可信区间/步长约束)]
    OldPi --> CLIP
    CLIP --> Update[策略更新\n(梯度下降)]
    Update --> P
```

DPO 偏好对齐流程（无需奖励模型）

```
flowchart LR
    Pref[人类偏好比较\n(A 优于 B)] --> Loss[对比损失]
    Policy[LLM 策略 π] --> Loss
    Ref[参考模型 π_ref] --> Loss
    Loss --> Update[更新策略 π]
    Update --> Policy
```

记忆—适应协同（RAG + 记忆服务）

```
flowchart TB
    STM[短期记忆\n(会话上下文)] --> RAG[检索增强]
    LTM[长期记忆/知识库\n(向量/结构化)] --> RAG
    RAG --> Policy[策略/生成]
    Policy --> Out[输出]
    Out --> FB[反馈/度量]
    FB --> Write[写入/压缩/过期策略]
    Write --> STM
    Write --> LTM
```

SICA 自我改进闭环（编码代理）

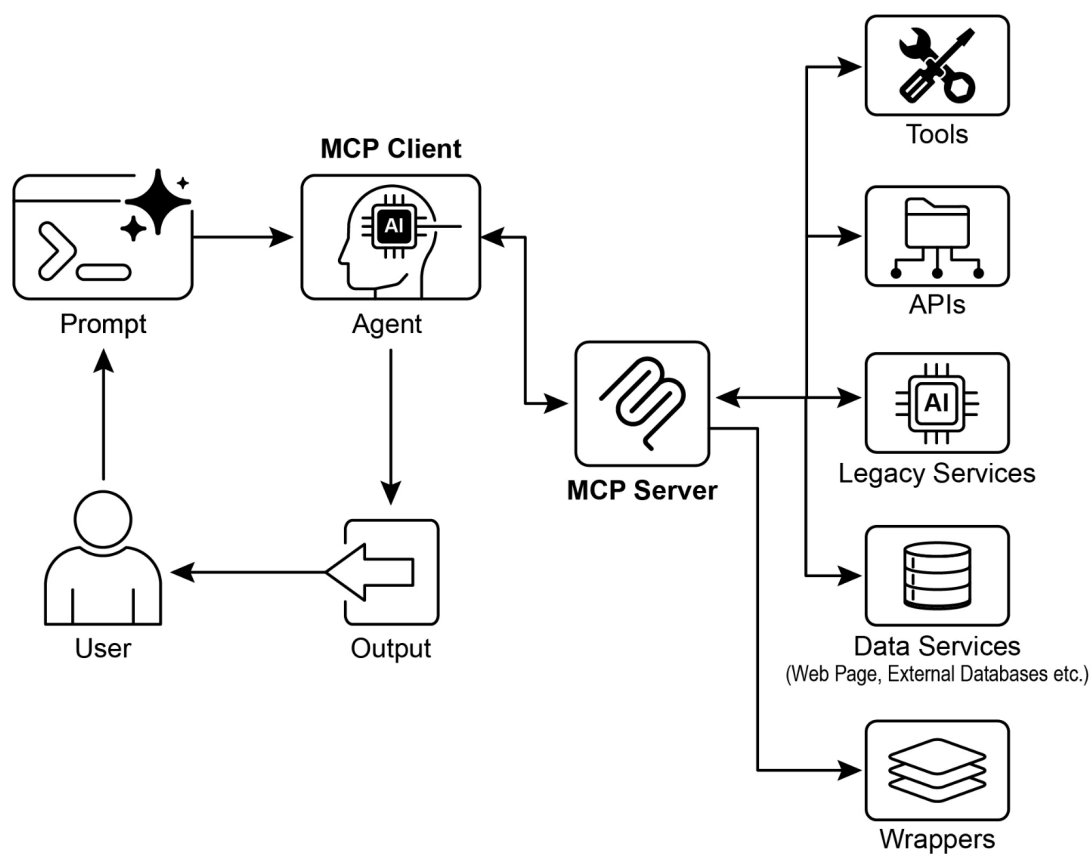
```
flowchart LR
    Gen[生成代码/补丁] --> Test[自动化测试/静态分析]
    Test --> Eval[指标评估/错误归因]
    Eval --> Reflect[自我反思/模式归纳]
    Reflect --> Patch[生成修复策略/补丁]
    Patch --> Gen
    Eval --> Memory[存储失败模式/成功策略]
    Memory --> Reflect
    Reflect --> Stop{收敛/停止条件?}
    Stop -- 否 --> Gen
    Stop -- 是 --> Done[结束/发布]
```

第 10 章：模型上下文协议（Model Context Protocol, MCP）

要让大型语言模型（LLM）有效充当“代理”，其能力必须超越多模态生成，能够与外部世界交互：访问最新数据、调用外部软件、执行具体操作任务。模型上下文协议（MCP）通过开放、标准化的接口，让 LLM 与外部资源一致、可预期地连接，是构建交互式 AI 代理的关键机制。

模式概述

- 定位：为 LLM/代理提供通用“适配器”，连接任何合规的系统、数据库或工具，无需为每种集成单独定制。
- 架构：典型的客户端—服务器（Client—Server）。MCP 服务器对外暴露资源（resources）、提示模板（prompts）与工具（tools）；MCP 客户端（LLM 宿主应用或代理）发现并消费这些能力。
- 目标：标准化“发现—通信—调用”，降低集成复杂度，提升互操作性、可组合性与复用性，使代理能够在真实系统中读取、写入与执行。



核心组件

- 资源（Resources）：可读数据源，如文件系统、数据库、知识库、API 返回数据等。
- 提示模板（Prompts）：交互模板或指令骨架，用于结构化地为 LLM 构造上下文与调用意图。
- 工具（Tools）：可执行的函数/操作，封装外部能力（如发起查询、修改数据、调用服务等）。
- 客户端（Client）：LLM 宿主或代理运行环境，负责能力发现、参数编排与结果消费。
- 服务器（Server）：对外暴露上述能力的 MCP 服务端，可由现有系统包装或专门实现。

设计注意事项

- 代理友好的 API：MCP 是“代理接口”的契约，但不强制数据可理解性。若底层 API 返回 PDF 等不易解析格式，代理难以利用；更佳实践是提供文本/结构化（如 Markdown/JSON）版本。
- 决定性支持：代理是“非确定性”工作流的编排者，并不替代确定性能力。若工单系统 API 只能逐条拉取详情，摘要高优先级工单将低效且易错；应为 API 增强过滤、排序等确定性特征，以助代理高效工作。

MCP 与工具函数调用的对比

- 工具函数调用：LLM 直接调用应用内预定义函数/工具，通常为一对一的特定请求，机制偏厂商私有。
- MCP：开放协议，面向“发现—通信—调用”的标准化接口，致力于任何合规工具可被任何合规 LLM 访问，促进跨系统互操作与组合。通过“联邦式”接入，可把异构、遗留服务包装进现代生态，并在新应用/工作流中进行组合协作。

与框架的结合与实践

- ADK (Agent Development Kit)：
 - 支持消费现有 MCP 服务器暴露的能力；
 - 支持将 ADK 内的工具通过 MCP 服务器对外暴露。
- FastMCP：
 - 简化 MCP 服务器的开发与管理，尤其适合以 Python 实现并对外暴露工具；
 - 便于快速把已有能力包装为 MCP 工具集合。
- MCP Tools for Genmedia (Google Cloud)：
 - 将 Imagen、Veo、Chirp 3 HD、Lyria 等生成式媒体能力以 MCP 工具形式提供；
 - 代理可编排生成媒体的端到端流程（资源管理、提示模板、调用与审核）。
- MCP 数据库工具箱：
 - 为数据库交互提供规范化工具接口（查询、模式发现、读写等）；
 - 支持将数据访问纳入统一的资源—工具视图，便于代理安全地执行读写操作。
- 文件系统服务器（示例）：
 - 通过 MCP 暴露文件浏览、读取与写入等能力；
 - 代理可在受控环境中管理项目资产、生成报告或更新配置。

应用场景

- 与业务系统联动：客服、工单、CRM、财务等系统的统一接入与编排。
- 数据工作流：以资源—工具的统一接口组织检索、清洗、转换、入库与分析。
- 生成式媒体：图片/视频/音频的生成、审核与发布流水线编排。
- 物联网与自动化：控制设备、采集传感数据、执行自动化操作与告警。

结论

MCP 为 LLM 与外部系统之间提供开放标准与客户端—服务器架构，统一“资源—提示—工具”的暴露与消费。通过与 ADK、FastMCP 等生态结合，开发者能构建超越语言生成的交互式代理，访问动态数据、控制工具与执行真实操作，实现数据库管理、生成媒体、IoT 控制、金融自动化等复杂场景的可组合工作流。

参考文献

- Model Context Protocol (MCP) 文档：<https://google.github.io/adk-docs/mcp/>

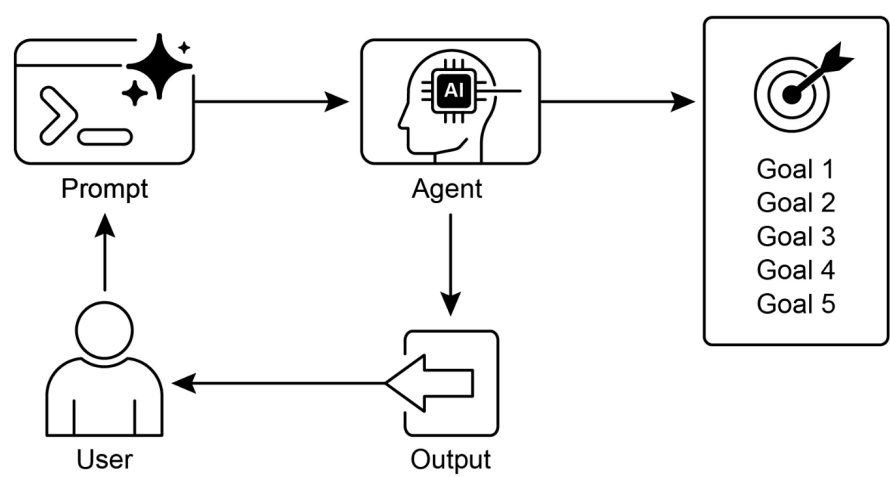
- FastMCP 文档: <https://github.com/jlowin/fastmcp>
 - MCP Tools for Genmedia Services: <https://google.github.io/adk-docs/mcp/#mcp-servers-for-google-cloud-genmedia>
 - MCP Toolbox for Databases 文档: <https://google.github.io/adk-docs/mcp/databases/>
-

第 11 章：目标设定与监控（Goal Setting and Monitoring）

要让代理系统真正“有目的、有担当”，仅有感知、推理与工具调用还不够。代理需要清晰的目标与可靠的监控机制，才能判断是否在正确轨道上，并据此调整策略、修正偏差或触发升级。本章阐述目标设定与监控的核心理念、设计范式、工程落地与框架实践。

模式概述

- 目标设定：为代理明确“期望达成的状态”（目标状态）及其约束与边界（时间、资源、质量、风险）。
- 监控执行：持续观察代理行为、环境状态与工具输出，依据指标与判据判断进度是否达标。
- 反馈闭环：将监控信号转化为计划修订、重试/降级、资源再分配或人工升级，保证稳健达成目标。
- 工程价值：让代理从“被动反应”转为“主动达成”，实现可测量、可解释与可问责的自治运行。



关键理念

- SMART 目标：具体（Specific）、可度量（Measurable）、可实现（Achievable）、相关（Relevant）、有时限（Time-bound）。
- 成功判据与度量：为每个目标设定清晰指标与阈值（KPI/SLI/SLO），避免模糊化导致监控失效。
- 监控范围：既关注代理行为（动作序列、工具调用）、也关注外部环境（状态变化、事件流）、还要检查输出质量（正确性、一致性、安全性）。

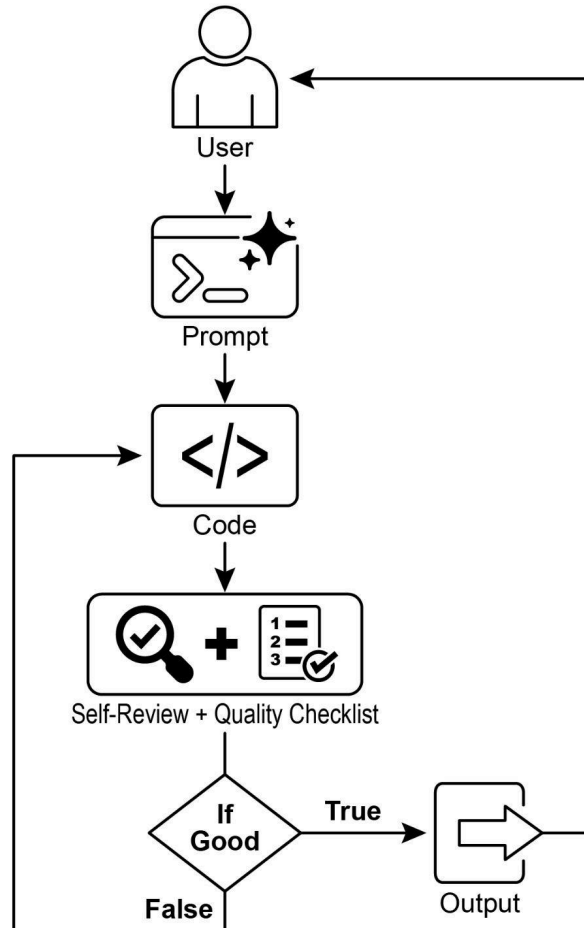
- 反馈机制：基于监控信号触发自适应，包括计划修订、策略切换、降级与回滚、告警与升级、多人/多智能体协作。
- 框架实践（以 Google ADK 为例）：目标通常经由 agent 指令传达；监控通过 state 管理与工具交互实现（记录上下文、评估进度、触发流程变更）。

目标设计范式

- 层级目标：顶层目标拆解为子目标/里程碑，便于并行推进与阶段性验收。
- 检查点与里程碑：在关键节点进行验收与风险评估，必要时重构计划或重新分配资源。
- 阈值触发：为延迟、错误率、成本等设阈值与告警策略，超阈触发降级或升级。
- 资源/约束对齐：将时间、预算、配额、合规要求建模为目标的硬/软约束，纳入监控判据。
- 优先级与重排：动态重排目标与子目标的优先级，维持整体收益最大化或风险最小化。

监控与反馈闭环

- 观测信号：
 - 行为层：调用链路、动作成功率、重试次数、工具返回码。
 - 环境层：外部状态变更、事件队列、数据质量与新鲜度。
 - 结果层：正确性、一致性、覆盖率、安全性、用户满意度。
- 判据与决策：用阈值/区间、规则/约束、统计与模型评估将信号映射为决策（继续、修订、降级、升级）。
- 自适应动作：
 - 计划修订（重新规划、路径替换、并行化/串行化调整）。
 - 鲁棒策略（重试、回滚、冗余、熔断、隔离）。
 - 升级/协作（人工在环、多智能体协同、跨团队转派）。



框架与实现（ADK 示例）

- 指令承载目标：目标与约束在 agent 指令中表达（角色/上下文/意图/成功条件）。
- 状态管理（State）：记录会话/任务进度、里程碑达成、失败原因与下一步动作建议。
- 工具交互：将监控与工具调用打通（如查询状态、拉取日志、执行修复），形成“检测—行动”一体化闭环。
- 记忆/持久化：在 Memory/Session 中保留历史轨迹与关键信号，支持长期评估与复盘改进。

应用场景

- 客服自动化：以首次响应时间、解决率与满意度为目标与监控指标，超阈自动升级或转派。
- 运营与流程自动化：以时限、质量与成本为约束，监控异常并触发补偿/回滚策略。
- 机器人与自动化系统：设定任务完成度与安全边界，实时监控传感器数据与行为偏差。
- 数据管道与分析：以数据新鲜度/完整性/准确度为指标，异常时降级或重跑与告警。

概念性实现示例（框架化思路）

- 目标声明：在 agent 指令中定义目标、里程碑、约束与成功判据。
- 状态建模：以 state 记录当前进度、剩余任务与风险。
- 监控执行：绑定工具调用与监控探针，持续采集信号并评估。
- 自适应策略：依据判据触发重试/降级/回滚或人工升级；必要时重新规划。
- 复盘与留存：将关键轨迹与结果写入 Memory/Session，用于长期优化与审计。

关键点总结（节选）

- 目标设定与监控为代理赋予“方向”与“度量”。
- 目标应符合 SMART 原则，成功判据必须清晰可测。
- 监控需覆盖行为、环境与结果三层信号。
- 反馈闭环促进自适应（修订计划、降级/回滚、升级问题）。
- 在 ADK 中，目标多由指令承载；监控通过 state 与工具交互完成。

结论

目标设定与监控让代理从“仅响应”走向“主动达成与可问责”。通过清晰目标、严格监控与有效反馈闭环，代理在多域场景（如客服、流程自动化、机器人）中实现可靠的自治运行。采用结构化框架（指令 + 状态 + 工具 + 记忆）可将目标的表达、执行与评估统一起来，支撑面向质量与安全的工程实践。

参考文献

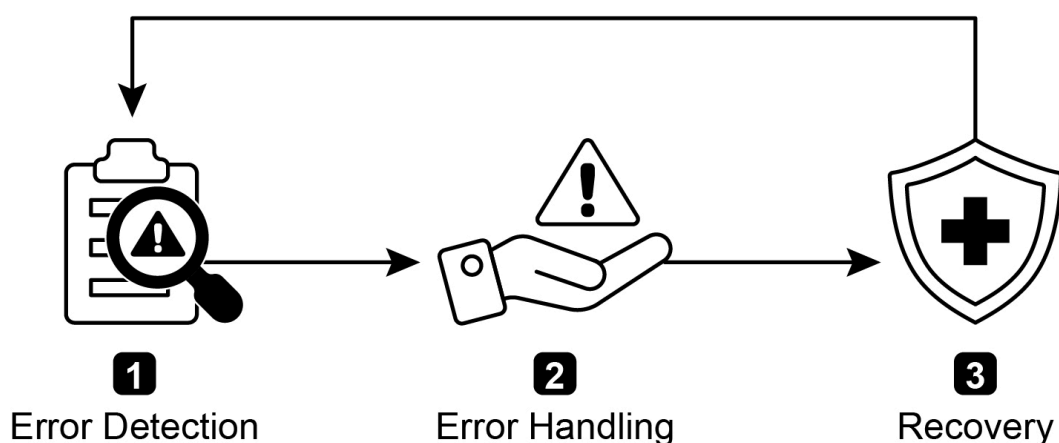
- SMART Goals Framework: https://en.wikipedia.org/wiki/SMART_criteria
-

第 12 章：异常处理与恢复（Exception Handling and Recovery）

真实世界充满不确定性与故障源。要让代理可靠地运行，必须具备发现异常、优雅处理并恢复到稳定状态的能力。本章阐述异常处理与恢复模式，强调面向复杂环境的鲁棒性与可用性工程。

模式概述

- 目标：开发具备高耐久性与高弹性的代理，即便出现错误与异常也能保持功能与完整性。
- 方法：在执行链路中引入异常检测、处理策略与恢复流程，同时支持受控失败（controlled failure）。
- 价值：提升稳定性与可信度，便于在复杂、不可预测场景中部署与扩展。

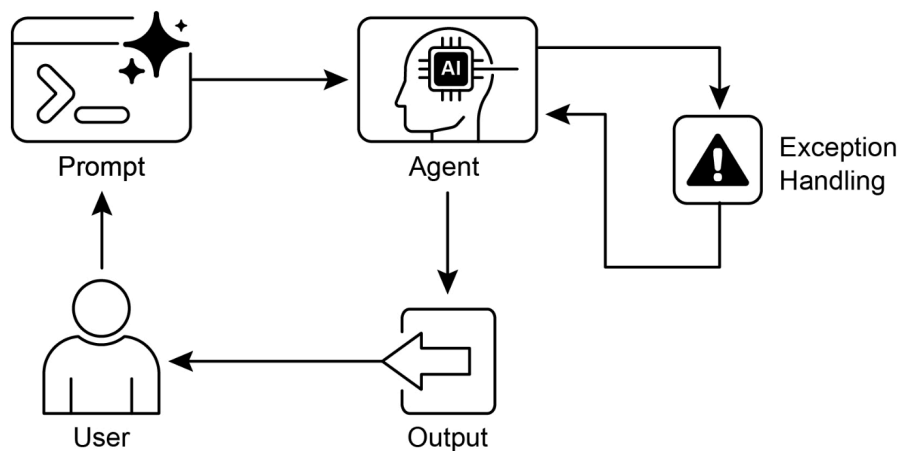


关键能力

- 异常检测：
 - 验证工具输出（结构/语义/一致性）。
 - 检查 API 错误码与返回体。
 - 设置超时与熔断，防止阻塞与雪崩。
- 异常处理：
 - 记录日志与诊断信息（便于复盘与审计）。
 - 重试与退避（指数退避、最大重试次数）。
 - 回退/兜底（fallback）、优雅降级（graceful degradation）。
 - 通知与告警（人类在环或跨系统协作）。

- 恢复策略：
 - 故障定位与诊断，进行自我修正或替代路径执行。
 - 状态恢复（快照/检查点/回滚）。
- 升级（escalation）：将复杂或高风险异常转交更强能力或人工通道。

提示：该模式可与“反思（Reflection）”结合使用。首轮失败产生异常后，反思过程分析原因并更改策略，进行有针对性的重试。



工程实践

- 观测与监控：统一采集与聚合指标（错误率、延迟、丢包、资源占用），建立异常告警与追踪体系。
- 设计约束：明确超时、重试、熔断、隔离与配额策略，防范级联故障。
- 数据与状态：保持操作幂等性；关键事务采用检查点与补偿交易，避免半成品状态。
- 安全与合规：为故障路径设定权限与审计；在恢复中避免泄露或越权操作。
- 受控失败：当无法恢复时，确保有界、可解释的失败行为，避免扩大影响面。

应用示例（概念）

- 服务编排：调用外部 API 超时 → 退避重试 → 超阈降级到缓存数据 → 记录告警并通知人类在环处理。
- 数据管道：某步校验失败 → 回滚到上一个检查点 → 修复规则或跳过坏数据 → 恢复流水线。
- 机器人任务：传感器读数异常 → 切换冗余传感器或安全模式 → 重新校准或请求人工协助。

结论

异常处理与恢复是构建“稳健且可信”代理的基础。通过检测、处理与恢复的完整闭环，并与监控/诊断系统协同，代理能在动态与复杂环境中保持可用性与一致性，显著提升自治系统的可靠性与可部署性。

参考文献

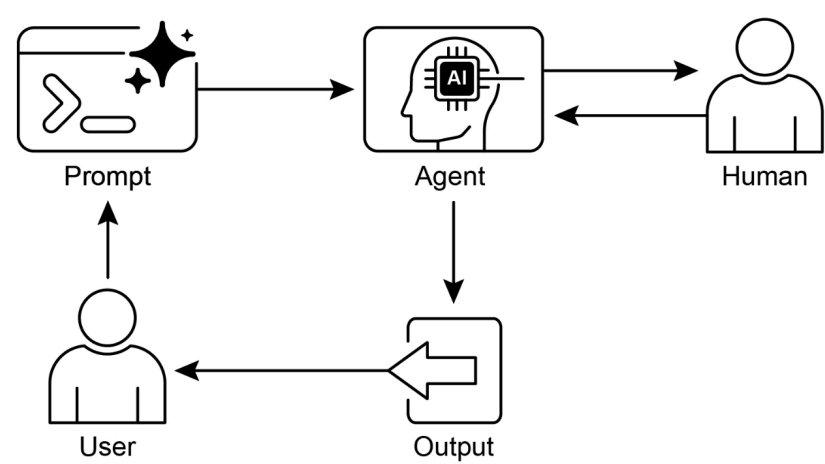
- McConnell, S. (2004). Code Complete (2nd ed.). Microsoft Press.
 - Shi, Y., Pei, H., Feng, L., Zhang, Y., & Yao, D. (2024). Towards Fault Tolerance in Multi-Agent Reinforcement Learning. arXiv:2412.00534.
 - O'Neill, V. (2022). Improving Fault Tolerance and Reliability of Heterogeneous Multi-Agent IoT Systems Using Intelligence Transfer. Electronics, 11(17), 2724.
-

第 13 章：人类在环（Human-in-the-Loop, HITL）

“人类在环”是构建健壮、安全、合乎伦理且高效的代理系统的关键策略。它将人类的判断力、创造性与细腻理解，与 AI 的计算能力与自动化效率相融合；在高复杂度与高风险领域尤为必要。

模式概述

- 核心原则：确保 AI 在伦理与安全边界内运行，同时以最佳有效性达成目标；当场景存在重大风险或歧义时，完全自治往往并不稳妥。
- 价值定位：不是“替代人类”，而是“增强人类”。通过自动化例行任务、提供数据驱动洞见、辅助决策，实现人机协作的协同效应。
- 应用图景：内容审核、医疗辅助诊断、自动驾驶的人工接管、客户支持升级等，均需人类监督与干预。



关键能力与机制

- 人类监督 (Oversight)：为关键步骤配置人工审阅与批准门槛，确保质量与合规。
- 人类干预 (Intervention)：在异常或风险状态下直接介入，进行修正或接管执行。
- 反馈用于学习 (Feedback for Learning)：将人工反馈转化为偏好对齐或数据标注，促进持续改进。
- 决策增强 (Decision Augmentation)：用 AI 提供数据洞见、备选方案与风险评估，辅助人类作出更优决定。
- 升级策略 (Escalation Policies)：明确何时、以何规则将任务或异常交由人类处理（阈值、类型、优先级）。

约束与挑战

- 可扩展性：HITL 天生难以无限扩展，存在准确性与吞吐量的权衡。
- 专家依赖：有效干预依赖领域专家，增加人力与培训成本。
- 操作挑战：需训练人工操作员生成高质量数据；敏感信息需匿名化以满足隐私要求。

实施范式

- 审阅/门控 (Review/Gating)：在生成或决策环节设置人工批准步骤。
- 标注/纠错 (Annotation/Correction)：用人工标注与更正输出，作为模型迭代的高质量数据源。
- 阈值升级 (Threshold-based Escalation)：当错误率、风险评分或不确定性超阈时转交人类。
- 人机共创 (Co-creation)：AI 生成草稿，人类完善与定稿；适用于复杂内容与设计流程。
- 渐进自动化 (Progressive Automation)：先以 HITL 保证质量，随着能力提升逐步扩大自动化范围。

框架实践（概念）

- 在代理开发工具（如 ADK）中，可通过升级机制/回调接口实现人与代理的交互：当监控触发升级规则时，将会话与上下文移交人工审阅模块，审阅结果再回写状态与记忆，继续流程。

工程要点

- 观测与审计：记录人与代理交互轨迹，确保可追溯与合规。
- 隐私与安全：对敏感数据进行脱敏/最小化访问；权限与审批链路明确。
- 体验与效率：为人工操作提供高效的界面与上下文聚合，减少认知负担与切换成本。
- 度量与改进：监控命中率、审阅时延、干预成功率与长期质量提升，迭代优化。
- 排队与调度：针对升级请求进行队列管理与负载均衡，确保 SLA。

结论

HITL 在复杂与敏感场景中确保系统的鲁棒性与可信性，是负责任 AI 的基石。通过人类监督、干预与反馈的系统化集成，代理性能与可解释性显著提升；即便在 AI 能力不断增强的未来，人类价值与专业知识仍应处于智能系统设计的中心位置。

参考文献

- A Survey of Human-in-the-loop for Machine Learning. Xingjiao Wu, Luwei Xiao, Yixuan Sun, Junhang Zhang, Tianlong Ma, Liang He. <https://arxiv.org/abs/2108.00941>
-

第 14 章：知识检索（RAG）

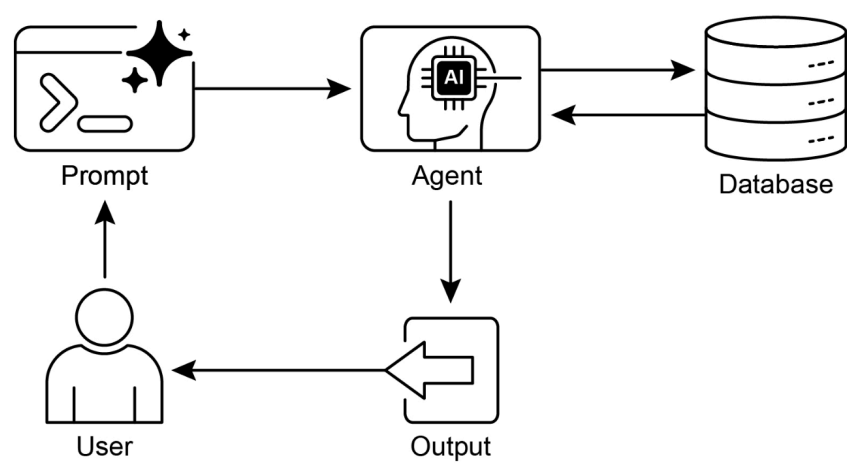
概述

大型语言模型（LLM）具备强大的文本生成能力，但其知识通常受限于训练数据，难以访问最新信息、企业内部资料或高度专业化细节。检索增强生成（Retrieval-Augmented Generation, RAG）通过在生成前检索并注入外部、实时、上下文相关的信息，显著提升答案的准确性、相关性与可验证性。

对代理（Agent）而言，RAG 尤其关键：它让代理能够以可验证的数据为依据执行复杂任务，例如基于最新政策回答问题、在下单前核验库存、或引用内部维基与文档提供依据。RAG 将“闭卷”的对话式模型转变为“开卷”的推理工具，使其能够完成更具价值的工作。

模式总览

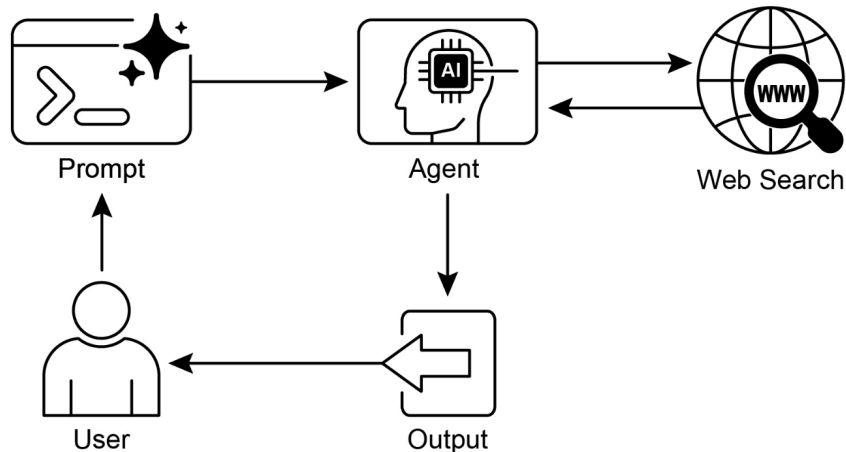
- 目标：在生成阶段前加入面向任务的外部知识，使输出“有据可依”。
- 要点：语义检索替代单纯关键词匹配；将检索片段注入到提示（prompt）中；生成时显式引用来源以增强信任。
- 效益：获取最新信息；降低幻觉风险；利用企业内知识；提供“引用/证据”以增强可审计性与可解释性。



典型流程

1. 接收查询：用户问题或代理任务被规范化为检索意图。
2. 语义检索：在外部知识库/文档集上进行语义搜索（可混合关键词检索）。

3. 候选片段：抽取最相关的文本片段（chunks），附带元数据（来源、时间、作者、权限等）。
4. 提示增强：将检索片段与任务约束注入提示，形成“检索增强的提示”。
5. 生成回答：LLM 基于增强提示生成答案，遵循任务格式与质量标准。
6. 引用与证据：返回时附上来源链接、片段 ID 或文档路径，支持审计与复核。



核心概念

- 向量嵌入（Embeddings）：将文本映射到高维向量空间，语义相近的文本距离更近，用于语义检索与相似度度量。
- 文本相似度：衡量文本在语义空间的接近程度，支持检索候选的排序与去噪。
- 分块（Chunking）：将长文档拆分为语义完整的片段，常用递归分块或标题保留策略，兼顾上下文连续性与检索精度。
- 索引：
 - 向量索引（ANN/向量库，如 FAISS、ScaNN、PGV）：加速相似度搜索。
 - 倒排索引（BM25 等）：支持关键词/词项匹配与布尔过滤。
- 检索策略：语义检索、关键词检索、混合检索（Hybrid）、多跳检索（Multi-hop）、查询扩展（Query Expansion）。
- 重排序（Reranking）：使用交叉编码器或专用模型对候选进行细粒度重排，提升相关性与忠实度。
- 上下文构建：在窗口限制内压缩/聚合候选（Context Compression），保留高价值证据与结构提示（如标题、摘要、表格）。
- 引用与忠实度（Faithfulness）：确保回答与注入内容一致，并以可追溯证据支撑。

设计与工程实践

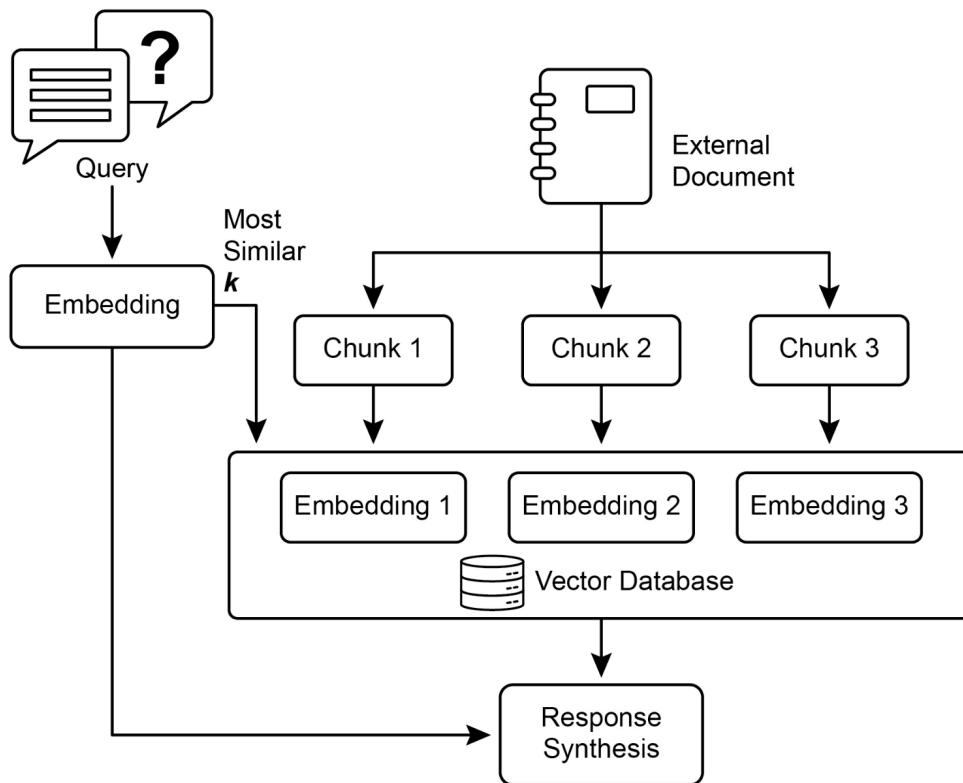
- 数据摄取 (Ingestion) 与治理：
 - 清洗/去重：移除重复、脏数据与不可解析片段（如扫描版 PDF）。
 - 元数据与权限：记录来源、更新时间、访问信息，检索时做访问控制与过滤。
 - 分块策略：根据文档结构（标题/小节/代码块）保留语义边界，尽量避免断句破坏。
- 检索管道：
 - 召回阶段：向量检索 + 关键词检索 (Hybrid)，并进行布尔/时间/类型过滤。
 - 重排阶段：交叉编码器或 LLM-Rerank，提升相关性与证据质量。
 - 压缩阶段：摘要/抽取关键句，控制上下文窗口消耗。
- 提示构建：
 - 明确回答规则：引用格式、不得臆测、无法回答时返回“未知并建议下一步”。
 - 结构化输出：表格、要点、步骤或 JSON，以便后续自动处理。
- 运行与成本：缓存（向量检索、重排结果）、并发/批处理、异步流水线与限流；对延迟敏感路径使用轻量模型重排。
- 监控与审计：
 - 指标：召回率/精确率、nDCG/MRR、事实一致性 (Faithfulness)、可引用率、用户反馈采样。
 - 日志：检索候选、重排分数、注入片段、最终回答与来源。

代理集成范式

- 指令到意图：将任务描述映射为检索意图（主题、实体、时间范围、文件类型）。
- 工具调用：通过检索工具（RAG Engine、搜索 API、企业知识库）获取候选；在状态 (state) 中记录证据。
- 记忆协作：结合工作记忆（短期上下文）与长期记忆（用户偏好、历史决策），在提示构建阶段做个性化与上下文增强。
- 决策与行动：当检索信息不足时，触发计划修订、外部工具补全或升级至人工 (HITL)。

高级变体

- Agentic RAG：在检索—生成之间加入推理层，对证据进行验证、对冲突信息做协调与综合；可开展多步查询、缺失信息补齐、结果自检与反思。
- GraphRAG：利用知识图谱显式关系进行导航与综合，适合复杂、互联的查询（实体关系、因果链、引用网络），提高深度与可解释性。
- Multi-Vector/多视角检索：为同一文档构建不同视角的嵌入（标题/摘要/实体/代码），提升检索召回与稳健性。
- 混合检索 + 重排组合：BM25 + 向量检索 + 交叉编码器重排，兼顾速度与质量；可根据场景配置不同权重。



安全与可信

- 引用与证据：在回答中附上来源（URL/文档 ID/片段编号），支持审计与复核。
- 对抗防护：提示注入（Prompt Injection）防御、越权访问控制、数据泄漏保护；对不可信来源做隔离或降权。
- 数据治理：隐私（PII）、合规（涉密、版权）、存取日志与审批流程；必要时只返回“引用摘要”，避免原文暴露。
- 置信与拒答：基于相关性/一致性阈值输出置信评分；低置信时返回“未知”并提供下一步建议（检索范围扩大、人工审核）。

评估与监控

- 信息检索指标：Precision/Recall、nDCG、MRR、Top-k 覆盖率。
- 生成一致性：FactScore、Faithfulness、Citation-Precision（引用正确率）。
- 用户体验：响应时间、上下文利用率、满意度调查与行为回传（点击、二次提问）。
- 离线基准与在线实验：构造标注集进行离线评估；通过 A/B 测试评估召回与生成质量的权衡。

应用场景

- 企业搜索与知识问答：面向员工的政策、流程、制度、技术文档检索与解答。
- 客服与支持：基于最新产品手册、FAQ 与工单知识库进行准确、可证据的答复。
- 合规与法律：提供条款引用与裁决依据，降低误判风险。
- 个性化内容：基于用户历史与偏好检索素材，进行内容生成或推荐。

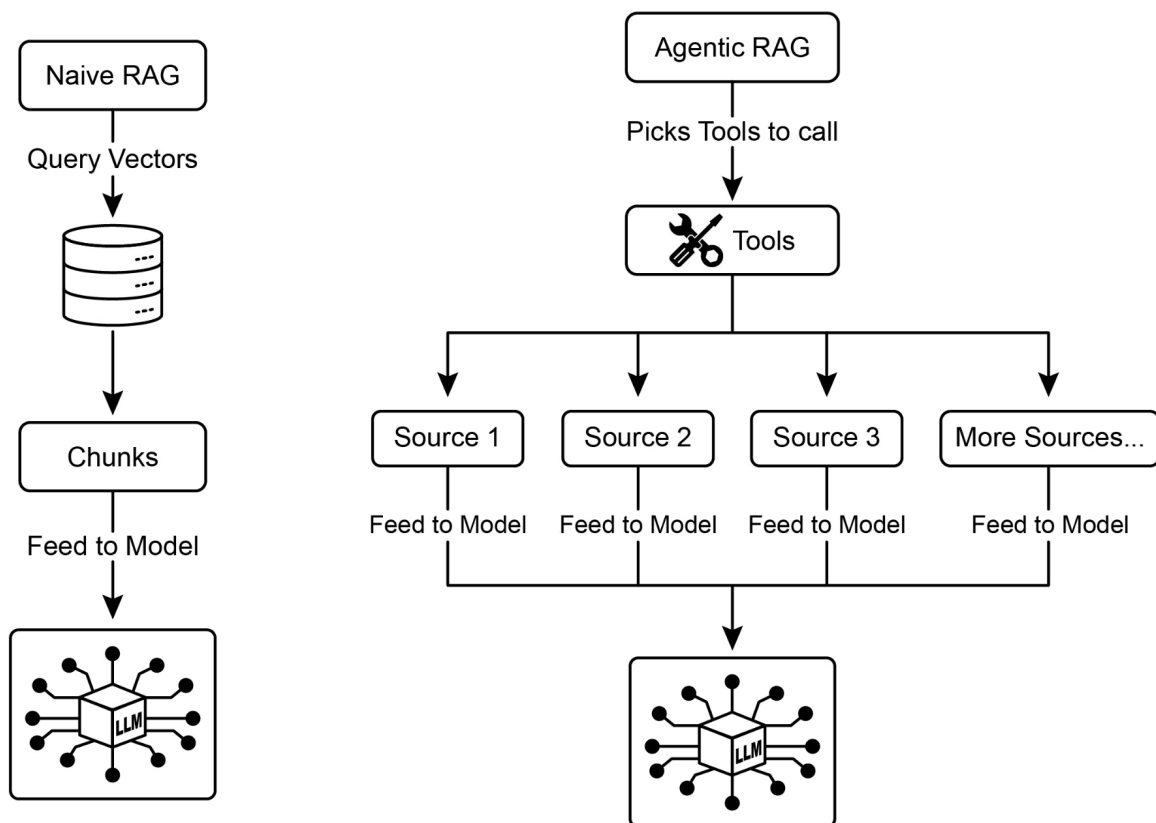
- 研发与数据分析：检索论文、报告与实验记录，支持技术决策与复现。

框架与实践示例（概念性）

- LangChain / LangGraph：
 - 构建检索器（向量 + 关键词），在图中定义“检索—重排—压缩—生成”节点。
 - 在代理流中以工具形式调用检索器，并将证据写入状态，生成阶段强制引用。
- Vertex AI RAG：
 - 管理 RAG 语料库（摄取、索引、权限）、配置检索器与生成器；支持企业级扩展与治理。
- LlamaIndex：
 - 提供灵活的索引类型与路由策略，支持多源数据融合与图结构检索。

（示例流程）

- 规范化用户问题 → 语义检索（Top-k）→ 交叉编码器重排 → 压缩关键证据 → 构造“不得臆测、必须引用”的提示 → 生成结构化回答（含来源）。



局限与权衡

- 上下文窗口限制：长文档可能超出窗口，需要摘要或分块压缩，易丢失细节。
- 片段边界问题：不恰当分块导致语义残缺，影响回答质量。
- 召回-精确权衡：过宽召回增加噪声与成本，过窄可能遗漏关键证据。
- 延迟与成本：检索与重排带来开销；可通过缓存、分层检索与异步管道优化。

结论

RAG 是让 AI 更“有知识、可验证、可用”的关键模式。通过在生成前检索并注入外部证据，LLM 能提供事实一致、可引用的回答。面向复杂任务与高可靠性需求，Agentic RAG（推理层）与 GraphRAG（知识图谱）进一步提升深度与可信度，已经在企业搜索、客服支持与个性化内容等场景产生显著价值。最终，RAG 将 LLM 从“闭卷型对话者”升级为“开卷型推理工具”。

参考文献

- Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. <https://arxiv.org/abs/2005.11401>
 - Google AI for Developers Documentation. Retrieval Augmented Generation. <https://cloud.google.com/vertex-ai/generative-ai/docs/rag-engine/rag-overview>
 - Retrieval-Augmented Generation with Graphs (GraphRAG). <https://arxiv.org/abs/2501.00309>
 - LangChain and LangGraph: Leonie Monigatti, "Retrieval-Augmented Generation (RAG): From Theory to LangChain Implementation." <https://medium.com/data-science/retrieval-augmented-generation-rag-from-theory-to-langchain-implementation-4e9bd5f6a4f2>
 - Google Cloud Vertex AI RAG Corpus. <https://cloud.google.com/vertex-ai/generative-ai/docs/rag-engine/manage-your-rag-corpus#corpus-management>
-

第 15 章：代理间通信（Agent-to-Agent, A2A）

概述

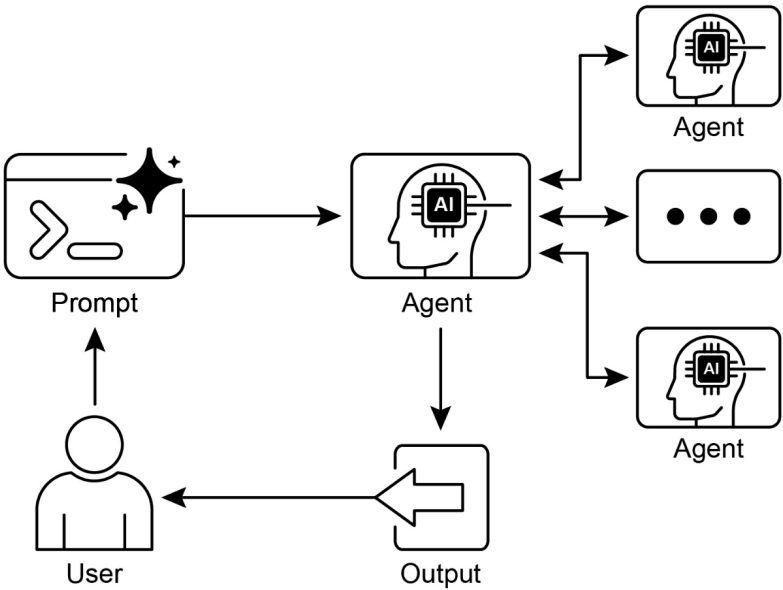
代理间通信（Agent-to-Agent, A2A）旨在为多代理系统提供标准化的高层协调与任务委派能力。相比面向工具与外部系统交互的 MCP，A2A 更关注代理之间的互操作、语义一致的消息交换与跨框架协作，从而编排复杂工作流、实现模块化与可扩展的系统构建。A2A 典型支持同步请求、异步轮询与实时流式通信，同时在安全层面内建 mTLS、显式认证与权限控制，保证跨代理通信的可靠性与合规性。

目标与价值

- 高层协调：在多代理体系中进行角色分工、任务委派与进度协同。
- 互操作性：跨框架（如 ADK、LangGraph、CrewAI 等）通信与能力复用。
- 组合式架构：以专职代理为组件，拼装复杂工作流与自治系统。
- 安全与治理：mTLS、认证、配额/速率限制、审计与可观测性。

通信模型

- 同步请求/响应：适合短事务与确定性任务。
- 异步消息/轮询：应对长任务、等待外部事件或多步协作。
- 流式交互：用于实时进度汇报、增量结果与在线协商。
- 事件驱动：基于订阅/发布（Pub/Sub）或黑板（Blackboard）进行去耦合协作。

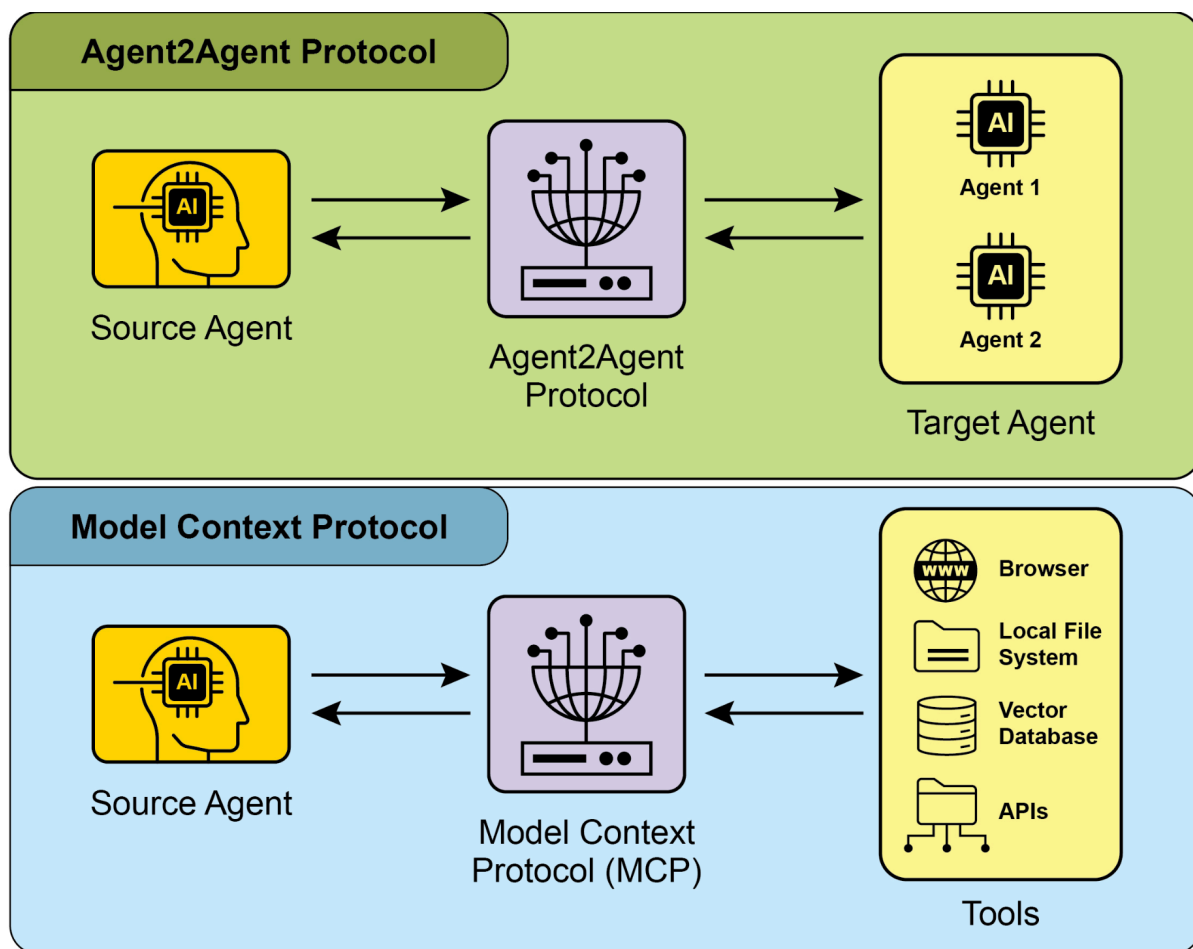


语义与协议要素

- 消息语义：任务意图 (Intent)、状态 (State)、上下文 (Context)、约束 (Constraints)。
- 模式与契约：任务委派 (Delegate)、协商 (Negotiate)、投票/共识 (Consensus)、冲突解决 (Arbitration)。
- 可序列化数据：采用 JSON/Schema 或协议缓冲区以提升跨语言/跨框架兼容性。
- 幂等与顺序性：消息去重、重试策略、顺序保证与补偿机制 (Compensation)。

设计范式与编排模式

- 监督者—工作者 (Supervisor-Worker)：单代理负责规划与分配，工作者执行并回报。
- 计划—执行—评审 (Plan-Execute-Review)：通过评审代理进行质量控制与纠错闭环。
- 合同网 (Contract Net)：基于能力与成本投标分配子任务，适合异构代理集。
- 黑板系统 (Blackboard)：共享知识面板，代理发布/消费部分结果以逐步收敛。
- 角色导向 (Role-Oriented)：定义专家、仲裁者、协调者、观测者等角色以稳定协作结构。



安全与治理

- 信任链：双向 TLS (mTLS)、令牌/密钥管理、来源验证与消息签名。
- 最小权限：细粒度授权 (Scopes)、资源/工具访问控制与隔离域。
- 合规与审计：通信日志、上下文留痕、可追溯的决策与证据链。
- 资源治理：速率限制、配额管理、背压 (Backpressure) 与熔断 (Circuit Breaker)。

与 MCP 的关系

- 互补定位：MCP 标准化代理与工具/系统之间的交互；A2A 标准化代理之间的协作与任务编排。
- 协同应用：
 - 通过 MCP 暴露工具能力；
 - 通过 A2A 将工具型代理与规划/评审型代理编排为完整流程；
 - 统一度量与治理，实现跨代理与跨工具的可观测性与安全控制。

框架实践

- Google ADK：提供 Agent 指令、状态管理与跨代理协作示例；与 A2A 协议协作实现发现与连接。
- LangGraph / CrewAI：通过图式编排与角色化协作实现多代理流程，可与 A2A 适配以跨生态协同。
- AgentDiscovery：提供代理发现与能力查询，支撑跨系统的互联与编排。

工程要点

- 语义契约：统一任务/消息 Schema，明确失败语义（可重试、需补偿、需升级）。
- 可靠交付：消息持久化队列、重试与幂等键、去重与顺序策略。
- 可观测性：度量与日志（延迟、吞吐、失败率）、分布式追踪与事件审计。
- 故障域隔离：按代理/服务划分故障域，避免级联失败；采用熔断与降级策略。

应用场景

- 采购与客服管家：规划代理协调工具型代理完成多步事务并提供进度流。
- 跨团队协同：研发/法务/合规代理协作生成材料、审阅与修订。
- 自动化运维：监控代理、修复代理与审批代理形成闭环运维流程。
- 数据工作流：检索代理、分析代理、撰写代理协作生成可引用报告。

结论

A2A 为多代理系统提供了坚实的通信与协作基石，使开发者能够构建模块化、可扩展且安全的协同式 AI 系统。与 MCP 等标准协同，A2A 进一步推动了跨生态的互操作与复用，为复杂自动化工作流与智能分布式系统铺平道路。

参考文献

- Chen, B. (2025, April 22). How to Build Your First Google A2A Project: A Step-by-Step Tutorial. Trickle.so Blog. <https://www.trickle.so/blog/how-to-build-google-a2a-project>
 - Google A2A GitHub Repository. <https://github.com/google-a2a/A2A>
 - Google Agent Development Kit (ADK). <https://google.github.io/adk-docs/>
 - Getting Started with Agent-to-Agent (A2A) Protocol. <https://codelabs.developers.google.com/intro-a2a-purchasing-concierge#0>
 - Google AgentDiscovery. <https://a2a-protocol.org/latest/>
 - Communication across frameworks (LangGraph, CrewAI, ADK). <https://www.trickle.so/blog/how-to-build-google-a2a-project>
 - Designing Collaborative Multi-Agent Systems with the A2A Protocol. <https://www.oreilly.com/radar/designing-collaborative-multi-agent-systems-with-the-a2a-protocol/>
-

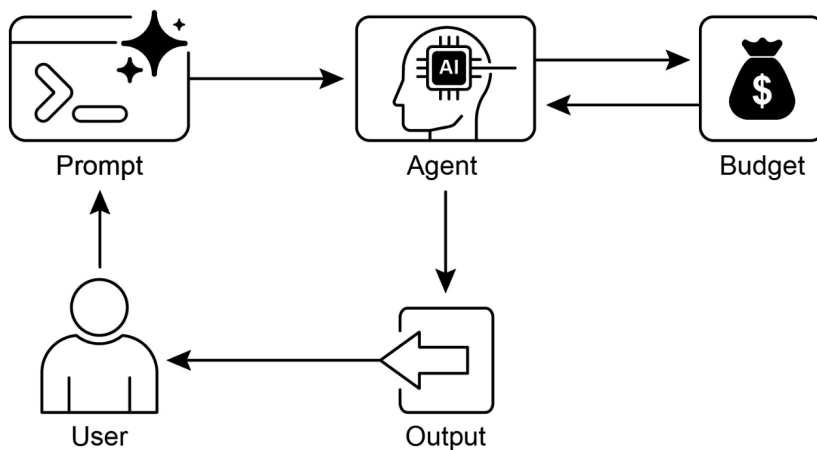
第 16 章：资源感知优化（Resource-Aware Optimization）

概述

资源感知优化旨在让代理在真实世界的约束下（算力、时间、费用、能耗、上下文窗口等）高效运作。通过动态选择模型与工具、裁剪上下文、并行/分布式感知、以及可学习的分配策略，代理在保证质量的同时降低成本与延迟，并在压力与故障下保持可退化与可恢复。

资源维度

- 计算与内存：模型推理时的 CPU/GPU/TPU 与显存占用。
- 时间与延迟：端到端响应时间、阶段性 SLA 与队列等待。
- 费用与预算：每次调用成本、月度配额、按任务计费策略。
- 上下文窗口与令牌：提示与检索片段的长度限制与压缩策略。
- 能源与部署：能耗、冷/热启动、跨区域/边缘部署的开销差异。



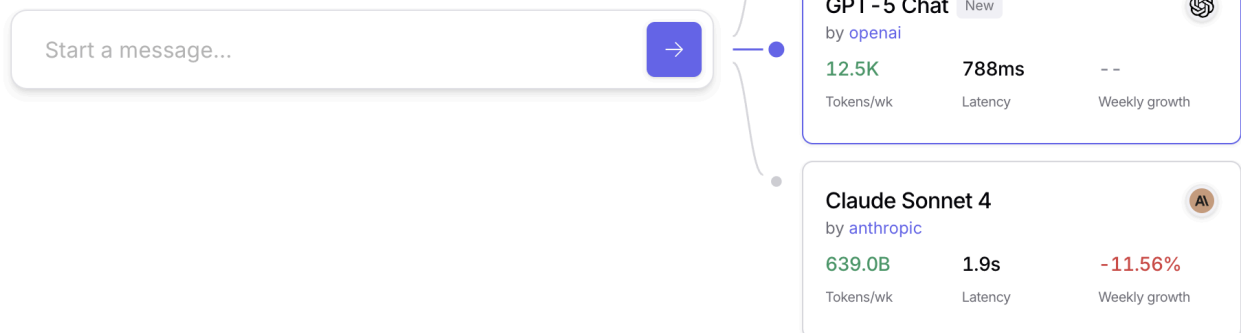
核心策略

- 动态模型切换（Routing）：
 - 简单查询用轻量模型（如 Gemini Flash），复杂任务用高阶模型（如 Gemini Pro）。
 - 基于复杂度估计、预算与质量阈值进行路由与回退。

- 自适应工具使用与选择：按任务目标与信号选择检索、计算或外部 API 工具；对冗余工具调用做去重与并发控制。
- 上下文裁剪与摘要（Contextual Pruning & Summarization）：
 - 在窗口内保留高价值证据，进行抽取式/生成式压缩与结构化提示。
 - 针对重复片段与低相关内容进行剔除与去噪。
- 预测性资源管理（Proactive Resource Prediction）：基于历史分布与工作负载预测，提前扩容或限流，避免高峰期拥塞。
- 成本敏感探索（Cost-Sensitive Exploration）：在多代理系统中控制探索开销，限制不必要的尝试与迭代。
- 能效优化与部署意识：
 - 亲近数据与算力的部署（数据局部性）；
 - 冷/热路径区分与按需扩缩容，降低空转开销。
- 并行化与分布式感知：
 - 任务分解与并行调度；避免过度并发导致外部依赖的速率限制触发。
 - 使用批处理与流式接口降低握手与往返成本。
- 可学习的资源分配策略（Learned Policies）：用强化学习/多臂老虎机（Bandits）学习模型/工具选择与预算分配策略，随场景动态适配。
- 优雅降级与回退（Graceful Degradation & Fallback）：当资源不足或失败时保留核心能力，提供“低保真但可用”的结果并给出后续建议。
- 关键任务优先级（Prioritization）：对关键路径与高价值任务赋予更高资源与调度优先级。

The Unified Interface For LLMs

Better **prices**, better **uptime**, no subscription.



架构与工程实践

- 路由器（Router）：根据复杂度、预算、延迟阈值与质量信号，选择模型与工具；支持灰度与 A/B 测试。
- 预算治理：按用户/团队/项目维度分配配额；超限时降级或排队，并记录原因与影响范围。
- 缓存与重用：
 - 语义缓存（相似查询复用）与检索重排缓存；
 - 中间结果缓存（摘要、计算结果），加速后续任务。
- 队列与调度：优先队列、速率限制、背压与动态并发；避免外部系统 429/503。
- 可观测性与反馈：
 - 指标：成本/延迟/成功率、缓存命中率、降级次数、用户满意度；
 - 反馈：Critique Agent 对路由与裁剪策略进行点评与自校正。

- 失败与补偿：对可重试与需补偿的操作做区分；记录上下文快照以便回滚。

与框架和平台

- Google ADK：通过指令与状态管理集成路由、预算与评估，支持多代理优化。
- 模型与路由平台：
 - Gemini (Flash/Pro) 按任务复杂度选择；
 - OpenRouter 等多模型路由平台，支持价格/延迟/质量维度综合选择。

评估与监控

- 效率：平均/分位响应时间、吞吐与资源使用率。
- 成本：单位任务成本、月度预算使用、降级与重试影响。
- 质量：任务完成率、事实一致性与用户满意度。
- 稳健：在资源紧张与失败场景下的可用性与恢复时间。

结论

资源感知优化是智能代理可落地的关键。通过动态模型切换、自适应工具使用、上下文裁剪与可学习分配策略，代理能够在真实约束下实现成本—性能—质量的平衡；辅以优雅降级与补偿机制，系统在压力与故障下仍能保持韧性与可持续性。

参考文献

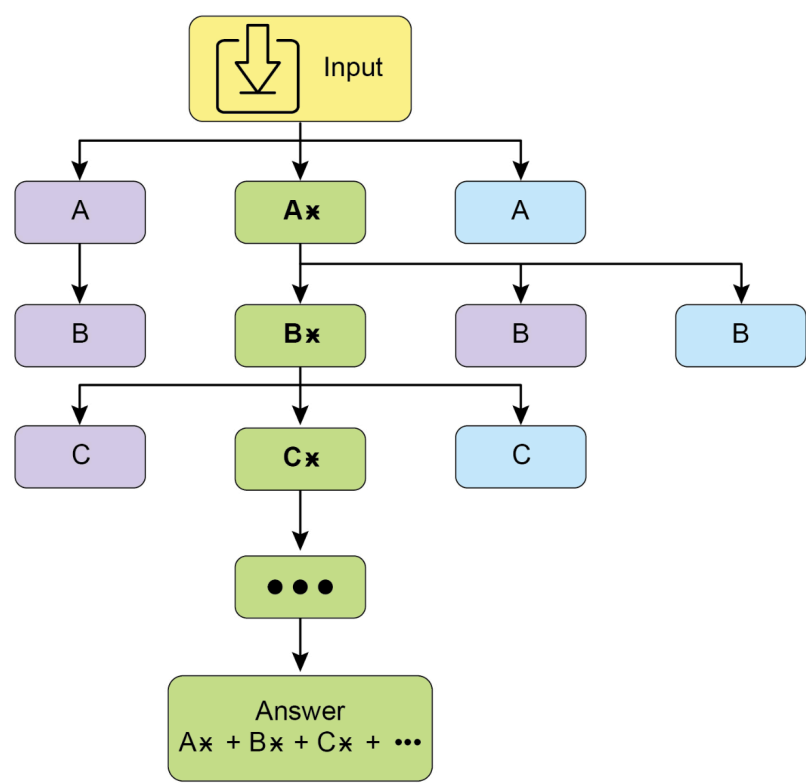
- Google Agent Development Kit (ADK). <https://google.github.io/adk-docs/>
 - Gemini Flash 2.5 & Gemini 2.5 Pro. <https://aistudio.google.com/>
 - OpenRouter. <https://openrouter.ai/docs/quickstart>
-

第 17 章：推理技术（Reasoning Techniques）

概述

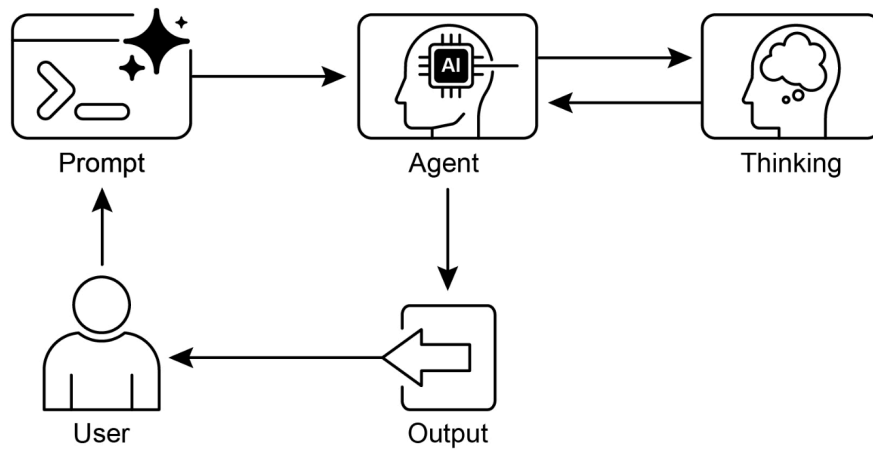
推理技术为代理提供可解释、可控与可验证的思考与行动机制。它既包含显式推理（如链式思维 CoT、树式思维 ToT、程序辅助 PAL），也涵盖“思考—行动—观察”的 ReAct 框架，从而形成真正的代理闭环：基于环境反馈动态调整策略。进一步地，推理容量可随“推理时间/计算预算”扩展（Inference Scaling Laws），而多代理协作（如 Chain of Debates, CoD）能够通过辩论与聚合达成更强的综合判断。

TOT: Tree of Thought



关键方法

- 链式思维（Chain-of-Thought, CoT）：显式展示中间步骤，提升复杂问题的解题稳定性与可解释性。
- 自一致性（Self-Consistency）：对多条思路采样并投票/聚合，降低偶然性与幻觉风险。
- 树式思维（Tree-of-Thoughts, ToT）：将问题拆解为多分支路径，通过搜索（DFS/BFS/启发式）选择最优方案。
- 程序辅助（Program-Aided Language Models, PAL）：将子问题转化为可执行程序片段，用工具/代码算子提升精度与确定性。
- ReAct（Reason + Act）：在思考与行动之间加入工具调用与观察反馈，形成“思考—行动—观察”的代理循环。
- 反思与自校正（Reflection/Critique）：引入自我评审或外部评审代理，对中间结论进行检查与修订。



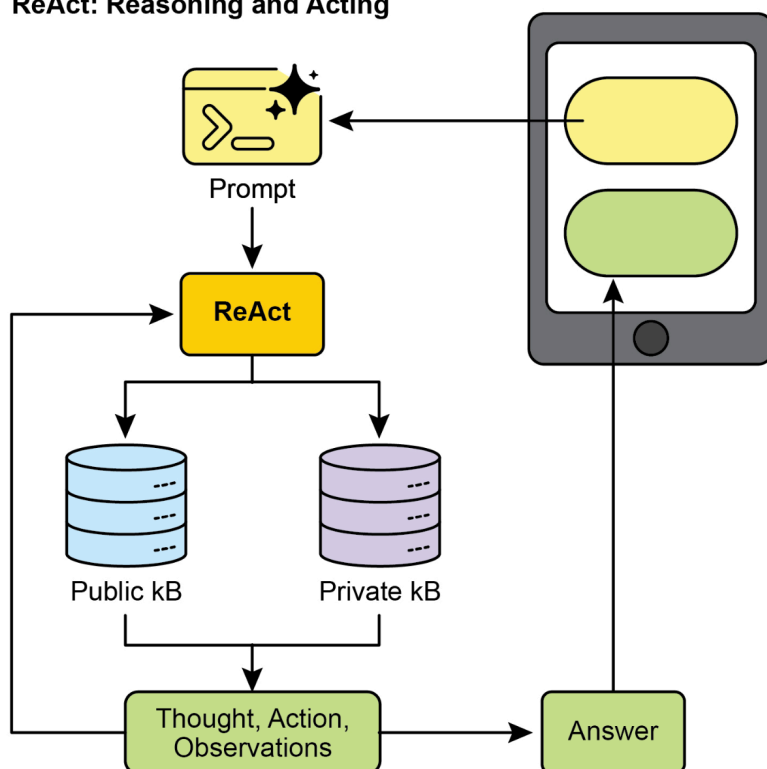
推理扩展与路由

- 推理预算 (Deliberate Time)：增加“思考步数/采样数”提升稳定性，但需控制成本与延迟。
- 动态路由：基于复杂度与质量门槛选择推理策略 (CoT/ToT)、模型与工具组合。
- 规划结合：与第 6 章“规划”衔接，先做高层计划，再以推理方法求解具体子问题。

多代理推理

- 辩论链 (Chain of Debates, CoD)：多代理各自提出方案并相互质询，最终由仲裁/聚合器输出结论。
- 角色化协作：专家代理、评审代理与执行代理分工，提升正确性与稳健性。
- 应用案例：如“Deep Research”类系统，能够自动执行多步调查与证据收集，并给出可验证结论。

ReAct: Reasoning and Acting



工程实践

- 结构化输出：以步骤/要点/表格或 JSON 形式组织推理过程，便于自动检查与复用。
- 工具结合：将计算、检索与执行作为可调工具嵌入推理流程（与第 5 章“工具使用”一致）。
- 检验与引用：与第 14 章“RAG”联动，要求关键结论附引用来源，防止幻觉。
- 采样与停止：控制温度、top-k、最大步数与中止条件；必要时加入早停与回滚。
- 评估指标：任务完成率、推理一致性（Self-Consistency 成功率）、事实忠实度（Faithfulness）。

安全与可靠性

- 幻觉防控：证据驱动、引用约束与结果自检。
- 故障处理：当工具调用失败时采用回退策略，并记录快照以便恢复（参见第 12 章）。
- 可观测性：日志记录思考链、工具调用与观察；支持审计与复盘。

结论

将“显式推理”与“可行动的工具调用”结合，代理从“会思考”迈向“能解决问题”。通过 CoT/ToT/PAL 与 ReAct 的协同，以及推理预算扩展与多代理辩论，系统能够在复杂任务中保持可靠、透明与可验证的决策过程。

参考文献

- Wei et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.
- Yao et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models.

- Gao et al. (2023). Program-Aided Language Models.
 - Yao et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models.
 - Inference Scaling Laws: An Empirical Analysis of Compute-Optimal Inference for LLM Problem-Solving (2024).
 - Multi-Agent Design: Optimizing Agents with Better Prompts and Topologies.
<https://arxiv.org/abs/2502.02533>
-

第 18 章：护栏与安全模式（Guardrails / Safety Patterns）

概述

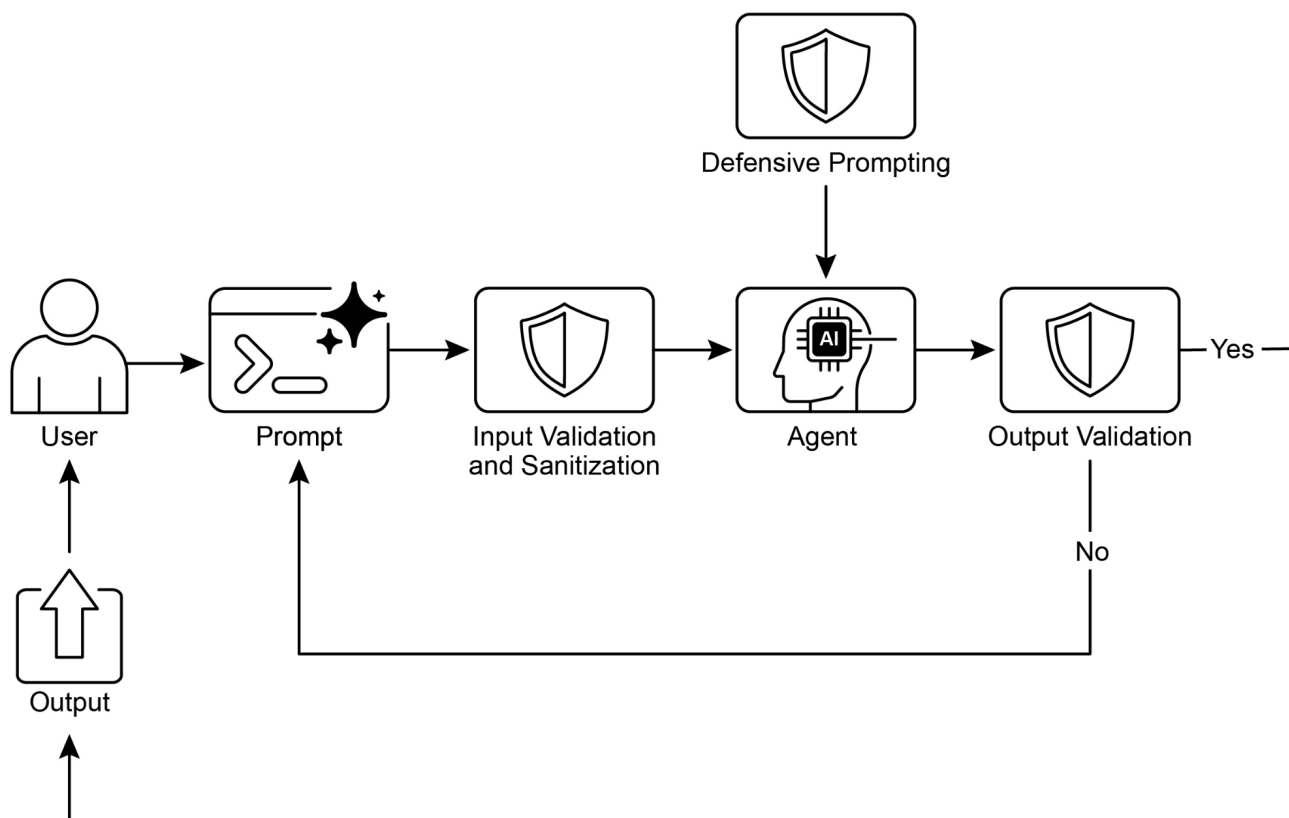
随着智能代理与 LLM 的自主性提高，如果缺乏约束与治理，其行为可能不可预测，并产生有害、偏见、违规或事实错误的输出，甚至造成现实世界风险。护栏（Guardrails）或安全模式是一组标准化的多层防御机制，旨在确保代理安全、伦理、可控并与预期目标一致。它们可在输入、推理、工具使用与输出等各阶段实施，并结合人类在环（HITL）以处理高风险决策。

风险一览（What / Why / Rule of Thumb）

- 风险（What）：
 - 生成有害/不当/偏见/不准确内容；受到越狱（Jailbreak）与对抗攻击影响；误用工具或触发未预期操作；带来法律与声誉风险。
- 目的（Why）：
 - 以分层防御确保安全、伦理与目标一致；在输入阶段阻断恶意内容，输出阶段过滤不当响应；通过行为约束与工具权限控制，引入 HITL 处理关键决策。
- 经验法则（Rule of Thumb）：
 - 只要代理输出会影响用户、系统或品牌，就需要护栏；在面对客户、生成内容、或处理敏感信息（金融、医疗、法律）时必须部署护栏。

护栏层次与技术

- 输入验证（Input Validation）：
 - 检测并拦截恶意/违规/越权内容（如提示注入、越狱），进行敏感词与策略匹配；校验结构化输入的 Schema 与范围。
- 行为约束（Behavioral Prompting）：
 - 明确禁止与允许的行为；要求引用与证据；设置拒答条件与升级机制；通过系统提示与工具规范固化政策。
- 工具使用限制（Tool Use Restrictions）：
 - 最小权限与白/黑名单；沙箱执行、超时与配额；危险操作需二次确认或人工批准；对高风险外部系统设置隔离域与审计。
- 输出过滤（Output Filtering / Moderation）：
 - 使用内容审核模型/规则过滤不当输出；实施拒绝/改写/标注策略；对引用不充分或置信度过低的结果进行降级或拒答。
- 外部监督与人类在环（External Moderation / HITL）：
 - 对高风险情境进行人工审阅与批准；引入审计链与可追溯证据。



对抗与合规

- 典型威胁：提示注入、越狱、数据泄漏、越权访问、投毒与误导。
- 防护策略：输入隔离与上下文分域；敏感数据脱敏；依据来源/权限进行检索过滤；对外部链接与代码执行进行沙箱化。
- 合规治理：遵循隐私与法律法规，记录访问与变更日志；设定保留期与删除策略。

工程实践与最佳做法

- 可观测性与审计：全面日志与度量（拦截率、拒答率、误报/漏报、升级次数）；分布式追踪关键链路与工具调用。
- 测试与评估：构建越狱/注入测试集与红队演练；离线评估 + 在线 A/B 实验；制定上线门槛与回滚标准。
- 故障容忍：与第 12 章协同，区分可重试/需补偿；在护栏触发时保证系统“受控失败”。
- 状态管理：在状态中记录决策与理由，支持复盘与问责。

与其它模式的协同

- 与 RAG（第 14 章）：要求引用与证据，提升忠实度并可审计。
- 与人类在环（第 13 章）：高风险环节升级人工，提供审阅界面与审批流。
- 与规划与工具使用（第 5、6 章）：在计划/工具层施加权限控制与安全策略。

结论

有效的护栏是负责任 AI 的核心承诺。通过分层防御、持续监控与迭代优化，开发者能够构建稳健、可信且高效的代理系统。将护栏视为复杂软件工程的一部分（容错、状态、测试），能确保系统长期可用并符合法规与伦理要求。

参考文献

- Google AI Safety Principles. <https://ai.google/principles/>
 - OpenAI API Moderation Guide. <https://platform.openai.com/docs/guides/moderation>
 - Prompt injection. https://en.wikipedia.org/wiki/Prompt_injection
-

第 19 章：评估与监控（Evaluation and Monitoring）

概述

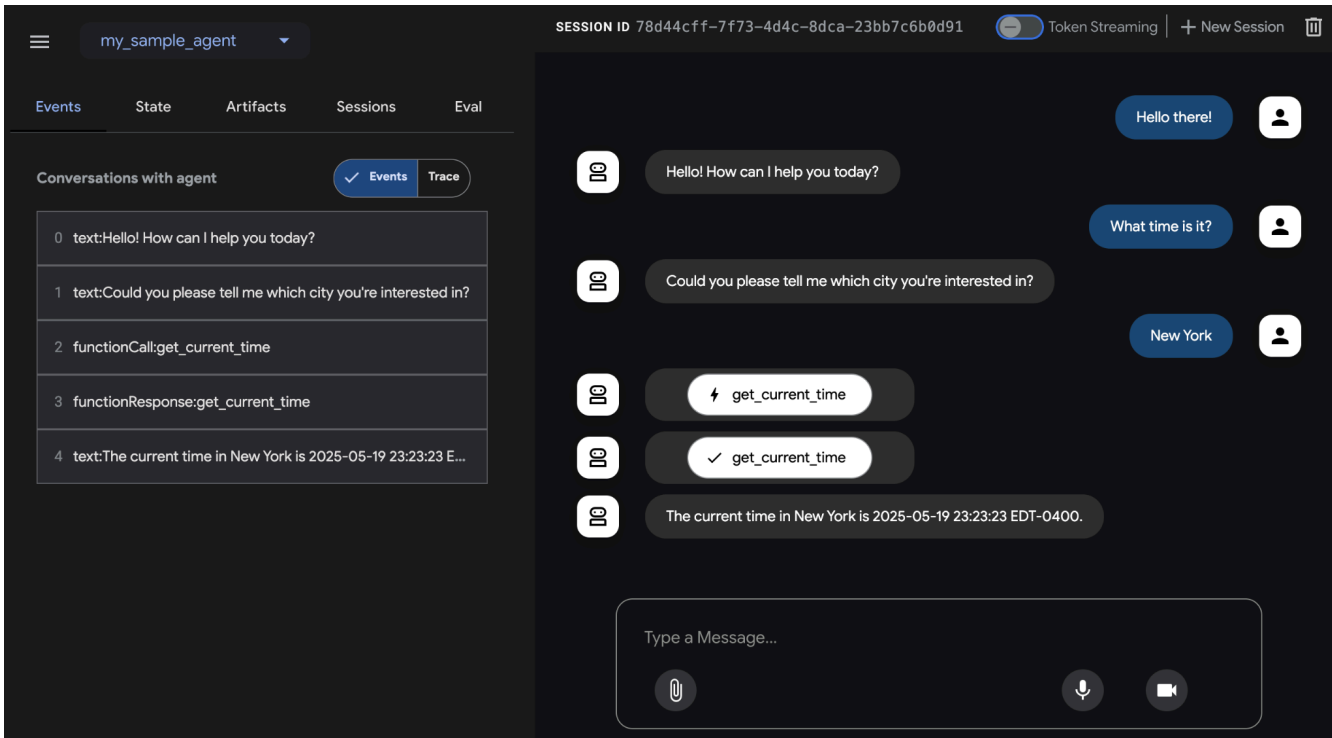
- 面向智能体（Agent）的评估不仅是一次性的准确率测试，更是围绕真实环境持续、全栈的质量保障体系。
- 评估需覆盖“过程与结果”：既衡量任务达成、成本与效率，也分析代理在执行过程中的决策质量与轨迹（Trajectory）。
- 在细腻维度（如“有用性”“礼貌性”“安全性”“一致性”）上，LLM-as-a-Judge 等方法正逐步成为必要补充；同时应结合框架化测试（如 Google ADK）进行单元与集成验证。

评估目标与维度

- 任务成功率：目标是否达到、输出是否可用、是否满足约束与验收标准。
- 准确性与忠实度：事实正确、引用可靠、推理链一致；避免幻觉与不当推断。
- 效率与成本：延迟、吞吐、推理步数、上下文长度、模型与工具调用成本。
- 稳健性与韧性：异常场景处理能力、策略切换、降级方案与可恢复性。
- 用户体验与可解释性：输出清晰、交互友好、可审计的思考过程与依据。

轨迹级评估（Trajectory Evaluation）

- 关注“决策过程”的质量：每一步的规划、工具选择、证据收集与反思是否合理。
- 将执行轨迹结构化记录：状态、动作、观察、理由（Thought）、自检（Self-Check）。
- 以“过程可见性”提升可解释性：便于复盘、定位问题根因与持续优化策略。



指标与度量（Metrics）

- 功能性：task_success_rate、spec_compliance、acceptance_pass_rate。
- 质量性：factual_accuracy、faithfulness、helpfulness、toxicity、bias。

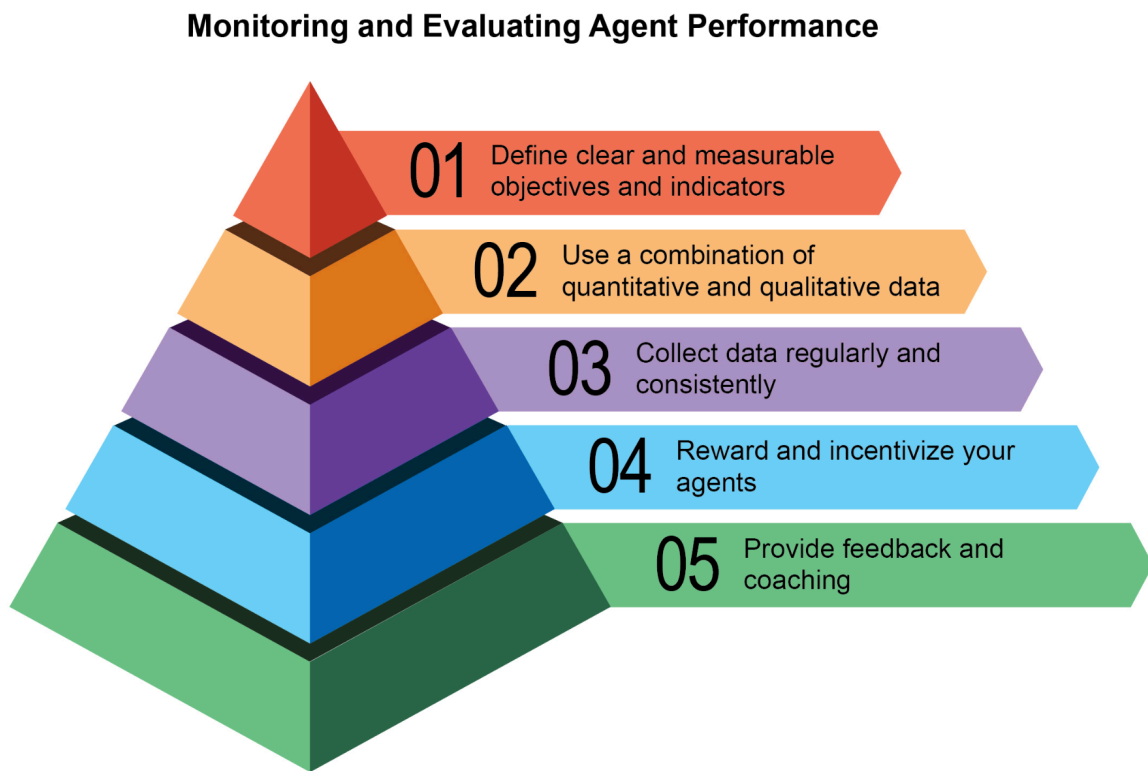
- 效率性: latency_pXX、throughput、tokens_in/out、tool_calls、cost_per_task。
- 过程性: steps_count、reruns、rollback_rate、self_check_rate。
- 可靠性: error_rate、retry_rate、degrade_invocations、SL0_breaches。

离线评估与测试

- 基准任务集: 针对场景构建高质量、可复现的标注集与验收规则。
- 单元测试与集成测试: 对工具适配器、计划模块、记忆/RAG 检索等进行可控验证。
- 回放与对比: 将历史轨迹重放, 比较不同模型/提示/策略的差异与改进幅度。
- 自一致性 (Self-Consistency): 多样化采样与投票, 抑制偶然错误、提高稳定性。

在线评估与监控 (Observability)

- 端到端监控: 采集日志、指标与事件, 贯穿“入口→推理→工具→输出”。
- A/B 与灰度: 在真实用户流量中观测效果差异; 控制风险与迭代速度。
- 采样与质检: 定期抽样由 LLM-as-a-Judge 或人类评审进行质量巡检与标注回流。
- 告警与 SLO: 明确延迟、错误率、质量阈值与错误预算, 驱动运营治理。



LLM-as-a-Judge 与人类评审

- LLM 作为评审者: 基于明确评分标准与参考答案, 对输出进行多维打分与解释。
- 人类在环 (HITL): 在关键任务或高风险输出上进行复核与仲裁, 提升信任度。
- 评审一致性: 通过校准题、双盲评审与共识机制, 降低评分主观性与漂移。

多代理系统评估

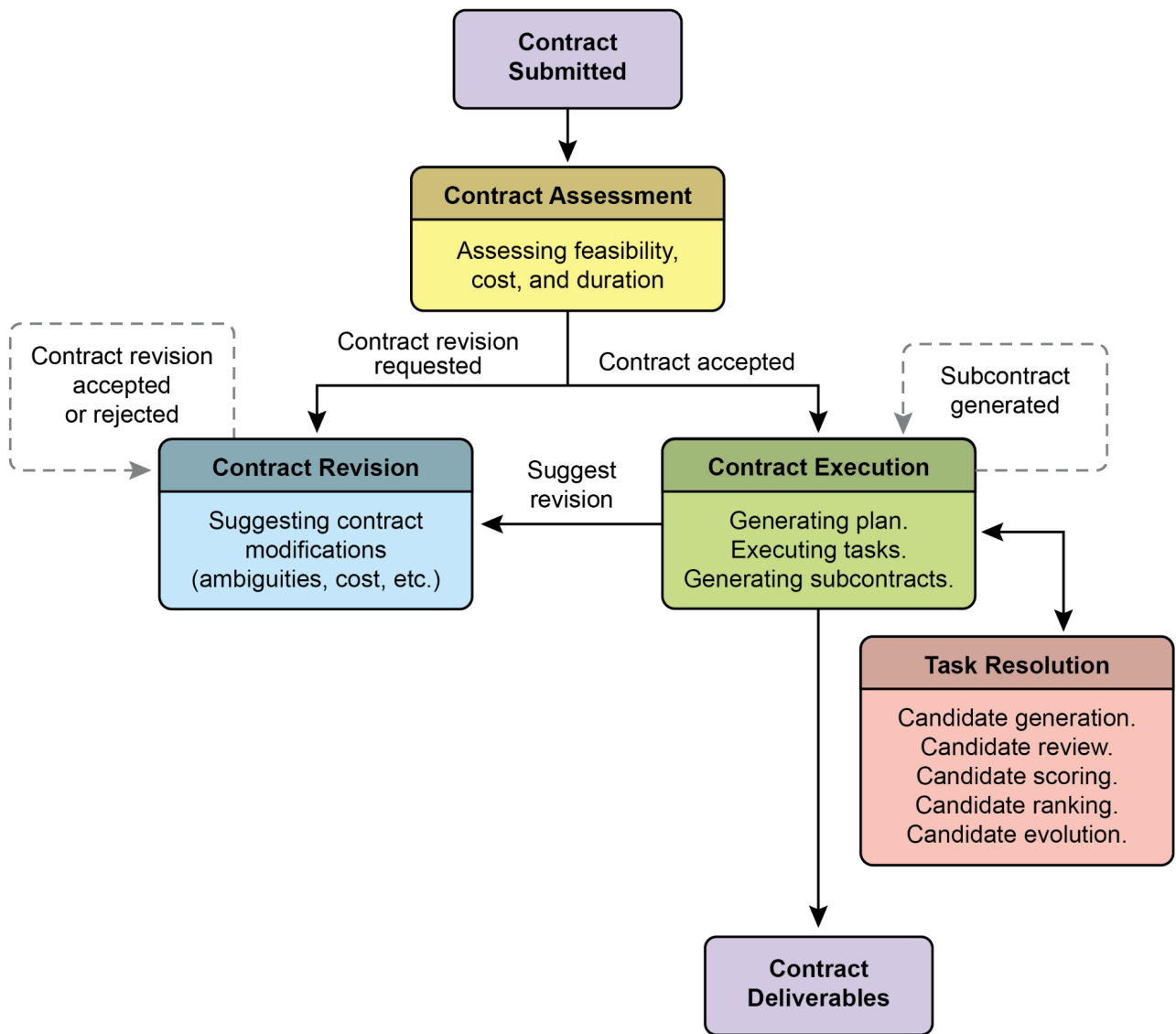
- 协同成功率：衡量角色分工、协议遵循（如 A2A/MCP）、消息质量与对齐程度。
- 交互效率：通信轮数、上下文复用率、信息冗余与冲突解决效率。
- 团队鲁棒性：单点故障影响、协作失败恢复、领导/投票/辩论机制有效性。

从“提示驱动代理”到“承包商代理”（Contractor Agents）

- 形式化合同：以明确、可验证的条款绑定代理的职责、交付物与质量标准。
- 任务分解与谈判：允许代理在合同框架下进行协商、分工与里程碑承诺。
- 自我验证与对账：代理对中间结果进行自检、对账与合规性验证，增强可靠性。
- 面向高风险场景：将不可预测的“工具”转化为“可追责的系统”。

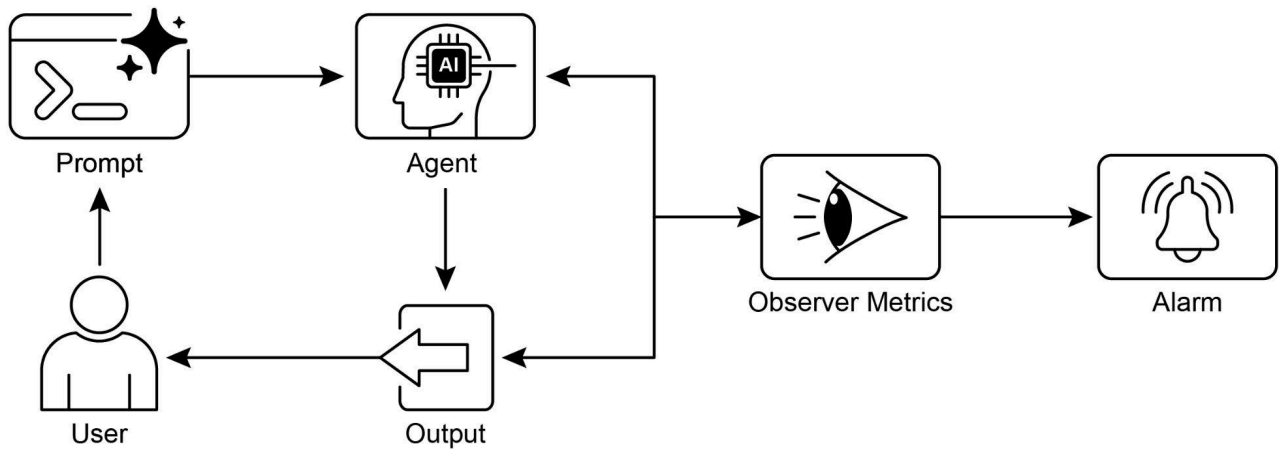
工程与落地实践（以 ADK 为例）

- ADK Evaluate：为评估设计用例、打分器（Judge）、数据集与报告汇总。
- ADK Web：提供交互式面板与实验管理，支持单元/集成/端到端测试。
- 评估管线：离线批处理与在线流式结合，自动化生成质量趋势与回归告警。
- 与 RAG/规划/工具集成：在链路级监控与模块级测试之间建立闭环。



监控与质量保障（SLO / Error Budget）

- 定义目标：如 95% 请求延迟 < Xs、任务成功率 > Y%、事实错误率 < Z%。
- 误差预算：允许在迭代中可控“冒险”，为创新与上线节奏提供缓冲。
- 根因分析：面向轨迹与调用链的可观测性，定位质量与稳定性问题。
- 质量运营：与产品/风控/合规协作，建立持续质量改进机制。



结论

- 智能体评估应当走向“持续、全栈、以过程为中心”的质量保障。
- 结合 LLM-as-a-Judge、ADK 等工具与 HITL 机制，既要提升自动化评估能力，也要在关键环节保持人类监督。
- 多代理系统中的协作评估与“承包商代理”范式，为在高风险领域落地提供了明确的工程路径与信任基础。

参考文献

- ADK Web: <https://github.com/google/adk-web>
 - ADK Evaluate: <https://google.github.io/adk-docs/evaluate/>
 - Survey on Evaluation of LLM-based Agents: <https://arxiv.org/abs/2503.16416>
 - Agent-as-a-Judge: Evaluate Agents with Agents: <https://arxiv.org/abs/2410.10934>
 - Agent Companion (gulli 等): <https://www.kaggle.com/whitepaper-agent-companion>
-

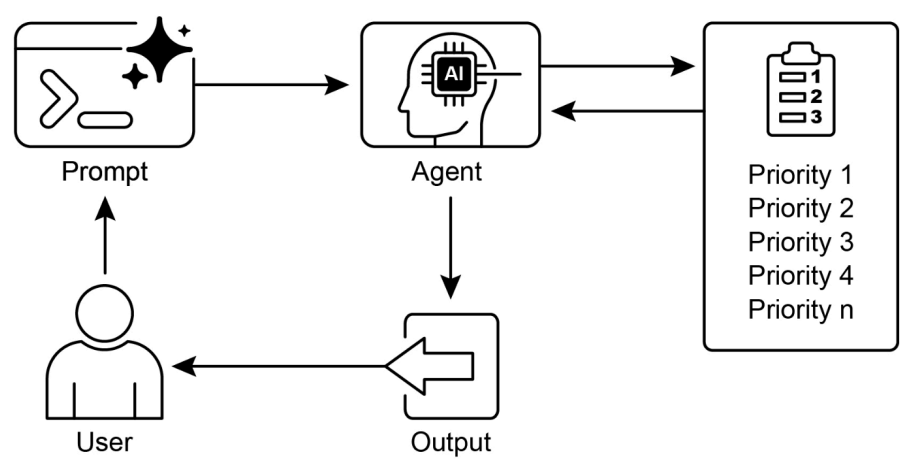
第 20 章：优先级（Prioritization）

概述

- 优先级是智能体在多目标、有限资源与动态环境中进行理性取舍的核心能力。
- 通过对“紧急度、重要性、影响、依赖、风险、截止期”等维度综合评估，代理能够自主管理工作流并持续重排任务顺序。
- 该模式将简单“执行脚本”提升为“主动的战略决策者”，显著增强效率与运行稳健性。

优先级维度

- 紧急度（Urgency）：与时间相关的迫切程度、SLA/SLO 约束与告警信号。
- 重要性（Importance）：业务目标对齐、关键路径位置与失败代价。
- 影响力（Impact）：对用户体验、收入、合规与安全的正负效应。
- 依赖关系（Dependencies）：前置条件、资源占用、外部系统可用性。
- 风险（Risk）：错误概率、滥用可能、审计难度与回滚成本。
- 截止期（Deadline）：任务到期时间与违约后果（合同/承诺场景）。
- 价值（Value）：单位成本带来的收益、学习价值与探索潜力。



评分与排序方法

- 规则/打分：为每个维度赋权，计算综合分并据此排序。
- 学习型排序：结合历史效果与标签，训练排序模型提升泛化与稳健性。

- Bandit/Thompson Sampling：在探索与利用之间动态权衡，提高总体收益。
- 成本敏感度：考虑模型/工具调用成本与队列等待时间，优化全链路效用。

调度与执行策略

- 优先队列（Priority Queue）：按分数或级别出队；支持分层队列（高/中/低）。
- 最短处理时间优先（SPT）：在相近价值时优先处理短任务以提升吞吐。
- 最早截止期优先（EDF）：面向到期风险的实时系统与合同场景。
- 批处理与并发：面向相似任务进行批量处理，降低模型/工具调用成本。
- 预占/抢占（Preemption）：高优任务可打断低优任务；需配合状态保存与回滚。
- 延迟队列/重试：对临时失败或资源受限任务进行延迟与指数退避。

动态重排（Dynamic Re-prioritization）

- 触发条件：新事件、指标变化、资源可用性与外部风险信号。
- 持续评估：周期性刷新评分，避免队列“饥饿”与“尾部拥堵”。
- 自主策略：代理可基于目标与上下文调整执行顺序并解释决策理由。

代理自主管理示例（行为特征）

- 解析模糊请求：从不明确描述中抽取目标、约束与优先维度。
- 自动选用工具：根据任务需求调用最合适的模型/工具链（含降级与回退）。
- 合理编排步骤：把复杂目标拆解为可执行序列，并在过程中自检与修正。

与其他模式的协同

- 资源感知优化：与第 16 章策略结合，兼顾成本、延迟与质量三角形。
- Guardrails/Safety：高风险任务提升优先级的同时加强审计与人类在环（HITL）。
- 规划与工具使用：结合 ReAct/ToT 等，确保优先级决策落地为高质量行动。

工程落地实践

- 数据结构：优先队列、分层队列、延迟队列与状态持久化。
- 策略配置：通过策略表/规则引擎或学习模型控制打分与阈值。
- 观测指标：出队等待时间、抢占次数、队列积压、SLO 违约与任务成功率。
- 回放与评估：对历史队列进行模拟重排，验证算法在不同流量/资源下的性能。

风险与安全考量

- 公平性：避免某类请求长期被低优先级压制导致饥饿。
- 合规与伦理：在涉及安全/合规的任务上设硬性阈值与强制审查链。
- 滥用防范：防止通过伪造紧急度或高影响声称绕过风控与配额。

结论

- 优先级是构建“会思考、会取舍、能自我管理”的智能体关键能力。
- 通过动态重排、策略化调度与清晰的决策解释，代理可在复杂环境中可靠运行。

- 掌握优先级模式是迈向稳健、可运营的 Agent 系统的重要基础。

参考文献

- Examining the Security of Artificial Intelligence in Project Management:
<https://www.irejournals.com/paper-details/1706160>
 - AI-Driven Decision Support Systems in Agile Software Project Management:
<https://www.mdpi.com/2079-8954/13/3/208>
-

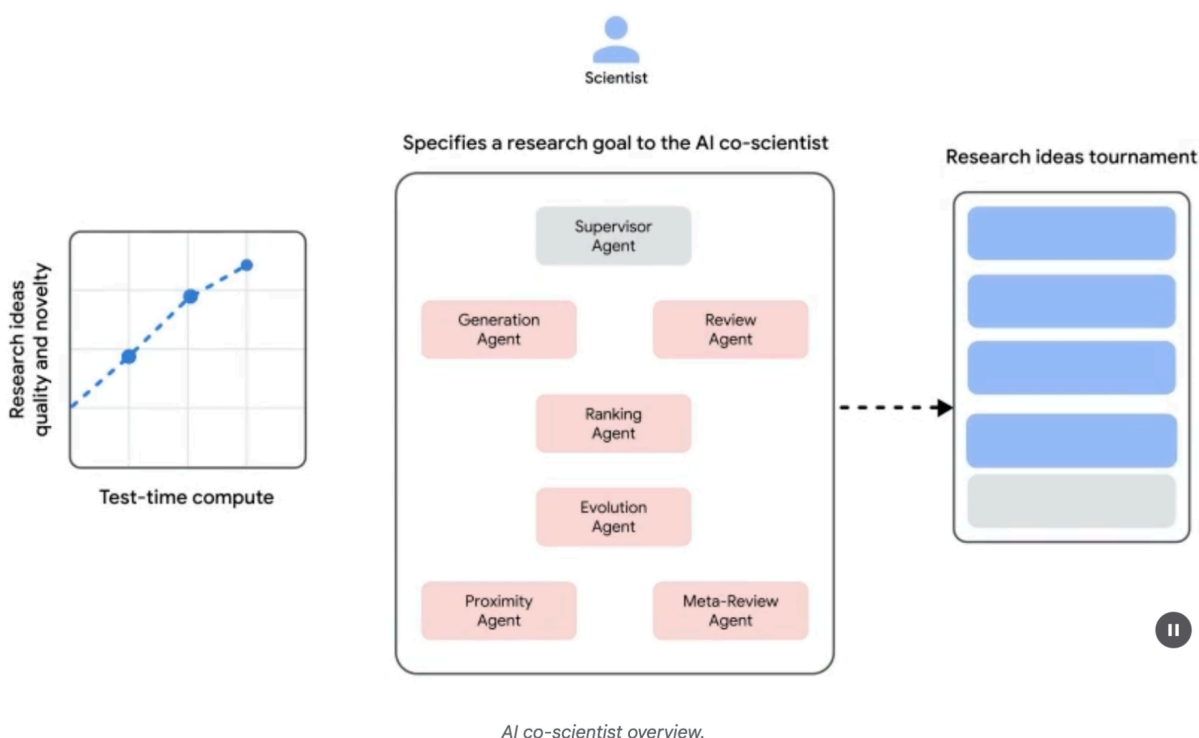
第 21 章：探索与发现（Exploration and Discovery）

概述

- 探索模式让智能体在不确定环境中主动寻求新信息、提出假设、设计实验并自我迭代。
- 核心在于平衡“探索-利用（Exploration-Exploitation）”，既要持续试探新路径，也要利用已知最优策略以获得稳定收益。
- 高度代理化系统可在较小的人类干预下追求长期、开放式目标，将 AI 从“工具”提升为“主动的知识伙伴”。

探索-利用权衡（E/E）

- 问题本质：在未知状态下如何分配资源到“尝试新事物”与“巩固已知最优”。
- 常见策略： ϵ -greedy、UCB、Thompson Sampling、上下文多臂老虎机（Contextual Bandits）。
- 动态平衡：随时间调整探索率；在环境变化或质量下降时提升探索强度。



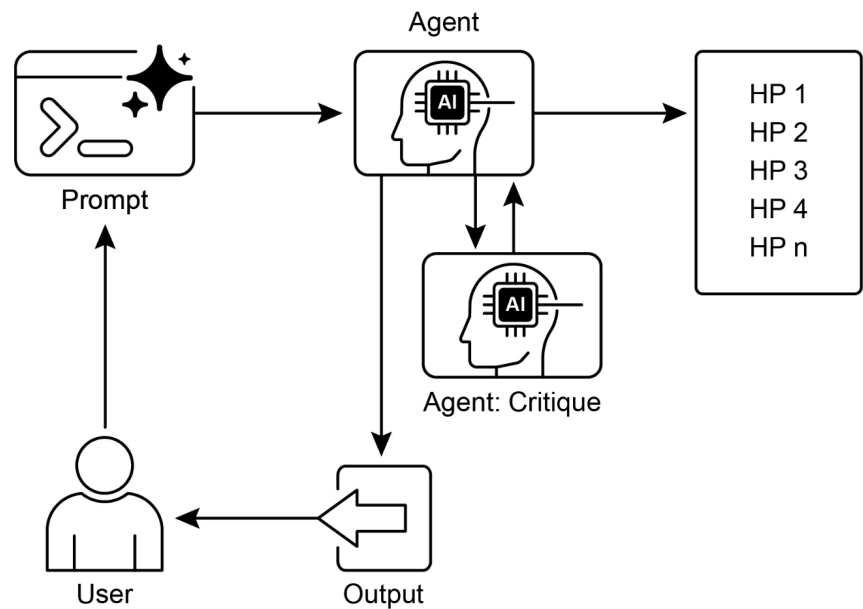
好奇心与新颖度（Curiosity & Novelty）

- 内在动机：以新颖度、预测误差或不确定性作为奖励，驱动代理主动搜寻信息缺口。
- 新颖检验：避免“伪新颖”（重复或噪声），通过去重、语义相似度与证据质量控制。
- 知识增量：将新发现结构化入库（向量索引/知识图谱），支持后续检索与推理。

方法与管线

- 文献/数据检索：系统性地探索外部资源，聚合多源证据（RAG/多搜索引擎）。
- 假设生成与批判：通过多代理辩论/评审对候选假设进行筛选与修正。

- 实验设计与执行：规划验证步骤、采集数据与进行统计分析（含仿真与实测）。
- 解释与报告：自动生成可读报告与可视化，附上溯源与不确定性说明。



多代理协同（示例角色）

- 假设生成者 (Hypothesizer)：提出可检验的研究假设与变体。
- 批评者 (Critic)：审视逻辑、识别偏差与风险、给出改进建议。
- 规划者 (Planner)：编排实验、分配资源、设定优先级与截止期。
- 执行者 (Executor)：具体实施、调用工具、记录轨迹与中间结果。
- 证据管理员 (Curator)：管理数据、引用与证据质量，维护可审计性。

开放式探索与层级结构

- Co-Scientist：多代理自主生成、辩论与演化科学假设，加速科学发现。
- Agent Laboratory：构建类似人类研究团队的层级结构，自主管理探索生命周期。
- 目标：追求长期、开放式计划 (Open-endedness)，在较少监督下推进创新。

安全与伦理

- 研究规范：遵守数据与实验合规要求；明确敏感领域的审批与限制。
- 偏见控制：在假设与证据筛选阶段引入多样性与一致性审查。
- 人类在环 (HITL)：关键结论与高风险输出由人类复核与仲裁。

工程实践

- 任务编排：以工作流/管线驱动探索环节，支持回放与断点续跑。
- 缓存与去重：避免重复检索与实验，提升资源利用率。
- 溯源与可审计：记录来源、方法与参数，支持结果复现与合规审核。
- 轨迹与版本：跟踪假设演化、实验迭代与模型/工具版本。

评估与监控

- 探索效率：单位时间/成本的有效发现数量与质量。
- 发现验证率：候选假设通过验证的比例与后续影响力。
- 质量与可靠性：证据充分性、统计显著性、结论可复现性。
- 成本-收益：在资源约束下的整体产出与迭代速度。

结论

- 探索与发现模式为构建“真正的代理化 AI”提供蓝图，把计算系统转变为独立、逐目标的知识伙伴。
- 结合多代理协同、内在动机与稳健的工程管线，可在安全与伦理约束下显著提升人类认知与创新速度。

参考文献

- Exploration–Exploitation Dilemma: https://en.wikipedia.org/wiki/Exploration%E2%80%93exploitation_dilemma
 - Google Co-Scientist: <https://research.google/blog/accelerating-scientific-breakthroughs-with-an-ai-co-scientist/>
 - Agent Laboratory (Using LLM Agents as Research Assistants) : <https://github.com/SamuelSchmidgall/AgentLaboratory>
 - AgentRxiv: Towards Collaborative Autonomous Research: <https://agentrxiv.github.io/>
-

附录 A：高级提示工程（Advanced Prompting Techniques）

概述

- 高级提示工程是将通用语言模型升级为“可感知、能行动、可治理”的智能体的关键技能。
- 全谱提示涵盖“思考（推理）—感知（检索/上下文工程）—行动（工具/函数调用）—治理（安全/评估/约束）”。
- 掌握这些技术可让模型从纯文本生成器跃升为具备自主性与环境感知的复杂代理。

推理型提示

- Chain-of-Thought (CoT)：显式思路展开，提升复杂推理的可解释性与稳定性。
- Self-Consistency：多样化采样与投票，减少偶然错误，增强答案一致性。
- Tree-of-Thoughts (ToT)：以树结构探索多条解题路径，支持回溯与分支评估。
- Step-back Prompting：抽象视角重述问题，再回到具体求解，缓解局部陷阱。

行动型提示与工具集成

- ReAct：推理（Reasoning）与行动（Acting）交替进行，边思考边调用工具。
- 函数/工具调用（Function Calling）：用结构化参数驱动外部 API/工具，确保可审计与可复现。
- 计划与执行：将复杂任务拆解为步骤，结合优先级与资源约束进行调度与回退。

感知与上下文工程（Context Engineering）

- RAG（检索增强生成）：从外部知识库检索最新、相关的信息，降低幻觉并提升事实性。
- 索引与分块：向量嵌入、分块策略、重排与上下文压缩，平衡覆盖与成本。
- 语境对齐：在提示中注入角色、目标、约束与输入模式，增强任务契合度。

提示模板化与参数化

- 模板抽象：将提示拆解为“角色/目标/输入/约束/格式/评估标准”等模块化片段。
- 变量注入：使用占位符参数化业务内容；支持配置化管理与 A/B 实验。
- DSPy 等程式化提示：通过声明式/程序化管线代替手工调提示，提升可维护性与复现性。

质量与安全治理

- 质量门禁：定义验收标准与拒绝条件，设定输出格式与合规约束。
- 审查与过滤：在输出层进行安全过滤（有害/偏见/合规），必要时启用 HITL 复核。
- 评估与监控：结合 LLM-as-a-Judge、轨迹记录与指标监控，实现持续质量改进。

工程实践要点

- 数据与提示版本化：记录提示、参数与索引版本，支持回滚与复现。
- 实验管线：批量评估不同提示与检索策略，生成报告与可操作洞见。

- 生态集成：与工具/函数调用、RAG、规划/优先级、Guardrails 等模式协同。

结论

- 高级提示工程贯穿推理、检索、行动与治理四个维度，是构建成熟 Agent 的基础。
- 通过模板化、程序化与评估监控的工程化实践，可持续提升系统的稳定性与可控性。

参考文献

- Prompt Engineering: <https://www.kaggle.com/whitepaper-prompt-engineering>
 - Chain-of-Thought Prompting: <https://arxiv.org/abs/2201.11903>
 - Self-Consistency for CoT: <https://arxiv.org/pdf/2203.11171>
 - ReAct: <https://arxiv.org/abs/2210.03629>
 - Tree of Thoughts: <https://arxiv.org/pdf/2305.10601>
 - Take a Step Back: <https://arxiv.org/abs/2310.06117>
 - DSPy: <https://github.com/stanfordnlp/dspy>
-

附录 B：AI 代理式交互——从 GUI 到现实世界（AI Agentic Interactions: From GUI to Real world environment）

概述

- 数字空间与物理环境正在收敛：桌面/浏览器的复杂工作流自动化与现实世界的多模态理解（摄像头、麦克风）相互融合。
- 目标是构建跨环境、上下文感知、实时可操作的通用 AI 助手，为日常任务提供主动协助。
- “Vibe Coding”等新范式推动人机协作更直观与目标导向，让开发者关注意图与产出而非实现细节。

数字空间交互（Desktop/Browser）

- 桌面自动化：跨应用工作流编排、窗口识别、输入法/剪贴板/文件系统操作。
- 浏览器自动化：导航与表单填充、抓取与解析、会话态维持与登录管理。
- 工具栈：Selenium/Playwright、系统 API/快捷键、Browser-use 等高层框架。

现实世界交互（Sensors/Actuators）

- 多模态输入：摄像头、麦克风、环境传感器（位置/温湿度/运动）。
- 实时理解：语音对话、场景识别与对象跟踪、事件流处理与响应。
- 执行器：语音播报、设备控制、机器人动作；面向安全的权限与速率限制。

代表性项目与能力

- Operator/ChatGPT Agent：更自然的人机协作与自主任务执行。
- Anthropic Computer Use：以 Claude 驱动的计算机操作能力。
- DeepMind Mariner、Project Astra：多模态理解与环境感知。
- Gemini Live：实时语音/视频交互，贴近人类沟通方式。

设计要点

- 会话与上下文：维护用户意图、历史操作与环境状态，避免“盲目行动”。
- 能力抽象与治理：将低级操作封装为安全的高层指令，具备审计与回滚机制。
- 模态融合：文本、语音、视觉、结构化数据的协同，提升鲁棒性与体验。
- 人类在环（HITL）：高风险场景引入审批与确认，降低误操作概率。

安全与合规

- 权限控制：最小权限、显式许可与范围限定；敏感操作（资金/隐私）需强校验。
- 速率限制与防滥用：限制自动化频率与并发，防止系统性骚扰或资源耗尽。
- 记录与透明：完整轨迹日志与可溯源性，便于审计与事故复盘。

工程实践

- workflow编排与恢复：支持断点续跑、失败补偿与状态持久化。
- 观察与度量：实时指标（延迟/成功率/资源消耗），与用户反馈闭环。
- 可测试性：模拟环境与沙箱，隔离风险并提升验证效率。

结论

- 数字与物理能力的融合将催生通用、主动、上下文感知的 AI 伙伴。
- 在目标导向的人机协作与严格的安全/合规框架下，代理式交互可显著提升生产力与创新速度。

参考链接

- OpenAI Operator: <https://openai.com/index/introducing-operator/>
 - ChatGPT Agent: <https://openai.com/index/introducing-chatgpt-agent/>
 - Browser Use: <https://docs.browser-use.com/introduction>
 - Project Mariner: <https://deepmind.google/models/project-mariner/>
 - Anthropic Computer Use: <https://docs.anthropic.com/en/docs/build-with-claude/computer-use>
 - Project Astra: <https://deepmind.google/models/project-astra/>
 - Gemini Live: <https://gemini.google/overview/gemini-live/?hl=en>
 - GPT-4 Research: <https://openai.com/index/gpt-4-research/>
 - Claude 4: <https://www.anthropic.com/news/claude-4>
-

附录 C：Agent 框架快速概览（Quick overview of Agentic Frameworks）

概述

- 生态提供从低层库到高层平台的多样选择，覆盖线性流程、图式推理到多代理编排。
- 选型关键在于抽象层级与运营能力：是需要细粒度的图式控制，还是要开箱的团队编排与监控。

代表框架与特性

- LangChain：易上手的线性工作流与工具/模型集成，适合原型与简单链路。
- LangGraph：具备状态与循环的图式推理，支持复杂决策与动态路径。
- LlamaIndex：面向数据密集型应用的检索/索引/RAG 能力，适合企业知识场景。
- CrewAI：多代理团队编排，预设角色/流程，强调协作与任务分工。
- Google ADK：以评估/测试与工程落地为中心，提供单元/集成/端到端质量工具与管理。
- MCP 集成：模型上下文协议为外部工具接入提供统一接口，便于跨框架复用能力。

取舍与适用性

- 线性序列 vs 图式循环：前者实现简单、后者可表达复杂状态机与回溯逻辑。
- 轻量 vs 重量：轻量框架灵活但需自行搭建监控/生命周期管理；重量平台开箱即用但更“有主见”。
- 数据中心 vs 协作中心：数据检索/索引优先或多角色编排优先，依应用侧重点选择。

选型建议

- 简单顺序任务：LangChain + 工具调用，快速落地。
- 复杂推理与循环：LangGraph 或具状态的自定义图式引擎。
- 知识与数据密集：LlamaIndex（与向量/图谱/重排搭配）。
- 多代理协作：CrewAI/ADK，关注角色、流程与评估监控闭环。

结论

- 框架选择取决于应用是否需要简单序列、动态推理环、或管理型团队。
- 生态持续演进，开发者可按项目需求选择合适抽象层级以加速构建。

参考链接

- LangChain: <https://www.langchain.com/>
 - LangGraph: <https://www.langchain.com/langgraph>
 - Google ADK: <https://google.github.io/adk-docs/>
 - CrewAI: <https://docs.crewai.com/en/introduction>
-

附录 D：使用 AgentSpace 构建智能体（在线内容） (Building an Agent with AgentSpace, on-line only)

概述

- AgentSpace 为企业环境提供从数据/知识图谱到前端界面的端到端智能体构建与部署能力。
- 其目标是通过无代码/低代码方式，将自治推理、企业知识映射与工作流自动化嵌入现有流程。

核心能力

- 数据服务集成：接入私有数据存储、Google Knowledge Graph 或自建知识图谱。
- 提示配置：在可视化界面定义代理的角色、目标与运行参数（如 Fig.4 所示）。
- Web 暴露：提供 Web 接口将智能体对外服务，并配套使用分析（Analytics）。
- 多代理支持：在同一平台编排与管理多个协作代理，面向企业工作流场景。

构建流程（示意）

- 选择模板或空白代理，配置数据源与功能模块。
- 定义提示与约束，绑定外部工具与 API。
- 预览与测试，校准行为与质量标准。
- 部署到 Web/内部系统，开启使用分析与监控。

界面与体验

- 图形化界面简化复杂后端流程的使用门槛，使非专业开发者也能搭建专用代理。
- 最终可通过 AgentSpace 提供的聊天界面与代理交互（如 Fig.6）。

工程要点

- 版本与配置管理：记录数据源、提示与工具接入的版本，支持回滚。
- 质量与监控：集成评估与使用分析，保障稳定性与合规。
- 安全与权限：按最小权限原则配置外部系统访问，建立审计轨迹。

结论

- AgentSpace 以界面驱动的方式将智能体能力嵌入企业工作流，降低构建门槛并提升运营效率。
- 可通过在线实验课程（如 Google Cloud Skills Boost 的“Build a Gen AI Agent with Agentspace”）进行实践学习。

参考链接

- Create a no-code agent with Agent Designer: <https://cloud.google.com/agentspace/agentspace-enterprise/docs/agent-designer>
 - Google Cloud Skills Boost: <https://www.cloudskillsboost.google/>
-

附录 E：命令行中的 AI 智能体（AI Agents on the CLI）

概述

- 终端是文本化、可沙箱的理想环境，便于构建与评测 CLI 智能体。
- 生态涌现多款工具：Claude、Gemini、Aider、GitHub Copilot CLI 等，各有优势与适配场景。
- Terminal Bench 提供标准化任务集与基准评测，支持可扩展的容器化或直接集成。

能力与特点

- 代码改写与重构：直接编辑文件、生成补丁、维护代码风格与依赖。
- 指令编排与工具调用：运行命令、收集输出并据此迭代决策。
- Git 工作流协同：分支管理、提交建议、变更解释与审阅辅助。
- 多模态扩展：在支持的环境下引入图表/图像/文档解析能力。

工具生态（示例）

- Claude (Anthropic)：适合复杂架构与长链路推理，具备强代码理解能力。
- Gemini CLI (Google)：多模态与通用问题求解，集成 Google 生态能力。
- Aider：git 中心的代码编辑与补丁生成，专注代码变更与审查。
- GitHub Copilot CLI：与 GitHub 工作流深度融合，辅助搜索与命令建议。

评测与基准

- Terminal-Bench-Core-v0：覆盖科学流程、数据分析等 80 个手工任务。
- Terminus 基线代理：提供最小化对照，实现不同模型在统一环境下的可比性。
- 扩展性：支持容器化/直接连接、并行评估与引入既有基准，鼓励开源协作。

工程实践

- 安全沙箱：限制文件与网络访问范围，启用权限提示与速率控制。
- 可观测与回放：记录命令、输出与变更，支持失败重试与补偿。
- 集成测试：对关键脚本与工作流建立回归测试，保障长期稳定。
- 人类在环 (HITL)：在危险操作（删除/覆盖/生产环境）前强制确认。

结论

- CLI 智能体让终端成为动态、协作的开发环境；工具选择取决于需求与生态偏好。
- 熟练运用这些代理将成为核心技能，重塑软件开发的构建、调试与运维方式。

参考链接

- Claude CLI: <https://docs.anthropic.com/en/docs/claude-code/cli-reference>
- Gemini CLI: <https://github.com/google-gemini/gemini-cli>
- Aider: <https://aider.chat/>
- GitHub Copilot CLI: <https://docs.github.com/en/copilot/github-copilot-enterprise/copilot-cli>
- Terminal Bench: <https://www.tbench.ai/>

附录 F：引擎之下——智能体推理引擎的内部机制

(Under the Hood: An Inside Look at the Agents' Reasoning Engines)

概述

- 主流 LLM 的“推理”体现为分阶段、结构化的处理过程：理解任务→检索/综合→规划→评估→输出。
- 这种链式过程为智能体提供“中心规划器”，将高层目标分解为离散、可执行的动作序列。
- 需要注意：LLM 的“理解”是基于统计模式的模拟，并非具有人类式的内部表征与意识。

典型阶段

- 任务解构：识别核心目标、意图与约束；明确输入/输出格式与验收标准。
- 信息检索与综合：利用模式识别与外部检索（RAG）聚合相关事实与证据。
- 策略制定与分解：将问题拆解为步骤，形成可执行的计划（结合优先级与资源约束）。
- 评估与自检：反思与校验中间结果，必要时进行回溯与替代路径探索。
- 结构化输出：生成工具可消费的结果（JSON/表格/代码），支持后续自动化。

与智能体的耦合方式

- 作为核心规划器：驱动动作选择与工具路由，维持任务进度与上下文一致性。
- 与工具/环境集成：通过函数/工具调用将计划映射为真实操作与反馈循环。
- 轨迹记录：沉淀思考与行动链路，支持复盘、评估与持续优化。

局限与认知说明

- 统计模拟：输出基于模式拟合与条件概率，并不具有人类式“内在理解”。
- 幻觉与不确定性：需配合检索、验证与 Guardrails 降低错误与风险。
- 透明性：通过显式思路（CoT）与过程记录提升可解释与可审计性。

工程要点

- 程序化结构化：以模板/协议约束输入输出，便于自动化测试与回放。
- 评估闭环：LLM-as-a-Judge、轨迹级评估与 SLO/告警，持续提升可靠性。
- 多代理协作：在角色分工与消息协议（A2A/MCP）下，强化团队推理能力。

结论

- LLM 不仅是文本生成器，更是下一代智能系统的“认知内核”。
 - 提升这种“模拟推理”的可靠性与可治理性，是打造更可信的智能体的关键路径。
-

附录 G：代码型智能体（Coding Agents）

概述

- 代码型智能体通过角色分工与可审计 workflow，辅助开发者完成从读码、变更到评审、测试的全链路开发。
- 人类开发者的角色被提升为“架构与决策的总控”，AI 代理承担重复与可结构化的战术执行。

典型角色与分工

- 构建者（Builder）：生成/修改代码、补丁与文档，遵循项目风格与依赖约束。
- 评审者（Reviewer）：进行代码审阅与“反思（Reflection）”，对问题与改进点提出具体建议。
- 测试员（Tester）：补充或修复测试用例，运行回归并收集覆盖与失败报告。
- 运维助理（Ops Assistant）：编排构建/部署脚本，检查性能与稳定性指标。

能力与 workflow

- 读码与语境构建：解析项目结构、依赖与约定，形成变更上下文。
- 变更生成与对比：以补丁形式输出改动，附变更理由与可能影响面。
- 迭代对话：基于人类反馈进行修正与重试；评审输出是协作的起点而非终点。
- 合并与发布：在满足验收标准与测试通过后，进入合并与发布流程。

质量与安全

- 人类最终把关：开发者是质量门，需对代理输出进行验证、挑战与批准。
- 风险控制：对破坏性操作（删除/覆盖/生产改动）设硬性确认与回滚预案。
- 透明与审计：记录每次变更的理由、证据与轨迹，支持合规与事故复盘。

评估与度量

- 生产率提升：统计代理参与的变更比例与交付速度提升。
- 质量指标：测试通过率、缺陷密度、审查意见采纳率与回归稳定性。
- 协同效率：人机迭代次数、响应时延、冲突解决耗时。

行业趋势

- 大型企业报告显示，AI 生成代码占比显著提升（>30%），协助开发进入“增强式开发”时代。
- 人机协作重塑开发范式：自动化常规任务，释放人类精力用于战略性创新与韧性架构。

结论

- 代码型智能体不是替代者，而是“倍增器”：通过专业分工与审计化流程提升质量与速度。
- 人与 AI 的关系从对抗转为共创，作为一支整合团队交付面向用户的卓越产品。

参考链接

- AI 生成代码占比 (Google) : https://www.reddit.com/r/singularity/comments/1k7rxo0/ai_is_now_writing_well_over_30_of_the_code_at/
 - AI 生成代码占比 (Microsoft) : <https://www.businesstoday.in/tech-today/news/story/30-of-microsofts-code-is-now-ai-generated-says-ceo-satya-nadella-474167-2025-04-30>
-

术语表 (Glossary, 中文版)

- 目的：统一中文术语与英文对应，确保跨章节语义一致，便于检索与协作。
- 约定：术语以中文为主，括号内为英文原文；指向相关章节仅做参考，详见《术语索引》。

基础概念

- 智能体 (Agent)：能感知环境、进行推理并自主采取行动以达成目标的系统。强调“闭环”能力（感知→决策→行动→反馈）。
- 大型语言模型 (LLM)：基于概率的序列预测模型，作为智能体的“计算性心智”，负责理解、生成与策略推理。
- 提示 (Prompt)：向模型提供的指令/上下文。包含系统提示、用户提示、助手提示三类角色语境。
- 提示链 (Prompt Chaining)：将复杂任务拆解为可组合的多步提示序列，逐步推进形成稳定流程。
- Token / 上下文窗口 (Context Window)：模型一次可处理的文本片段容量与单位，影响信息保真与成本。
- 温度/采样 (Temperature/Top-p/Top-k)：影响生成多样性与确定性的采样参数；需要与任务目标匹配。

推理与提示技术

- 链式思维 (Chain of Thought, CoT)：在回答前显式生成逐步推理路径，提高复杂推理任务的稳定性与可解释性。(参见：第17章、附录A)
- 自一致性 (Self-Consistency)：通过多次独立推理并投票择优，降低随机性导致的误差。(参见：第17章)
- 树式思维 (Tree of Thoughts, ToT)：并行探索多条推理分支，评估并选择最优路径，适合复杂搜索空间。(参见：第17章、附录A)
- 图式思维 (Graph of Thoughts, GoT)：以图结构组织与合并多线推理结果，支持跨路径信息汇聚。(参见：第17章)
- 程序式思维 (Program of Thoughts, PoT)：将推理结构化为可执行程序或伪代码，强调可验证与可重用。(参见：第17章)
- 退一步提示 (Step-back Prompting)：先抽象再具体，先获取更高层次的原则或结构，再回到具体问题。(参见：附录A)
- ReAct (Reason + Act)：在“思考—行动—观察”的循环中结合工具调用与推理更新，形成稳健的任务求解。(参见：第5章、附录A)
- PAL (Program-Aided Language)：由程序辅助语言模型进行计算/检索，提升可靠性与可重复性。(参见：第5章、附录A)
- 规划 (Planning)：将目标拆解为可执行的子任务序列，处理依赖与资源约束，并支持重规划。(参见：第6章)
- 反思 (Reflection)：对中间输出进行自我评审与修订，提高质量与一致性，常与“自评-重写”循环结合。(参见：第4章)
- 深度研究 (Deep Research)：迭代检索、综合与追问，构建主题的系统性理解而非一次性查询。(参见：附录A)

记忆与检索 (RAG)

- 嵌入 (Embedding)：将文本/对象映射到向量空间以支持相似性检索与重排序。(参见：第14章)
- 向量库 (Vector Store)：存储与检索嵌入向量的系统，支撑语义级检索与召回。(参见：第14章)
- 索引/分块 (Indexing/Chunking)：构建检索索引与合理切分文档，兼顾召回率与上下文连贯性。(参见：第14章)
- 查询重写 (Query Rewriting)：对用户查询进行澄清与结构化，以提升检索精准度与相关性。(参见：第14章)
- 重排序与融合 (Re-ranking/Fusion)：在多路召回上进行排序与结果融合，提升整体质量。(参见：第14章)

- 摘要与记忆 (Summarization/Memory): 将经验/会话压缩为可检索的长期记忆, 支持知识演化。(参见: 第8章、第14章)

工具与集成

- 工具使用 (Tool Use): 通过结构化接口调用外部 API/数据库/执行环境, 形成闭环。(参见: 第5章)
- 函数调用 (Function Calling): 以模式 (schema) 驱动参数填充与调用, 提升安全性与确定性。(参见: 第5章)
- 幂等性与错误处理 (Idempotency & Error Handling): 确保重复调用安全可控, 并具备重试/补偿机制。(参见: 第5章、第12章)
- 模型上下文协议 (Model Context Protocol, MCP): 工具/资源/上下文的标准化接入层, 供应商中立。(参见: 第10章)
- 终端/浏览器智能体 (CLI/Browser Agents): 在命令行或浏览器 DOM 环境中执行复杂工作流。(参见: 附录B、附录E)

协作与通信 (A2A)

- 多智能体协作 (Multi-Agent): 角色分工、协调与冲突解决机制, 提升系统规模化能力。(参见: 第7章)
- 监督-工作者 (Supervisor-Worker): 由监督者分配任务、检查质量, 工作者执行具体子任务。(参见: 第7章)
- 黑板模型 (Blackboard): 共享工作空间用于信息发布/订阅与协作。(参见: 第7章)
- 专家团队 (Team-of-Experts): 按专长路由请求与投票聚合, 提高准确性与稳健性。(参见: 第7章)
- 智能体间通信 (Agent-to-Agent, A2A): 消息模式与协议设计, 支持异步/同步对话与协调。(参见: 第15章)

安全与治理 (Guardrails)

- 护栏 (Guardrails): 内容策略、行为约束与工具安全边界的综合治理机制。(参见: 第18章)
- 提示注入 (Prompt Injection): 通过恶意指令操纵模型行为的攻击方式, 需要输入/上下文隔离与校验。(参见: 第18章)
- 数据外泄 (Data Exfiltration): 模型或工具不当泄露敏感数据的风险, 需要访问控制与审计。(参见: 第18章)
- 人类在环 (Human-in-the-Loop, HitL): 在关键环节引入人工审批/复核, 保障质量与合规。(参见: 第13章)
- 安全测试 (Red Teaming): 对系统进行对抗性测试与弱点挖掘。(参见: 第18章)

评估与监控

- 评估 (Evaluation): 基准、指标与任务级测评, 用于质量保证与迭代改进。(参见: 第19章)
- LLM 评审 (LLM-as-a-Judge): 以模型进行自动化打分与评注, 需配合抽样与人工验证。(参见: 第19章)
- 遥测与可观测性 (Telemetry/Observability): 采集日志、追踪与事件, 支持问题定位与优化。(参见: 第11章、第19章)
- 漂移检测 (Drift Detection): 识别数据/行为分布变化, 触发再训练或策略更新。(参见: 第19章)

资源与优化

- 预算与配额 (Budget/Quota): 控制成本/时延/Token 等资源, 进行多目标权衡。(参见: 第16章、第20章)

- 缓存与降级 (Caching/Graceful Degradation)：在资源紧张或失败时保持系统可用与体验稳定。(参见：第16章)
- 吞吐与并发 (Throughput/Concurrency)：在并行化与调度中平衡速度与稳定性。(参见：第3章、第16章)
- 优先级与调度 (Prioritization/Scheduling)：根据价值/风险/时效对任务排序与分配。(参见：第20章)
- 探索-利用 (Exploration-Exploitation)：在不确定环境中平衡试探与收益最大化。(参见：第21章)
- 多臂老虎机 (Multi-Armed Bandit)、汤普森采样 (Thompson Sampling)、UCB：典型的在线决策与优化算法。(参见：第21章)

生态与工具

- LangChain / LangGraph：以链/图组织工作流与状态，适合复杂编排。(参见：引言、各章)
- LlamaIndex：面向检索增强的索引与路由能力。(参见：第14章、附录C)
- CrewAI：团队化多智能体编排与角色分工。(参见：第7章、附录C)
- Google ADK (Agents Development Kit)：面向生产级智能体开发的工具链与运行时。(参见：附录C)
- DSPy：将提示工程参数化/程序化，提升可维护性与可迁移性。(参见：附录A、附录C)
- Terminal-Bench-Core-v0：命令行智能体能力的评测基准。(参见：附录E)
- Aider / Copilot CLI：典型的命令行编程助手与工作流工具。(参见：附录E、附录G)
- Project Astra：向真实世界扩展的多模态感知与行动系统示例。(参见：附录B)

注：本术语表为中文整编版本；如需定位出处与延伸阅读，请参考《术语索引 (Index of Terms, 中文版)》。

术语索引 (Index of Terms, 中文版)

- 用法：按字母顺序检索关键术语，快速跳转到涉及该术语的中文译文章节/附录。
- 说明：链接为本地相对路径，指向《Agentic Design Patterns (ZH)》目录下的对应 Markdown 文件。

A

- Agent (智能体)： [Introduction.md](#)、Chapter 7_ Multi-Agent Collaboration.md、Appendix B - AI Agentic Interactions_ From GUI to Real world environment.md
- Agent-to-Agent (A2A, 智能体间通信)： Chapter 15_ Inter-Agent Communication (A2A).md
- ADK (Google Agents Development Kit)： Appendix C - Quick overview of Agentic Frameworks.md
- Aider (命令行编程助手)： Appendix E - AI Agents on the CLI.md、Appendix G - Coding agents.md

C

- Chain of Thought (CoT, 链式思维)： Chapter 17_ Reasoning Techniques.md、Appendix A_ Advanced Prompting Techniques.md
- CrewAI (多智能体编排)： Appendix C - Quick overview of Agentic Frameworks.md、Chapter 7_ Multi-Agent Collaboration.md
- Critique Model (评审模型)： Chapter 19_ Evaluation and Monitoring.md、Appendix A_ Advanced Prompting Techniques.md
- Copilot CLI (命令行助手)： Appendix E - AI Agents on the CLI.md

D

- DSPy (提示参数化与程序化)： Appendix A_ Advanced Prompting Techniques.md、Appendix C - Quick overview of Agentic Frameworks.md
- Deep Research (深度研究)： Appendix A_ Advanced Prompting Techniques.md
- Drift Detection (漂移检测)： Chapter 19_ Evaluation and Monitoring.md

E

- Embedding (嵌入)： Chapter 14_ Knowledge Retrieval (RAG).md
- Evaluation (评估)： Chapter 19_ Evaluation and Monitoring.md
- Exploration-Exploitation (探索-利用)： Chapter 21_ Exploration and Discovery.md

F

- Function Calling (函数调用): Chapter 5_ Tool Use.md、Appendix A_ Advanced Prompting Techniques.md
- Fusion (融合检索): Chapter 14_ Knowledge Retrieval (RAG).md

G

- Guardrails (护栏与安全): Chapter 18_ Guardrails_Safety Patterns.md
- Graph of Thoughts (GoT, 图式思维): Chapter 17_ Reasoning Techniques.md

H

- Human-in-the-Loop (人类在环, HitL): Chapter 13_ Human-in-the-Loop.md

K

- Knowledge Retrieval (RAG, 检索增强生成): Chapter 14_ Knowledge Retrieval (RAG).md

L

- LangChain / LangGraph: [Introduction.md](#)、Appendix C - Quick overview of Agentic Frameworks.md
- LlamaIndex: Appendix C - Quick overview of Agentic Frameworks.md、Chapter 14_ Knowledge Retrieval (RAG).md
- LLM-as-a-Judge (模型评审): Chapter 19_ Evaluation and Monitoring.md

M

- MCP (Model Context Protocol, 模型上下文协议): Chapter 10_ Model Context Protocol (MCP).md
- Memory (记忆管理): Chapter 8_ Memory Management.md
- Multi-Agent (多智能体): Chapter 7_ Multi-Agent Collaboration.md
- Multi-Armed Bandit (多臂老虎机): Chapter 21_ Exploration and Discovery.md

P

- Planning (计划): Chapter 6_ Planning.md
- Prompt (提示): [Introduction.md](#)、Appendix A_ Advanced Prompting Techniques.md
- Prompt Chaining (提示链): Chapter 1_ Prompt Chaining.md
- Prioritization (优先级): Chapter 20_ Prioritization.md
- Program of Thoughts (PoT, 程序式思维): Chapter 17_ Reasoning Techniques.md

R

- ReAct (Reason + Act): Chapter 5_ Tool Use.md、Appendix A_ Advanced Prompting Techniques.md
- Reflection (反思): Chapter 4_ Reflection.md
- Re-ranking (重排序): Chapter 14_ Knowledge Retrieval (RAG).md
- Resource-Aware Optimization (资源感知优化): Chapter 16_ Resource-Aware Optimization.md

S

- Scheduling (调度): Chapter 20_ Prioritization.md、Chapter 3_ Parallelization.md
- Self-Consistency (自一致性): Chapter 17_ Reasoning Techniques.md
- Step-back Prompting (退一步提示): Appendix A_ Advanced Prompting Techniques.md
- Summarization (摘要): Chapter 8_ Memory Management.md、Chapter 14_ Knowledge Retrieval (RAG).md

T

- Terminal-Bench-Core-v0 (终端评测基准): Appendix E - AI Agents on the CLI.md
- Thompson Sampling (汤普森采样): Chapter 21_ Exploration and Discovery.md
- Tool Use (工具使用): Chapter 5_ Tool Use.md
- Tree of Thoughts (ToT, 树式思维): Chapter 17_ Reasoning Techniques.md、Appendix A_ Advanced Prompting Techniques.md

U

- UCB (上置信界): Chapter 21_ Exploration and Discovery.md

V

- Vector Store (向量库): Chapter 14_ Knowledge Retrieval (RAG).md

其他

- Browser Automation (浏览器自动化/DOM): Appendix B - AI Agentic Interactions_ From GUI to Real world environment.md
- CLI Agents (命令行智能体): Appendix E - AI Agents on the CLI.md
- Coding Agents (编程型智能体): Appendix G - Coding agents.md
- AgentSpace (构建与运行时平台): Appendix D - Building an Agent with AgentSpace (on-line only).md

- Under the Hood（推理引擎内部机制）：Appendix F - Under the Hood_ An Inside Look at the Agents' Reasoning Engines.md
-

注：索引不等同于完整定义，术语解释请参阅《Glossary（术语表，中文版）》；若存在路径差异，请以工作区中文译文文件实际命名为准。

结语 (Conclusion)

贯穿全书，我们从 Agentic AI 的基础概念出发，逐步走向能够实际落地的复杂自治系统。我们以这样的前提开始：构建智能体，就像在技术画布上完成一件复杂的艺术品——不仅需要强大的认知引擎（如大型语言模型），还需要一套稳健的架构蓝图。这些蓝图，即 Agentic 设计模式，为把简单的、被动的模型，转化为主动、面向目标、能够进行复杂推理与行动的实体，提供了结构与可靠性。

本结语将对全文的核心原则进行综合与提炼：我们首先回顾关键的 Agentic 模式，并将它们按能力框架进行归纳，从而强调其整体重要性；随后，我们展示这些模式如何以“组合”的方式构成更复杂的系统，形成强大的协同效应；最后，我们展望智能体开发的未来，讨论将塑造下一代智能系统的趋势与挑战。

回顾关键的 Agentic 原则

本书详述的 21 个模式，构成了智能体开发的一套完整工具箱。虽然每个模式都针对特定的设计挑战，但我们可以按“智能体核心能力”的维度进行归类与理解：

- **核心执行与任务分解**：在最基础层面，智能体必须能够执行任务。Prompt Chaining（提示链）、Routing（路由）、Parallelization（并行化）与 Planning（规划）构成了智能体行动能力的基石。提示链通过把问题拆解为线性、离散的步骤，使每次操作的输出能够逻辑性地引导下一步；当工作流需要更动态的行为时，路由模式提供条件逻辑，使智能体能够根据上下文选择最合适的路径或工具；并行化通过并发执行相互独立的子任务来提升效率；规划模式则把智能体从“执行者”提升为“策略家”，能够为高层目标制定多步骤计划。
- **与外部环境交互**：智能体的实用性在很大程度上取决于其与外界交互的能力。Tool Use（工具/函数调用）在此尤为关键，它使智能体能够调用外部 API、数据库与软件系统，从而让操作“落地”到真实世界的的数据与能力。为了有效使用这些工具，智能体往往需要从海量信息中检索特定且相关的内容。Knowledge Retrieval（知识检索，尤其是 RAG）通过查询知识库并把结果融入生成，使输出更准确、更加具备上下文意识。
- **状态、学习与自我改进**：若要超越单轮任务，智能体必须具备维持上下文与持续改进的能力。Memory Management（记忆管理）为智能体提供短期会话上下文与长期知识留存；更进一步，真正智能的体系系统具备自我改进的能力。Reflection（反思）与 Self-Correction（自我纠错）使智能体能够自我批评输出、识别错误或不足，并迭代打磨结果；Learning and Adaptation（学习与适应）则进一步让智能体根据反馈与经验演化行为，使其随时间变得更有效。
- **协作与通信**：许多复杂问题最适合通过协作来解决。Multi-Agent Collaboration（多智能体协作）使我们能够构建由多个具备专长的智能体协同工作的系统，每个智能体都有明确的角色与能力分工。这种劳动分工使系统能够处理单个智能体难以解决的多面性问题。为了让协作高效，清晰且标准化的通信至关重要，Inter-Agent Communication（智能体间通信，A2A）与 Model Context Protocol（MCP）致力于规范智能体与工具交换信息的方式。

当这些原则通过各自的模式落地，它们就为构建智能系统提供了稳健的框架。它们引导开发者打造的不仅是“能做复杂事”的智能体，更是结构化、可靠、可适应的系统。

将模式组合成复杂系统

Agentic 设计的真正力量，并非源于单一模式的孤立应用，而是源于将多个模式“艺术性”地组合起来，构成复杂的多层系统。我们的画布很少被一个简单工作流填满，它更常是一幅由多种模式交织而成的“织锦”，共同协作以达成复杂目标。

以“自治 AI 研究助理”为例，这一任务需要规划、检索、分析与综合等能力的组合：

- **初始规划**：当收到用户请求，例如“分析量子计算对网络安全格局的影响”，首先由 Planner（规划者）智能体接收。它使用规划模式把高层请求拆解为结构化的多步骤研究计划，例如“识别量子计算的基础概念”“研究常见密码算法”“查找专家对量子威胁的分析”“将发现综合成结构化报告”等。

- 使用工具的信息收集：为执行该计划，系统将大量依赖工具调用模式。计划的每一步都可能触发对搜索引擎（如 Google）或企业搜索（如 vertex_ai_search）的调用；对于结构化数据，可能使用工具查询学术数据库（如 ArXiv）或金融数据 API。
- 协作分析与写作：一个智能体也许能完成这一流程，但更稳健的架构会采用多智能体协作：由“研究员”智能体负责执行检索计划并汇总原始信息（摘要与来源链接），随后把结果交给“写作”智能体；写作智能体以初始计划为大纲，把收集到的信息综合成连贯草稿。
- 迭代反思与改进：初稿鲜有完美。我们可以引入第三个“批评者”智能体来实施反思模式，专门检查写作草稿的逻辑一致性、事实准确性或表达清晰度。它的批评将反馈给写作智能体，后者依据自我纠错模式进行修订，融入反馈以产出质量更高的最终报告。
- 状态管理：在整个过程中，记忆管理至关重要。它维护研究计划的状态、保存研究员收集的信息、存储写作智能体的草稿、追踪批评者的反馈，从而保证多步骤、多智能体工作流的上下文连贯。

在这个例子中，至少有五个不同的 Agentic 模式被编织在一起：规划提供高层结构，工具调用让操作落地到真实数据，多智能体协作实现专长分工，反思确保质量，记忆管理维持整体一致性。这种“组合”把一组单独的能力，转化为能够完成复杂任务的自治系统——远非一个提示或简单链路所能及。

展望未来

如同 AI 研究助理的例子所示，把 Agentic 模式组合成复杂系统，并不是故事的终章，而是软件开发新篇章的序幕。展望前路，若干涌现的趋势与挑战将定义下一代智能系统，推动能力边界，也对其构建者提出更高的要求。

向更强自治与推理能力迈进，是 Agentic AI 发展的主旋律。本文所述模式为目标导向行为提供了脚手架，但未来的智能体需要能够在不确定性中航行，执行抽象与因果推理，甚至展现某种常识。这很可能需要与新的模型架构、更紧密的整合，以及将 LLM 的模式匹配优势与经典 AI 的逻辑严谨相融合的“神经-符号”方法。我们将看到系统从“human-in-the-loop（人参与）”向“human-on-the-loop（人监督）”的转变——智能体被信任去执行复杂、长时任务，只有在目标完成或关键异常发生时才回报。

与此同时，智能体生态与标准化将崛起。多智能体协作模式凸显了专长分工的力量；未来我们将见证开放市场与平台的出现，开发者可以部署、发现并编排“智能体即服务”的车队。为实现这一愿景，MCP（模型上下文协议）与 A2A（智能体间通信）背后的原则将愈发重要，推动行业就“智能体、工具与模型如何交换数据、上下文、目标与能力”形成统一标准。

一个正在成长的生态例证是 GitHub 上的“Awesome Agents”仓库，它以精选清单的形式汇聚了开源的 AI 智能体、框架与工具，按应用场景（从软件开发到自治研究与对话式 AI）组织前沿项目，体现出该领域的快速创新节奏。

当然，这条道路并不平坦。随着智能体愈发自治与互联，安全、对齐与鲁棒性将更加关键。如何确保智能体的学习与适应不会偏离初衷？如何构建能抵御对抗性攻击与不可预期真实环境的系统？回答这些问题，需要一套新的“安全模式”，以及以测试、验证与伦理对齐为核心的工程纪律。

最后的思考

在本指南中，我们把智能体的构建比作在技术画布上的艺术实践。Agentic 设计模式是你的颜料与笔触——它们让你走出“只写提示”的初级阶段，创造出动态、响应、面向目标的实体；它们提供必要的架构纪律，使得大型语言模型的原始认知能力，得以被塑造成可靠且有目的性的系统。

真正的技艺不在于精通某一个模式，而在于理解它们的“合奏”——把画布作为整体去看待，构建一个规划、工具调用、反思与协作相互协调的系统。Agentic 设计的原则是一种新“创作语言”的语法，它使我们能够指示机器不仅“做什么”，还“如何成为怎样的存在”。

Agentic AI 是当下技术领域最令人振奋且迅速演化的方向之一。本文所述的概念与模式，并非终极、静态的教条，而是起点——一个坚实的基础，供你继续构建、试验与创新。未来并非仅仅由我们作为 AI 的使用者来定义，

而是由我们作为智能系统的架构师共同打造，它们将与我们一道解决世界上最复杂的问题。画布已在你眼前，模式已在你手中。是时候开始创作了。